

XOI

Redes de Ordenadores e Internet

104353 - Ingeniería de Datos - 1^{er} curso
Universitat Autònoma de Barcelona

GUÍA DE ESTUDIO



Miguel Hernández-Cabronero
<miguel.hernandez@uab.cat>

UAB



Este documento está disponible en: [Castellano](#), [Català](#), [English](#).

Este documento se compiló el 23 de septiembre de 2026.

Licencia y créditos

Licencia

Copyright 2024-* © Miguel Hernández-Cabronero <miguel.hernandez@uab.cat>.

Este documento y los materiales que lo acompañan pueden **distribuirse y modificarse libremente** para usos no comerciales, siempre que se cite a su(s) autor(es) y se mantengan las mismas condiciones de uso compartido que los originales (ver más abajo).

La versión más reciente de este documento y de los materiales que lo acompañan puede obtenerse en <https://github.com/miguelinux314/uab-xoi>.



Distribuido bajo la **Licencia Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0** (la "Licencia"). No puedes usar este archivo salvo de conformidad con la Licencia. Puedes obtener una copia de la Licencia en <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Salvo que lo exija la legislación aplicable o se acuerde por escrito, el software distribuido bajo la Licencia se distribuye "TAL CUAL", SIN GARANTÍAS NI CONDICIONES DE NINGÚN TIPO, ya sean expresas o implícitas. Consulta la Licencia para conocer el texto específico que rige los permisos y las limitaciones de la Licencia.

Créditos

La estructura y los contenidos de esta guía se basan en gran medida en los materiales de clase elaborados por Sergi Robles y otros profesores del departamento DEIC de la UAB. Muchos de los contenidos visuales fueron creados por artistas externos y publicados bajo licencias compatibles:

- [Mathias Legrand and Vel](#): plantilla base de $\text{\LaTeX}$.
- [TheDigitalArtist/pixabay](#): portada y cabecera del capítulo 6.
- [victorsteep/pixabay](#): cabecera del capítulo 1.
- [OpenClipart-Vectors/pixabay](#): cabecera del capítulo 2.
- [D-Kuru](#), [Niridya](#), [Joe Ravi](#), [Shaddack](#) / Wikimedia Commons (WC) : Ej. 2.6
- [Timwether](#), [Timewalk](#) / WC : Ej. 2.7.
- [deepai](#) : Iceberg de la sección 2.2.
- [Chetvorno](#) : Diagrama de la antena de la sección 2.2.
- [Berserkerus](#) : Diagrama de modulación de la sección 2.2.
- [Graham Rhind](#) : cabecera del capítulo 3.
- [mrcolo](#) : cabecera del capítulo 4.
- [Geek2003](#) : Switch (conmutador) de la sección 3.1.
- [ignartonosbg](#) : cabecera del capítulo 5.
- [Toffelginkgo](#) : cabecera del capítulo 7.
- [Doodles43](#) : cabecera del capítulo 8.
- [Bru-no](#) : Puertos del router (encaminador) de la sección 8.
- [PublicDomainPictures](#) : cabecera del capítulo 9.
- [joo213](#) : cabecera del capítulo 10.
- [Frank_Rietsch](#) : cabecera de la introducción a las misiones.
- [succo](#) : icono de la misión 1 (capítulo 11.1).
- [Pixabay](#) : icono de la misión 2 (capítulo 11.2).
- [mmisof](#) : icono de la misión 4 (capítulo 11.4).
- [DeepAI](#) : icono de la misión 5 (capítulo 11.5).
- [1zoom.me](#) : icono de la misión 6 (capítulo 11.6).
- [DeepAI](#) : icono de la misión 7 (capítulo 11.7).
- [StockSnap](#) : icono de la misión 9 (capítulo 11.9).
- [jarmoluk](#) : cabecera del índice de conceptos.

Uso de IA

Uso de IA recomendado

La IA es una herramienta excitante, también para el aprendizaje.

A día de hoy (curso 2026-27), las IAs comerciales dan **soluciones incorrectas** a preguntas, ejercicios y prácticas de esta asignatura con probabilidad no desdeñable.

Desaconsejamos completamente el uso de IA como fuente de soluciones.

Recomendamos estas herramientas para:

- Describir tu configuración de red.
- Generar ejemplos de uso de red.
- Explorar la documentación proporcionada.

Uso de IA en la creación de esta guía

En esta guía se ha usado IA para:

- **Creación de imágenes:** ilustraciones decorativas (no los diagramas con contenido).
- **Creación de código:** parte del software que genera la guía (sitio web y PDF).
- **Revisión:** ortografía y gramática del texto.



Índice

I	Redes de ordenadores	
2	El velo analógico-digital	8
2.1	Información ↔ Datos ↔ Mensajes binarios	8
2.2	Analógico ↔ Digital	10
3	Direccionamiento	12
3.1	Por favor, contacta conmigo	12
3.2	Necesito encontrarte	13
3.3	¿Cómo llego hasta ahí?	14
4	Paquetes	16
4.1	Tengo un paquete para ti	16
4.2	Por favor, rellena el formulario	17
4.3	No paran de llegar	19
5	Una pila de capas	20
5.1	Envío de datos	20
5.2	Recepción de datos	21
II	Internet	
6	Internet	25
6.1	La pila TCP/IP	25
6.2	Visión general de los protocolos TCP/IP	26
6.3	¿Quién manda en Internet?	27
7	Capa 2: comunicación en red (LAN)	28
7.1	Direccionamiento de capa 2	29
7.2	Protocolo Ethernet de capa 2	29
8	Capa 3: comunicación Internet(work)	32
8.1	Direccionamiento en la capa 3	33
8.2	IP – Internet Protocol	36
8.3	Encaminamiento IP	38
8.4	Subnetting IP	40
8.5	Fragmentación IP	42
8.6	ARP – Address Resolution Protocol	43
8.7	ICMP – Internet Control Message Protocol	45

9	Capa 4: Transporte	47
9.1	Direccionamiento de capa 4	48
9.2	UDP – User Datagram Protocol	49
9.3	TCP – Transport Control Protocol	50
9.4	TCP: “Conectando”	51
9.5	TCP: “Conexión establecida”	52
9.6	TCP: “Cerrando la conexión”	55
10	Capa 7: Aplicación	59
10.1	DNS – Domain Name System	59
10.2	DHCP – Dynamic Host Configuration Protocol	61
10.3	Sistemas autónomos	63

III

Misiones

11	Misiones	66
11.1	Misión 1: Compartir es vivir	67
11.2	Misión 2: Scroll infinito	68
11.3	Misión 3: Pastoreando el caos	69
11.4	Misión 4: Inter { planetary net	70
11.5	Misión 5: El dinero es la raíz	71
11.6	Misión 6: Molestar a dos planetas enteros	72
11.7	Misión 7: TCP o no TCP	73
11.8	Misión 8: ¿Me hablas a mí?	74
11.9	Misión 9: Pasillo interplanetario	75

1. Introducción: Redes de Ordenadores e Internet (X)

Lo que hemos pagado

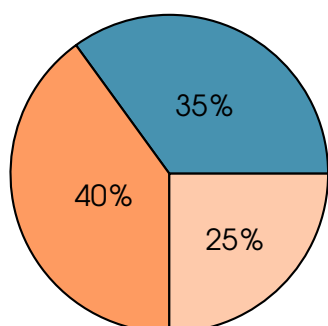
	Lunes	Martes	Miércoles	Jueves	Viernes
8:00					
8:30					
9:00				Problemas PAUL/811	Prácticas PLAB/812
9:30				Prácticas PLAB/813	Prácticas PLAB/814
10:00				Prácticas PLAB/815	Problemas PAUL/812
10:30				Prácticas PLAB/811	Problemas PAUL/813
11:00					
11:30					
12:00					
12:30					
13:00		Teoría Clase completa			
13:30					
14:00					
14:30					
15:00					

- 13x sesiones de **Teoría** de 2h
- 12x sesiones de **Problemas** de 1h
- 12x sesiones de **Laboratorio** de 1h
- Ilimitadas sesiones de **tutoría** (*tutories*), individuales o en grupo: envía un correo
- 90h **Trabajo autónomo** (tuyo)
- 3x **Controles**
- 1x **examen Final** + revisión
- 1x **Recuperación** del examen final + revisión
- 3x **entregas de laboratorio** con feedback
- 1x **examen de laboratorio** + revisión

Evaluación

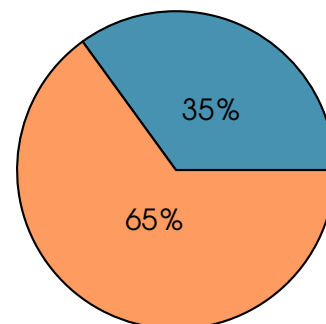
Para determinar tu nota se usará la mejor de las dos opciones siguientes:

Opción A



- *Prácticas
- *Máx(Final; Recuperación)
- Controles

Opción B



*: Se exige nota mínima en esta parte para aprobar la asignatura. Todos los detalles en la [guía docente de la asignatura](#).

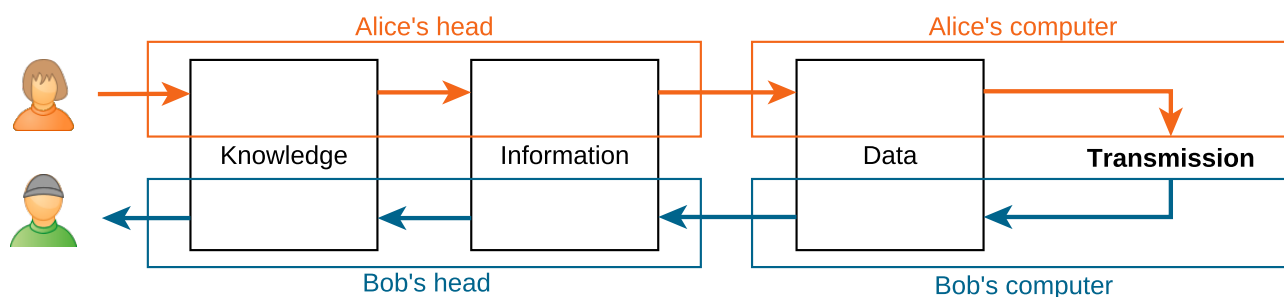
Redes de ordenadores

¿Cómo intercambian datos los ordenadores? ¿Cómo podemos localizar otras máquinas situadas en cualquier parte del mundo y comunicarnos con ellas? Esta parte de la guía explora algunos de los principales retos a los que nos enfrentamos si queremos interconectar dispositivos a escala planetaria, y más allá.

2	El velo analógico-digital	8
2.1	Información ↔ Datos ↔ Mensajes binarios	
2.2	Analógico ↔ Digital	
3	Direccionamiento	12
3.1	Por favor, contacta conmigo	
3.2	Necesito encontrarte	
3.3	¿Cómo llego hasta ahí?	
4	Paquetes	16
4.1	Tengo un paquete para ti	
4.2	Por favor, rellena el formulario	
4.3	No paran de llegar	
5	Una pila de capas	20
5.1	Envío de datos	
5.2	Recepción de datos	

2. El velo analógico-digital

Enviamos y recibimos mensajes constantemente en nuestras esferas personal, profesional y política. La disponibilidad casi instantánea de prácticamente cualquier información que deseemos es una característica clave de la era actual. Cuando usamos un ordenador de cualquier tipo para compartir u obtener conocimiento, primero hay que transformarlo en datos que puedan entender tanto los humanos como los ordenadores. Podemos imaginarlo como una secuencia de pasos:



Los ordenadores trabajan con datos digitales y transmiten secuencias de símbolos binarios (les solemos llamar 0 y 1). Sin embargo, la transmisión sucede en el mundo físico donde 0 y 1 son ideas y no objetos tangibles.

Este capítulo estudia cómo esos símbolos digitales *atraviesan el velo* del mundo digital al analógico (físico) en la transmisión, y de vuelta del analógico al digital en la recepción.

2.1 Información ↔ Datos ↔ Mensajes binarios

Conocimiento, información y datos no son lo mismo, aunque a menudo se usen indistintamente. En contextos técnicos, deben usarse e interpretarse con precisión.

Ejercicio 2.1 Supón que estamos al aire libre y queremos contarle a un amigo qué tiempo hace. Tenemos conocimiento sobre todo lo relacionado con el tiempo, porque estamos allí. Para comunicarnos, necesitamos centrarnos en una parte de ese conocimiento y decidir qué información enviar. Por ejemplo, podríamos querer comunicar la temperatura y decir algo como “la temperatura es de unos 22°C”. Hay muchas formas de almacenar esa información como datos dentro de un ordenador. Lo más probable es que representemos el número 22 como parte de esos datos. ¿Cómo definirías conocimiento, información y datos?

Aunque hay muchos tipos de dato (numéricos, textuales, visuales, científicos, *etc.*), todos se representan mediante números. Por ejemplo, se podría usar la codificación ASCII para representar la cadena “Bobby” como la siguiente secuencia: 66,111,98,98,121.

Dentro de un ordenador, esos números se almacenan en registros binarios, p. ej. en la CPU y en la RAM. Hay múltiples formas de representar datos numéricos en formato binario. En concreto, debemos prestar mucha atención como mínimo a los siguientes aspectos siempre que leamos o escribamos datos:

- ¿entero o con decimales?
- ¿profundidad de bits? (p. ej. 8 bits por número)
- ¿con signo o sin signo?
- Para números de varios bytes: ¿big endian o little endian?
- Para números con decimales: ¿coma flotante o coma fija?

Ejercicio 2.2 Explica cómo completar la segunda fila a partir de la primera (y viceversa), y qué suposiciones necesitas hacer en cada caso.

Decimal	7	2	-3	0.75	260
Binario	00111	00000010	1101	1100	0000 0100 0000 0001

¿Y qué hay de $\rightarrow 1100\ 0011\ 1010\ 0000$?

A veces necesitamos inspeccionar el contenido de un registro binario (p. ej. para depuración u otros fines de análisis) o establecer ese contenido manualmente. Para los humanos, manejar cadenas binarias de más de unos pocos bits es incómodo y *muy* propenso a errores. Cuando necesitamos inspeccionar o modificar el contenido de un registro binario (p. ej. para depuración), a menudo usamos la base 16, es decir, la notación hexadecimal.

En hexadecimal hay exactamente $16 = 2^4$ dígitos distintos (0 a 9, a a f) con valores decimales de 0 a 15, cada uno de los cuales representa exactamente 4 bits. Cuando escribimos valores hexadecimales para un ordenador, es decir, en código fuente, los valores hexadecimales suelen ir precedidos de `0x`, p. ej. `0x58a5b0`. Del mismo modo, las expresiones binarias van precedidas de `0b`, p. ej. `0b0011`.



- Los espacios, e incluso los saltos de línea, entre dígitos no tienen ningún significado; solo se usan para facilitar la inspección visual.
- En algunos textos, cuando en un contexto pueden aplicarse varias bases, estas se indican como subíndices, como en `58a5b016` y `00112`.

Ejercicio 2.3 Considera el siguiente mensaje, compuesto por una concatenación de enteros sin signo de 3 bits: `a4 3f 20`.

- ¿Cuántos bits y bytes de longitud tiene?
- ¿Cuáles son los cinco primeros enteros contenidos en el mensaje?

La mayoría de las veces dejaremos que nuestro código se encargue de manipular los datos. Para ello, necesitamos operaciones a nivel de bits y de lógica booleana como las siguientes

Operación	AND (bits)	AND lógico	OR (bits)	OR lógico	Despl. izq.	Despl. der.
Python	<code>&</code>	<code>and</code>	<code> </code>	<code>or</code>	<code><<</code>	<code>>></code>

Otras herramientas útiles de python son las funciones `bin()` y `hex()`, que convierten un entero en su representación binaria y hexadecimal, respectivamente, e `int()`, que puede analizar una cadena que describe un número y devolver ese número. También podemos controlar el formato con el que mostramos los números; p. ej. `print(f'{n:08b}')` imprimiría el valor de la variable `n` en forma binaria, usando al menos 8 posiciones y rellenando con ceros las posiciones iniciales vacías (más información en la [documentación de python](#)).

Ejercicio 2.4 Considera el código que se muestra a continuación. Los números que imprime al ejecutarse tienen una importancia especial en redes y en informática. ¿Qué tienen en común? (Pista: quizá quieras modificar el código para mostrar sus representaciones binarias).

```

.py .out snippets/bitwisemanipulation.py
1 a = 0xff
2 print(a)
3 for _ in range(8):
4     a = (a << 1) & 0xff
5     print(a)

```

2.2 Analógico ↔ Digital

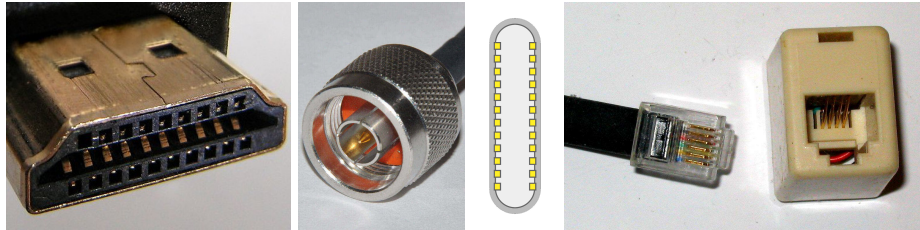
Una vez decidimos qué bits transmitir, necesitamos encontrar una forma de hacer que esos bits lleguen a otra máquina a distancia. Primero, tenemos que decidir qué medio usar y qué propiedades físicas podemos controlar y monitorizar. Entre las opciones más populares están usar el voltaje en cables de cobre, enviar luz por fibra óptica y manipular el campo electromagnético mediante antenas.

Ejercicio 2.5 Reflexiona:

- ¿Podemos hacer un cable de cobre tan largo como queramos?
- ¿Qué medio usan las conexiones Wi-Fi?
- ¿Necesitamos fibras ópticas para realizar comunicaciones basadas en la luz?

Si optamos por cables de cobre, podemos poner varios en paralelo y enviar más datos a la misma velocidad de reloj. Esto, sin embargo, tiene el precio de una complejidad y un coste mayores que los diseños en serie. Pueden producirse interferencias de voltaje en las líneas de datos por diversos motivos, incluidos fenómenos físicos como la inducción electromagnética en el cable. No obstante, los cables que cumplen los estándares modernos de transmisión de datos (p. ej. IEEE 802.3) suelen emplear pares trenzados de cables y apantallamiento, entre otras estrategias, para garantizar errores de bit por debajo de 1 de cada 10^{12} bits.

Ejercicio 2.6 Observa e identifica los siguientes conectores de cable:

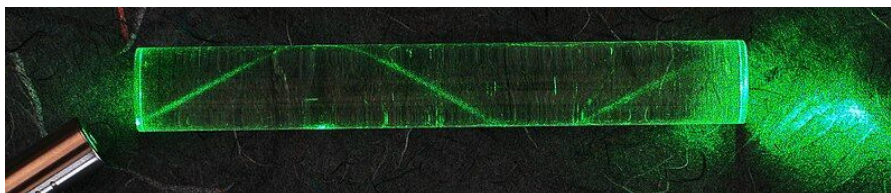


¿Alguna vez has buceado en una piscina, has mirado hacia arriba y el agua parecía un espejo? Los cables de fibra óptica funcionan gracias a este fenómeno, llamado refracción. La luz que entra en la fibra por un extremo permanece dentro hasta que sale por el otro. El recubrimiento de la fibra es importante para preservar su integridad, pero no participa en el “rebote” de la luz mientras avanza por la fibra.

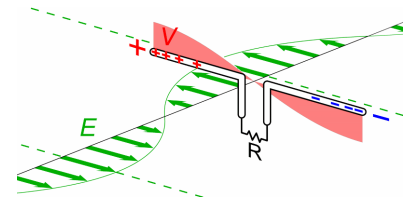
En una fibra óptica monomodo, el sensor del extremo receptor detecta la presencia o ausencia de un único rango de longitudes de onda. Estos cables son relativamente finos y baratos, pero transportan una sola señal. Otra opción es transmitir varias longitudes de onda (p. ej. usando varias fuentes láser) al mismo tiempo, por la misma fibra. En el extremo receptor, se usa un prisma para volver a dividir el haz de luz en sus componentes monocromáticos, que se detectan por separado.

De este modo, la fibra óptica multimodo permite multiplexar varias señales, lo que aumenta enormemente la velocidad efectiva de transmisión.

Ejercicio 2.7 Cuando un haz de luz entra en una fibra óptica, lo hace con un cierto ángulo de ataque. Este ángulo afecta al tiempo que tarda en llegar al otro extremo. Si proyectamos un cono de luz, la luz entra en la fibra con múltiples ángulos de ataque. ¿Qué ocurre en el otro extremo si encendemos y apagamos este cono de luz muy rápidamente? ¿Importa si usamos haces monomodo o multimodo?



Aunque hay muchas clases de antenas, su propósito principal es detectar campos electromagnéticos y/o crearlos. La ecuación de Maxwell-Faraday nos dice que los *cambios* en el *campo* eléctrico $\vec{E}$ crean un *campo* magnético perpendicular $\vec{B}$, y viceversa.



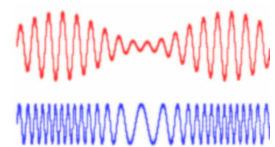
Una antena de dipolo como la de la derecha aprovecha este concepto aplicando un voltaje alterno V , que genera ondas electromagnéticas que se propagan hacia fuera desde el eje del dipolo.



Cuando emitimos una energía E desde un punto en todas las direcciones, esa energía se distribuye sobre una esfera creciente. Como el área de una esfera de radio r es $4\pi r^2$, la cantidad de energía que llega a un punto a distancia r será del orden de E/r^2 .

Esto se conoce a veces como la ley de la inversa del cuadrado, y es la razón por la que las corrientes inducidas en la antena del receptor suelen estar entre nanovoltios (10^{-9} V) y picovoltios (10^{-12} V). Sorprendentemente, esto basta para que los detectores modernos lean la señal de datos.

Independientemente del método elegido para que viajen los datos, sigue siendo necesario atravesar el velo entre las señales analógicas y los datos digitales. Una forma de hacerlo es la modulación: se produce una señal sinusoidal base llamada portadora, y los datos se codifican modificando esta portadora. Por ejemplo, se puede cambiar la amplitud de la portadora (ASK), su frecuencia (FSK), o incluso la amplitud y la fase al mismo tiempo (QAM).



Ejercicio 2.8 La figura anterior muestra un ejemplo de modulación en amplitud y otro de modulación en frecuencia. ¿Cuál es cuál? Para cada caso: ¿esa modulación transmite una señal analógica o digital?

Ejercicio 2.9 ¿Cuál es la diferencia entre ancho de banda y velocidad de transmisión? ¿Por qué crees que se usan indistintamente en contextos no técnicos?

3. Direcccionamiento

Conectar dos ordenadores los hace mucho más útiles que dos máquinas aisladas. Sin embargo, su verdadero potencial solo se libera cuando pueden conectarse a *muchos* otros ordenadores.

Las limitaciones prácticas, como el coste y la ubicación geográfica, hacen imposible conectar directamente cada par de ordenadores. En su lugar, los dispositivos se distribuyen en un grafo de redes interconectadas, que puede extenderse incluso más allá de la escala de un planeta.

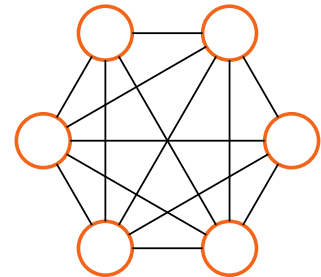
Ya sea localmente, al alcance físico, o en otro continente, se necesita un sistema de direccionamiento para identificar, localizar y encaminar los mensajes de modo que lleguen a los peers (pares) a los que van dirigidos, igual que hacemos con las ubicaciones físicas.

3.1 Por favor, contacta conmigo

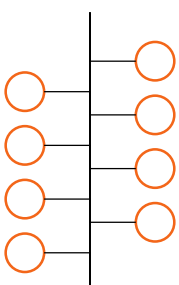
Cuando solo intervienen 2 dispositivos, se puede usar una conexión punto a punto. En este tipo de conexión solo existen “este lado” y “el otro lado” del cable. Además, cuando un dispositivo quiere comunicarse con el otro extremo, solo hay un enlace (p. ej. un cable) entre el que elegir, así que no hay confusión posible. Como podemos operar esta conexión con solo 2 tarjetas de red (también: controlador de red, NIC) y 1 cable, las conexiones punto a punto son viables para $N = 2$ dispositivos.

Si queremos conectar $N > 2$ dispositivos, una opción ingenua es conectar cada dispositivo con todos los demás formando un *grafo completo*. Ahora cada dispositivo necesita manejar $N - 1$ cables hacia los otros dispositivos, pero comunicarse con cualquier otro dispositivo es tan sencillo como elegir el cable correcto y establecer una conexión punto a punto.

El principal problema de este enfoque es que el número total de conexiones es $N \cdot (N - 1) / 2 = O(N^2)$ (más sobre esto en tu fiel asignatura de *Matemática Discreta*). Esto significa que, para tener $N = 10$ dispositivos conectados de esta manera, necesitarías por ejemplo 40 cables y 90 tarjetas de red. Para $N = 1000$ dispositivos, habría medio millón de conexiones punto a punto, lo cual ya resulta bastante inmanejable.



$O(N^2)$ es la notación de una cota de complejidad cuadrática. Esto *no* es lo mismo que exponencial y nunca debe confundirse. ([leer más...](#))



Nos queda un truco en la manga: los buses de datos, donde una o más líneas de datos están conectadas simultáneamente a múltiples dispositivos. Esto conserva la simplicidad de las conexiones punto a punto, porque cada dispositivo solo tiene que manejar 1 cable y puede usarlo para comunicarse con todos los demás dispositivos. También se conserva la rentabilidad, ya que solo hacen falta N tarjetas de red para N dispositivos ($O(N)$).

La principal ventaja de los buses de datos es también su principal debilidad: todos los dispositivos envían y reciben datos usando la misma línea de datos, pero no pueden hacerlo al mismo tiempo porque solo hay una línea. Si dos o más lo hacen, se produce una colisión que hace imposible la transmisión de datos mientras dura.



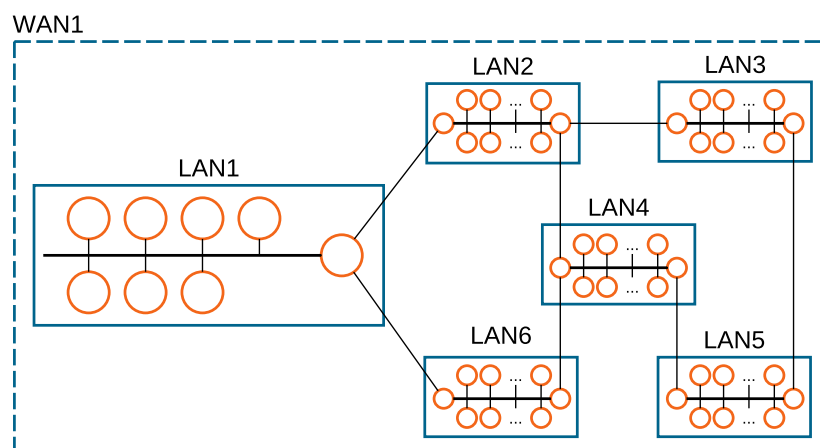
A medida que se desarrollaban las redes de ordenadores, se experimentó con muchas alternativas a la topología de bus. Un ejemplo curioso es Token Ring, donde los dispositivos se conectan formando un círculo y los mensajes se van pasando en un único sentido.

En la práctica, no es trivial implementar una tecnología de bus en la que se puedan ir añadiendo nuevos dispositivos con el tiempo. Históricamente, esto se hacía a veces perforando físicamente el cable principal del bus y añadiendo un cable nuevo hacia el nuevo ordenador que se conectaba. Más adelante, esto se simplificó con unos dispositivos llamados hubs (concentradores). Funcionaban de forma similar, pero ofrecían puertos físicos prefabricados y conectores extraíbles. Hoy en día, los hubs han sido sustituidos por los switches (conmutadores). Los switches ofrecen la funcionalidad de un bus, conectando todos los dispositivos entre sí, y además evitan las colisiones. Por razones obvias, estructurar las conexiones en torno a un switch se denomina topología en estrella.

Ejercicio 3.1 ¿Qué hace falta en un switch que no hace falta en un hub?



Las topologías de bus funcionan muy bien a escala local y se emplean ampliamente en las LANs (redes de área local). Sin embargo, no resultan rentables (a veces ni siquiera son físicamente posibles) cuando los dispositivos están muy separados. En su lugar, los dispositivos pueden organizarse en LANs separadas, cada una conceptualmente un bus, y luego esas LANs separadas pueden conectarse mediante conexiones punto a punto más baratas y factibles. Considerados en conjunto, esos dispositivos formarían entonces una WAN (red de área extensa).



3.2 Necesito encontrarte

Una vez más, la principal ventaja de los buses de datos es también su principal debilidad: todos los dispositivos envían y reciben datos usando la misma línea de datos. Aunque no se produzcan colisiones, todos los dispositivos reciben cada mensaje enviado al bus. Sin embargo, lo que realmente queremos es que nuestro mensaje llegue a su destino, y solo a su destino. Llamemos a esto el *problema de encontrar-mi-dispositivo*.

El problema de encontrar-mi-dispositivo no es exclusivo de los buses ni de las LANs. Al contrario, se vuelve muy importante cuando consideramos las WANs, donde no todos los dispositivos están directamente interconectados. En este caso, tenemos que asegurarnos de poder encaminar nuestros mensajes entre distintas LANs y de que lleguen a su destino.

Sorprendentemente, ni siquiera las conexiones punto a punto se libran del problema de

encontrar-mi-dispositivo. Suponiendo que los dispositivos A y B están conectados de ese modo, dos programas distintos pueden querer intercambiar información al mismo tiempo, p. ej. un software de control de alarmas y una aplicación de streaming de música. En este escenario, quieres que el cliente y el servidor del servicio de monitorización de alarmas se comuniquen entre sí, pero no con el servicio de streaming de música, y viceversa.

La solución más común a este problema es usar identificadores que nos permitan distinguir entre dispositivos o incluso entre procesos en ejecución dentro de un sistema operativo (SO). Algunos de estos identificadores se denominan direcciones, precisamente porque se usan para encontrar y alcanzar un dispositivo remoto.

Las direcciones son normalmente identificadores *numéricos*, es decir, simplemente un número. Como tales, pueden expresarse en distintas bases, incluidas la decimal y la hexadecimal. Estos números se toman de un conjunto predefinido, y la elección de ese conjunto determina el número máximo de elementos que podemos identificar de forma unívoca.

 Las direcciones expresadas como `d8:43:ae:61:ed:f1` o `192.168.1.1` (como se verá más adelante en el curso) son también números que podríamos haber expresado como `237785200061937` y `3232235777`, respectivamente (lo cual no resulta tan cómodo).

Ejercicio 3.2 ¿Cuántos elementos se pueden identificar de forma unívoca (como máximo) con una dirección que expresamos usando 5 bits? ¿Y con 10? ¿Y con 20? Expresa la solución general matemáticamente.

Ejercicio 3.3 Supón que hay 1000 máquinas en una LAN. ¿Cuántos bits de dirección hacen falta? ¿Y con 800 máquinas? ¿Y con 1100? Expresa la solución general matemáticamente.

 En 2019 nos quedamos sin direcciones de Internet (v4). ¡Uy!

3.3 ¿Cómo llego hasta ahí?

En muchos escenarios se necesitan varias direcciones para llevar a cabo la comunicación extremo_a_extremo deseada. Por ejemplo, puede que necesitemos identificar un único dispositivo dentro de una LAN, y esa LAN dentro de una WAN. A veces necesitaremos identificar un proceso (un programa en ejecución) dentro de ese dispositivo. Esto no es muy distinto de que alguien te visite en persona:

Joseph Edgar Foreman
123 South Fake St.
Adams County
Ohio

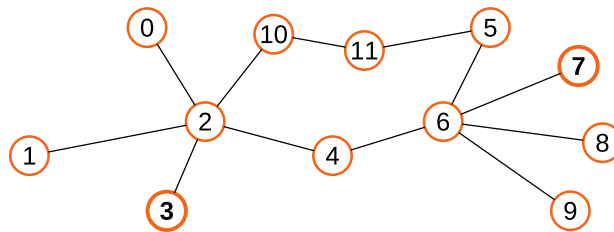
Process #1717403
Device #51688798
LAN 3141 (Data Engineering Labs)
WAN 442 (UAB)

Sea cual sea el sistema de direccionamiento que usemos, debe contener información suficiente para localizar y alcanzar el destino, es decir, para encaminar nuestros mensajes a través del grafo de conexiones. El proceso de encaminamiento implica múltiples pasos o “saltos” entre redes hasta alcanzar la LAN de destino.

El nombre router (encaminador) designa un tipo de dispositivo de red cuya tarea es reenviar estos mensajes entre redes. Estos dispositivos deben contener información suficiente para que, cuando se les entrega un mensaje con una dirección de destino, puedan decidir qué interfaz_de_red usar. A su vez, los dispositivos que no son routers solo necesitan configurarse para poder alcanzar el siguiente router.

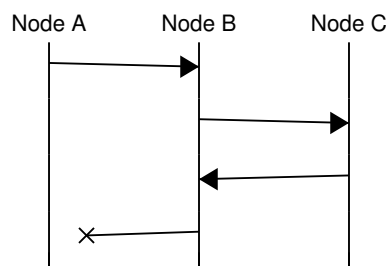
 En casa, tu router normalmente desempeña también el papel de switch, pero son conceptos diferentes.

Ejercicio 3.4 Esta figura representa un puñado de dispositivos y sus conexiones físicas. Estos dispositivos solo pueden comunicarse enviando mensajes a través de esas conexiones. Aun así, ③ quiere enviar un mensaje a ⑦.



- ¿Puedes identificar algún dispositivo que probablemente esté actuando como switch? ¿Y como router?
- ¿Cuál es el número mínimo de mensajes enviados para que ③ pueda contactar con ⑦, y ⑦ pueda responder a ③? Enuméralos en el orden en que se producen.
- ¿Qué información hay que incluir en esos mensajes para que pueda tener lugar la comunicación ③ ↔ ⑦?

Los diagramas de carriles como el de abajo se usan para representar las interacciones (p. ej. el intercambio de mensajes) entre actores (p. ej. dispositivos de red). En ellos, el eje vertical representa el tiempo, lo que resulta valiosísimo para identificar relaciones causa-efecto.



Ejercicio 3.5 A partir de tu respuesta al Ej.3.4, elabora un diagrama de carriles que represente los intercambios de mensajes. Para cada mensaje (p. ej. encima de las flechas), incluye la información de direccionamiento presente en él.



4. Paquetes

La información viaja por este grafo de redes en forma de paquetes, que contienen una pequeña cantidad de información y de meta-información.

El formato de los paquetes debe estar acordado por ambos extremos de la comunicación, de modo que esos paquetes puedan producirse e interpretarse de forma eficiente y automática.

Cuando los paquetes viajan por una o varias redes, pueden perderse o desordenarse. Si tiene que fluir un flujo continuo de datos entre dos ordenadores, los paquetes deben contener mecanismos para detectar pérdidas y restaurar el orden correcto.

4.1 Tengo un paquete para ti

No es viable establecer conexiones físicas entre cada par de ordenadores que quieren comunicarse. En su lugar, se comparten relativamente pocas conexiones para transportar mensajes entre muchos pares de dispositivos.

Si un único dispositivo transmite datos de forma continua durante mucho tiempo, esa conexión queda bloqueada y puede convertirse en un cuello de botella que impida comunicarse a otros dispositivos. Para evitar este problema, las conexiones limitan la cantidad máxima de datos que pueden enviarse sin interrupción. Como resultado, los dispositivos se ven obligados a enviar los datos en ráfagas discretas llamadas paquetes. El intercambio de este tipo de mensajes a través de una o varias redes se denomina conmutación de paquetes.



Leonard Kleinrock, uno de los pioneros de la interconexión de redes, disputa la autoría de la conmutación de paquetes, que a menudo se atribuye a Paul Baran y a Donald Davies.

Los paquetes deben ser pequeños para que las conexiones no queden bloqueadas demasiado tiempo. Al mismo tiempo, queremos que los paquetes contengan tantos datos como sea posible, porque cada paquete contiene datos de overhead (sobrecarga), como las direcciones que se comentan en la Sección 3, que hay que enviar además de los datos de payload (carga útil) del usuario. Las redes fijan la longitud máxima de paquete mediante un parámetro llamado unidad máxima de transmisión (MTU), p. ej. 1500 bytes en Ethernet.

La transmisión de paquetes a través de un enlace de datos debe hacerse con cuidado, en particular cuando ese enlace es un bus compartido por varios dispositivos. Cada paquete debe poder distinguirse individualmente del resto, lo que puede conseguirse mediante al menos tres estrategias:

1. *Longitud fija*: la longitud de todos los paquetes es la misma. El emisor y el receptor deben haber acordado este valor de antemano.
2. *Longitud explícita*: la longitud del paquete se incluye en el propio paquete, p. ej. usando el primer byte del paquete. El emisor y el receptor deben ponerse de acuerdo sobre cómo se incluye esta información y cómo interpretarla.
3. *Secuencias de escape*: ciertas secuencias de escape binarias dentro de los datos tienen un significado especial. Por ejemplo, podría definirse que el byte `11110000` (`0xf0`) significa “fin de paquete”: estos bits deben aparecer después de cada paquete, y solo entonces. Así, el emisor podría empezar a transmitir el contenido de un paquete, seguido de `0xf0`. Ambas partes deben acordar qué secuencias de escape usar, qué significan y cómo enviar datos iguales a esas secuencias (p. ej. debería ser posible

enviar `0xf0f0f0` sin provocar un “fin de paquete” hasta que se hayan transmitido todos los bytes).



También es posible señalar el *principio* de un paquete. En ese caso se utiliza una secuencia previa llamada preámbulo.

Ejercicio 4.1 Todas las estrategias descritas arriba se usan hoy en día en escenarios reales, dependiendo de las características, los costes y los compromisos de cada una.

Discute cuál de estas estrategias sería la más apropiada para cada uno de estos escenarios:

- Un único fabricante diseña el hardware y el software que comunica dos extremos. Una única línea de datos debe multiplexarse para varios comandos de control y también para varios flujos de datos. La prioridad es un uso eficiente de la energía y del buffer (memoria intermedia).
- Varios dispositivos comparten un bus. De vez en cuando envían mensajes de distintas longitudes, pero no un volumen enorme de datos. La prioridad es el coste.
- Varios dispositivos comparten un bus. Envían continuamente mensajes de distintas longitudes, intentando maximizar la tasa efectiva de transmisión por el canal. La prioridad es el throughput (caudal efectivo).

4.2 Por favor, rellena el formulario

Independientemente de la estrategia empleada, los paquetes de datos suelen constar de dos partes:

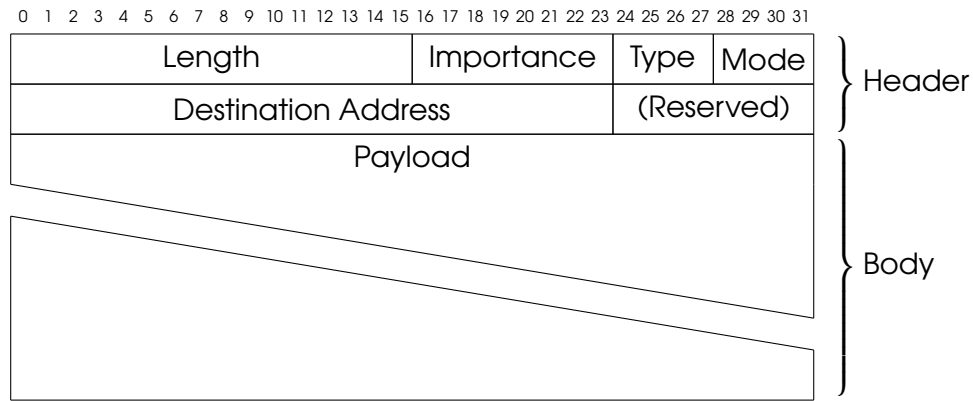
- payload de datos: la información que hay que transmitir, y
- metadatos: la meta-información necesaria para entregar el payload.

La longitud puede ser uno de los campos de metadatos incluidos en el paquete, junto con otros que ayudan a identificar su tipo y su función. La ubicación del payload y de los metadatos, así como los campos de metadatos exactos que se incluyen, su longitud y su significado deben ser conocidos por ambos extremos de la comunicación. Todas estas decisiones constituyen el formato de paquete o estructura de paquete.

Una de las formas más habituales de disponer el payload y los metadatos es con un formato cabecera-cuerpo. En este caso, toda la meta-información se incluye primero, seguida de los datos. Otros formatos pueden incluir un trailer (cola), también conocido como tail, que se usa sobre todo para la detección de errores y su corrección, en combinación con la cabecera o en su lugar. Todas las configuraciones siguientes son posibles.

Header		Body	
Header	Body		Trailer/Tail
Body			Trailer/Tail
Header			

El formato del payload de un paquete normalmente lo define el usuario. El formato de la cabecera, en cambio, está estrictamente definido para que pueda producirse y analizarse con facilidad. Este formato suele presentarse mediante un diagrama que ayuda a identificar las posiciones individuales de bit y de byte, como en el siguiente ejemplo (ficticio):



Los campos también pueden tener longitudes fijas o variables, que no encajan necesariamente con las fronteras de byte. Los campos también pueden tener una longitud igual a 1 bit, en cuyo caso normalmente se le llama `flag` o bit de flag.

En diagramas como el anterior se usan a menudo varias líneas (filas) para que puedan imprimirse y leerse con facilidad. En ese caso, el significado es el mismo que si todos los campos se hubieran presentado a lo largo de la misma línea (fila).

Ejercicio 4.2 El siguiente contenido de un paquete fue capturado de una red, expresado en hexadecimal. Suponiendo que el paquete tiene el formato del ejemplo anterior:

- Indica el valor de cada campo (longitud, importancia, tipo, modo, dirección de destino y payload).
- ¿Puedes adivinar el significado del payload?

00 0c 00 f3 bb bb bb 00 68 65 6c 70

Ejercicio 4.3 El siguiente código acepta dos argumentos: (1, `sys.argv[1]`) la ruta de un fichero de entrada, y (2, `sys.argv[2]`) la ruta de un fichero de salida. Se leen los primeros 255 bytes del contenido del fichero de entrada, se formatean como un paquete y los bytes de ese paquete se escriben en el fichero de salida.

```
.py .out snippets/simplepacketoutput.py
1 with (open(__file__, "rb") as input_file,
2       open("/dev/stdout", "wb") as output_file):
3     # Read at most 255 bytes from the file
4     payload: bytes = input_file.read(255)
5     # The + operation concatenates bytes
6     output_file.write(bytes(len(payload)) + payload)
```

- Describe el formato de paquete usado en el código, y sus limitaciones.
- Extiende el formato de paquete para que
 - los paquetes puedan ser más largos de 255 bytes
 - pueda codificarse la dirección del dispositivo de destino
- Proporciona un diagrama de bytes del nuevo formato, así como una explicación del sistema de direccionamiento que estés usando.
- Modifica el código para que produzca paquetes con el formato que has propuesto.

Lenguajes como Python pueden distinguir entre ficheros que contienen texto y ficheros que contienen datos binarios. En el código anterior, los ficheros se abren en modo binario usando los modos `'rb'` y `'wb'`.

Cuando se abre en modo binario, los bytes del fichero se representan directamente mediante un objeto `bytes`, un array de enteros entre 0 y 255. En modo texto, esos bytes son procesados (decodificados) además por el método `open`, que devuelve una cadena que puede contener caracteres especiales como acentos o glifos no latinos. ([leer más...](#))

4.3 No paran de llegar

Al desarrollar aplicaciones que usan capacidades de red, a menudo resulta útil imaginar la comunicación como un flujo, es decir, como una tubería conceptual donde ponemos nuestros datos (cualquier cantidad de datos) por un extremo y salen por el otro. Si tenemos esta capacidad, resulta mucho más fácil enviar ficheros de cualquier tamaño y transmitir una cantidad interminable de audio o de vídeo. Por desgracia, lo único que tenemos para simular esos flujos son paquetes.

Los paquetes a menudo tienen que reenviarse a través de varias redes. Los routers (enaminadores) no son perfectos y a veces están fuera de servicio o mal configurados, así que no todos los paquetes llegan a su destino. Además, paquetes distintos pueden entregarse por rutas distintas que pueden tener longitudes distintas, de modo que los paquetes pueden llegar desordenados.

Si queremos simular un flujo de datos, los paquetes deben contener suficiente meta-información para que puedan detectarse las pérdidas y restaurarse el orden correcto. Esto introduce un overhead que no es adecuado en todos los escenarios –p. ej. latencia muy baja–, así que algunas aplicaciones basan su comunicación en red en mensajes en lugar de en flujos.

Cuando se requieren flujos, se usa el concepto de offset (desplazamiento) para reensamblar varios mensajes en un único flujo. Cada paquete contiene algunos bytes del flujo de datos, y el offset indica la posición exacta dentro de ese flujo.

Ejercicio 4.4 El siguiente diagrama describe un flujo de 48 bytes producido por un nodo origen, así como los paquetes en los que se dividió antes de la transmisión:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47

<u>Packet 0</u> Offset 0 Length 10	<u>Packet 1</u> Offset 10 Length 14	<u>Packet 2</u> Offset 24 Length 6	<u>Packet 3</u> Offset 30 Length 18
--	---	--	---

- Describe la cabecera mínima que puede usarse para transmitir este flujo.
- Suponiendo que el receptor recibe primero el Paquete 2, después el Paquete 0, y que los demás paquetes se pierden: ¿qué bytes del flujo conoce el nodo de destino y qué bytes desconoce?
- ¿Cómo puede saber el nodo origen que algunos paquetes no se entregaron?
- ¿Qué ocurriría si el destino recibiera un mensaje espurio, con offset de 7 bytes y longitud de 4 bytes?

5. Una pila de capas

La comunicación entre ordenadores moderna se consigue mediante una pila de capas de red. Cada pila se centra en algunos problemas y es responsable de proporcionar ciertas funcionalidades, p. ej. direccionamiento y secuenciación correcta (introducidos en los capítulos anteriores).

Cada capa define un *par* de funciones *send/receive*, que es responsable de algunas de estas funcionalidades. Cada capa utiliza el *send* y el *receive* de la capa inferior, es decir, las funcionalidades de una capa se apoyan en las funcionalidades de todas las capas que tiene por debajo.

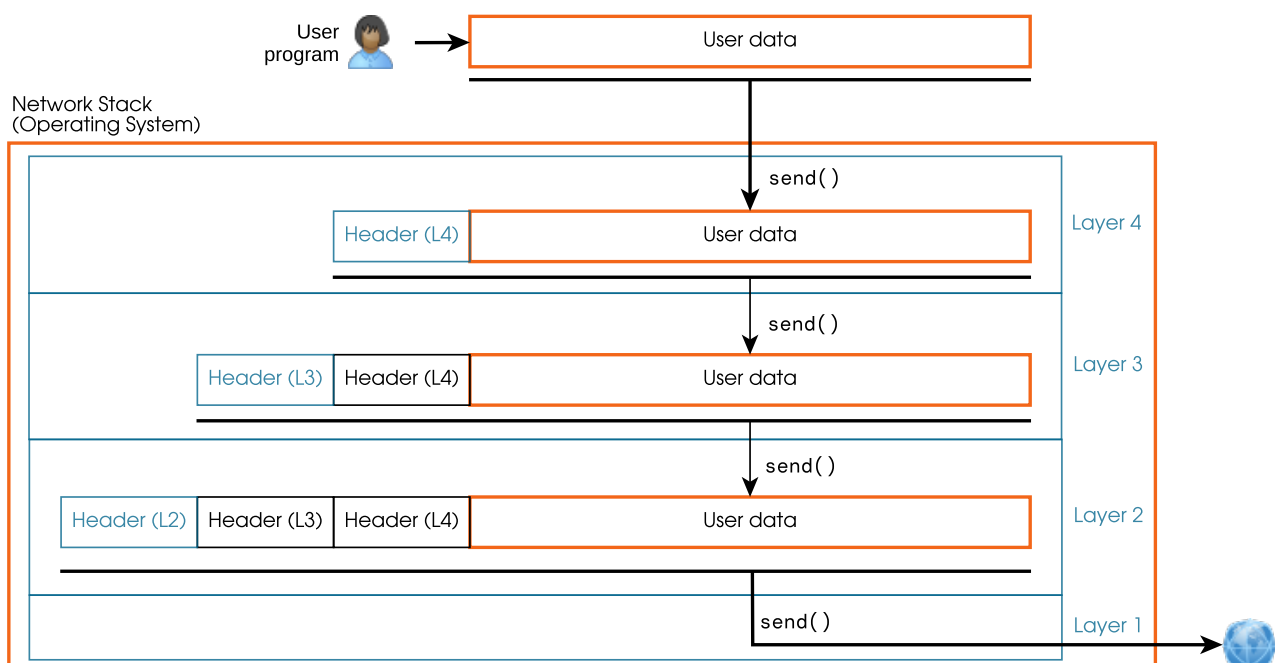
Para coordinar las capas y hacer posible la comunicación virtual entre ellas, cada capa define un formato de paquete distinto, que encapsula un paquete de la capa que tiene por encima.

5.1 Envío de datos

Para implementar estas capacidades, cada capa transmite no solo los datos proporcionados por la capa superior, sino también una cabecera con toda la metainformación necesaria para esa capacidad. Así, cada capa necesita definir el formato de paquete, añadiendo cierto overhead (sobrecarga) (típicamente una cabecera) al payload (carga útil), p. ej.:



Cuando un programa de usuario quiere enviar algunos datos, utiliza el método *send* de la capa *superior* de la pila, lo que acaba provocando que la capa física transmita ciertos datos, p. ej. a través de un cable o una antena. La siguiente figura representa la estructura de una pila de red de 4 capas:



Los paquetes producidos por cada capa se denominan genéricamente PDUs (Protocol Data Unit). Las PDU de cada capa tienen nombres específicos como frame (trama), datagrama o

segmento. Estos nombres deben usarse con precisión en contextos técnicos.

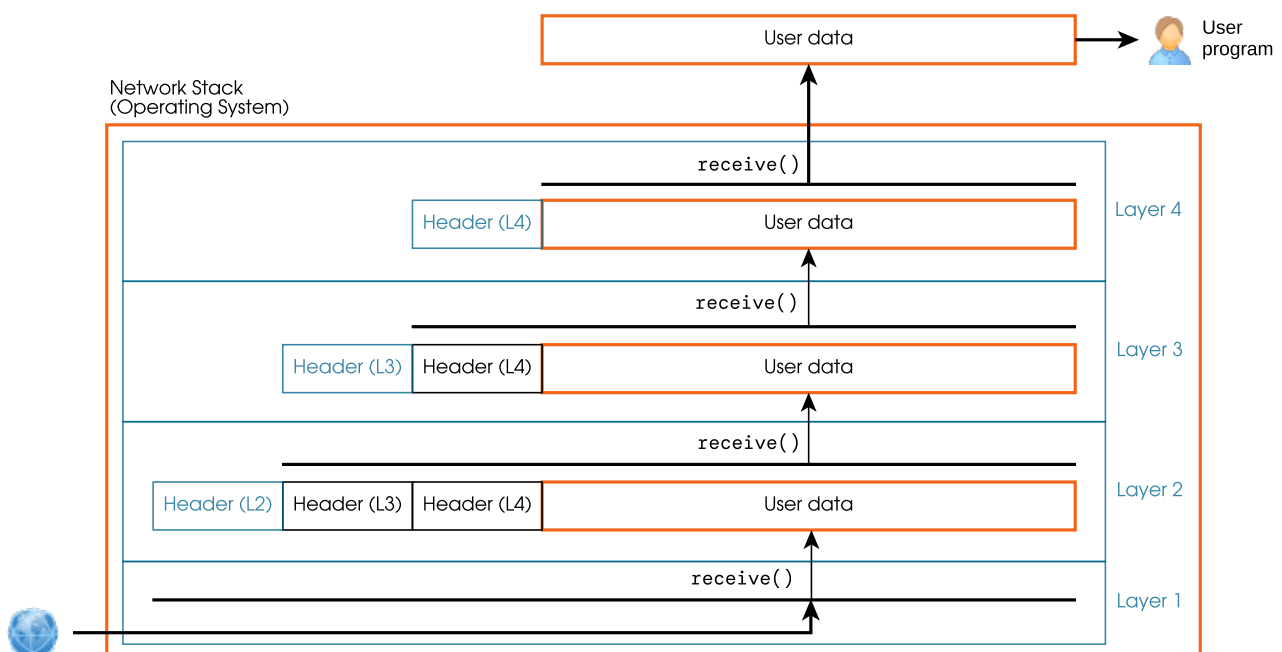
Cada capa incluye la PDU de la capa superior dentro de la definición del packet format de su propia PDU. Este proceso se denomina encapsulación.

Ejercicio 5.1 El programa de usuario quiere enviar 420 bytes de datos utilizando la pila de red de la figura anterior. Supón un tamaño de cabecera para las capas 2, 3 y 4 de 14 bytes, 20 bytes y 20 bytes, respectivamente.

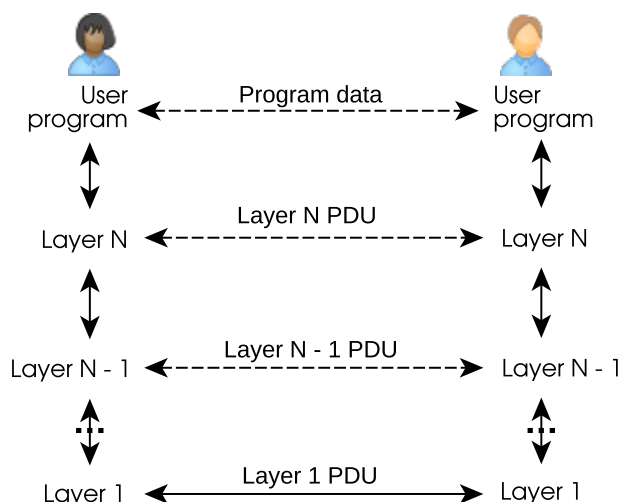
- ¿Cuál es el tamaño del payload y el tamaño del overhead de la PDU de cada capa?
- ¿Cuál es el tamaño del payload y el tamaño del overhead de toda la pila de red?

5.2 Recepción de datos

Cuando un paquete llega a la pila de red del destinatario, se procesa en orden inverso, es decir, las capas se recorren de abajo arriba. Cada capa inspecciona *solo* la cabecera correspondiente a esa capa. Si todo es correcto, la capa elimina la cabecera y pasa el resto del paquete a la capa superior. A su vez, la capa de arriba pasa los datos restantes (el payload) al destinatario. Continuando con el ejemplo de la pila de 4 capas:



En una pila de red, solo la capa inferior interactúa directamente con el exterior, p. ej. solo la tarjeta de red física de un ordenador está conectada a un cable que lleva datos a otros ordenadores. Al mismo tiempo, los datos enviados por la capa inferior incluyen las PDU de todas las capas mediante la encapsulación. Gracias a esto, existe una comunicación virtual entre cada capa de un extremo y su homóloga en el otro extremo de la comunicación. La siguiente figura representa la comunicación real (línea continua) y la virtual (línea discontinua) en nuestro ejemplo:



Ejercicio 5.2 Otro ejemplo de pila de red tiene 3 capas: L1, L2 y L3 de abajo arriba.

La PDU de la capa L2 es la siguiente:

0	7	8	15	16	23	24	31
Destination address				Payload length (bytes)			
Payload (variable length)							
⋮							

La PDU de la capa L3 es la siguiente:

0	7	8	15	16	23	24	31
Fragment offset				Payload length (bytes)			
Payload (variable length)							
⋮							

La capa L1 recibe un paquete de datos y su función `receive` devuelve los siguientes datos (supón enteros unsigned (sin signo) y orden big endian cuando sea necesario):

01 23 00 0a 01 00 00 06 00 aa 00 bb 43 20

- ¿Qué payload de datos se envió al programa destinatario en este paquete?
- Si este es el último paquete de la comunicación, ¿cuántos datos (de payload) se enviaron al programa de usuario destinatario?
- Como máximo, ¿cuántos ordenadores distintos pueden conectarse usando esta pila de red?
- Enviamos un fichero como payload de esta pila. ¿Cuál es su tamaño máximo posible?

```

.PY .out snippets/encapsulation.py
1 import typing
2 import struct
3
4 def layer2_encapsulate(address: int, data: bytes) -> bytes:
5     """Encapsulate a PDU of layer 3 into a PDU of layer 2."""
6     assert 0x0000 <= address <= 0xffff, "Invalid layer 2 address"
7     assert len(data) <= 0xffff, "Too much data for a layer 2 PDU"
8     return bytes(struct.pack(">H", address)
9                 + struct.pack(">H", len(data))
10                + data)
11
12 def layer3_encapsulate(offset: int, data: bytes) -> bytes:
13     """Encapsulate a chunk of user data into a PDU of layer 3."""
14     assert 0x0000 <= offset <= 0xffff, "Invalid layer 3 offset"
15     assert len(data) <= 0xffff, "Too much data for a layer 3 PDU"
16     return bytes(struct.pack(">H", offset)
17                 + struct.pack(">H", len(data))
18                + data)
19
20 def stack_send(data: bytes, output_file: typing.BinaryIO):
21     layer3_pdu = layer3_encapsulate(offset=0, data=data)
22     layer2_pdu = layer2_encapsulate(address=0xabcd, data=layer3_pdu)
23     output_file.write(layer2_pdu)
24
25 if __name__ == '__main__':
26     with (open(__file__, "rb") as input_file,
27          open("/dev/stdout", "wb") as output_file):
28         payload = input_file.read()
29         stack_send(data=payload, output_file=output_file)

```

Ejercicio 5.3 El código anterior toma unos datos y escribe los bytes que se enviarían a la capa 1 (la capa física) de la pila de red del ejercicio 5.2. Implementa el extremo receptor de la pila de la siguiente manera:

- Implementa una función `layer2_decapsulate(data: bytes) -> (int, bytes)` que tome una PDU de la capa 2 y devuelva la tupla (`address`, `payload`), donde `address` es la dirección de destino y `payload` es la PDU de la capa 3 encapsulada.
- Implementa una función `layer3_decapsulate(data: bytes) -> (int, bytes)` que tome una PDU de la capa 3 y devuelva la tupla (`offset`, `payload`), donde `offset` es el offset (desplazamiento) de la carga útil, y `payload` es el fragmento de datos enviado por el otro extremo de la comunicación.
- Implementa una función `stack_receive(data: bytes)` que decodifique una PDU de la capa 1 (usando las dos funciones anteriores) y muestre una línea de mensaje similar a
"Received a chunk of data. Length: 12 bytes, Address: 0xabcd, Offset: 0."
- Completa la función `stack_send` de forma que los datos de entrada se dividan en fragmentos de como mucho 100 bytes, se produzca una PDU de la capa 1 válida para cada uno de esos fragmentos, y esas PDU de la capa 1 se pasen a `stack_receive`.
- ¿Funcionaría tu implementación si las PDU de la capa 1 se ordenaran aleatoriamente antes de pasarlas a la pila receptora?



La biblioteca `struct` de Python es útil, aunque no obligatoria, para dar formato a los bytes de un paquete.

Internet

La parte anterior de esta guía exploró algunos de los principales retos de comunicar varios ordenadores entre sí. Internet es una solución que funciona para esos retos: la que los humanos hemos creado a lo largo del último medio siglo, y que hoy resulta central en muchas actividades sociales, comerciales y políticas. Esta parte estudia las principales prestaciones que ofrece Internet, así como el conjunto de protocolos que se emplean para conseguirlas.

6	Internet	25
6.1	La pila TCP/IP	
6.2	Visión general de los protocolos TCP/IP	
6.3	¿Quién manda en Internet?	
7	Capa 2: comunicación en red (LAN) .	28
7.1	Direccionamiento de capa 2	
7.2	Protocolo Ethernet de capa 2	
8	Capa 3: comunicación Internet(work)	32
8.1	Direccionamiento en la capa 3	
8.2	IP – Internet Protocol	
8.3	Encaminamiento IP	
8.4	Subnetting IP	
8.5	Fragmentación IP	
8.6	ARP – Address Resolution Protocol	
8.7	ICMP – Internet Control Message Protocol	
9	Capa 4: Transporte	47
9.1	Direccionamiento de capa 4	
9.2	UDP – User Datagram Protocol	
9.3	TCP – Transport Control Protocol	
9.4	TCP: “Conectando”	
9.5	TCP: “Conexión establecida”	
9.6	TCP: “Cerrando la conexión”	
10	Capa 7: Aplicación	59
10.1	DNS – Domain Name System	
10.2	DHCP – Dynamic Host Configuration Protocol	
10.3	Sistemas autónomos	

6. Internet

Las comunicaciones modernas se basan en pilas de red organizadas por capas. Quizá acabas de descargar esta guía de Internet. ¿Qué capas se usan exactamente? ¿Qué protocolos se emplean, y con qué finalidad? Este capítulo ofrece una primera aproximación para responder a estas preguntas.

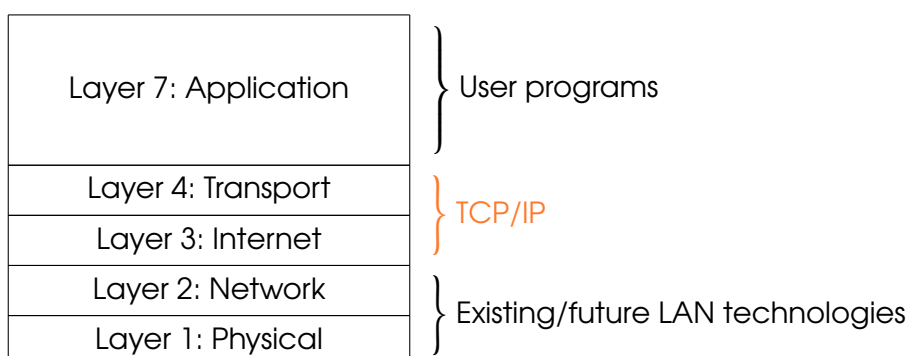
6.1 La pila TCP/IP

Internet es una solución práctica al problema de la comunicación global entre ordenadores, cuyo desarrollo comenzó a finales de los años 60 – principios de los 70, y todavía continúa.

La conmutación de paquetes se concibió a principios de los años 60, y la primera implementación de área extensa, ARPANET, comenzó en 1969. Un reto clave en aquel momento era interconectar las redes existentes, propiedad de organizaciones distintas y basadas en tecnologías y protocolos diferentes. Para que aquello fuese viable, había que cumplir todos los requisitos siguientes:

- Las redes existentes debían funcionar sin actualizar la infraestructura física ni sus protocolos, solo la forma en que se usan esos protocolos.
- Las redes existentes debían seguir gestionándose de forma independiente a nivel local (organización interna autónoma).
- Los paquetes no siempre se entregan: algunos se pierden.
- Los mensajes que llegan pueden no estar en orden, y pueden repetirse.

La solución adoptada por ARPANET –y más tarde por el mundo entero– es el conjunto de protocolos TCP/IP, descrito por primera vez en 1974. Es un enfoque por capas, diseñado para “enchufarse” encima de las LANs (redes de área local) existentes, con el objetivo de ofrecer capacidades de red a todas las aplicaciones de usuario.



En resumen, los principales servicios que ofrecen estas capas son los siguientes:

Capa 1 (Física): Enviar datos digitales a través de medios analógicos.

Capa 2 (Red): Identificar y comunicar dispositivos dentro de una misma LAN.

A veces se usa el término capa de “Enlace de datos” (del modelo OSI).

Capa 3 (Internet): Identificar y comunicar dispositivos entre todas las LANs.

A veces se usa el término capa de “Red” (también de OSI).

Capa 4 (Transporte): Identificar y comunicar programas (servicios).

Además (elige *solo una*):

- a) Asegurar que un flujo de datos llega correctamente al otro extremo, y en orden – pero se introduce un overhead (sobrecarga) y hay que establecer conexiones (TCP).

- b) Usar mensajes sencillos y relativamente eficientes, con poco overhead – pero los mensajes pueden perderse, reordenarse o duplicarse (UDP).

Capa 7 (Aplicación): Hacer cualquier cosa que el usuario pueda necesitar.

No olvides la seguridad (p. ej. criptografía) ni la eficiencia (p. ej. compresión): la capa 4 y las inferiores no proporcionan eso.

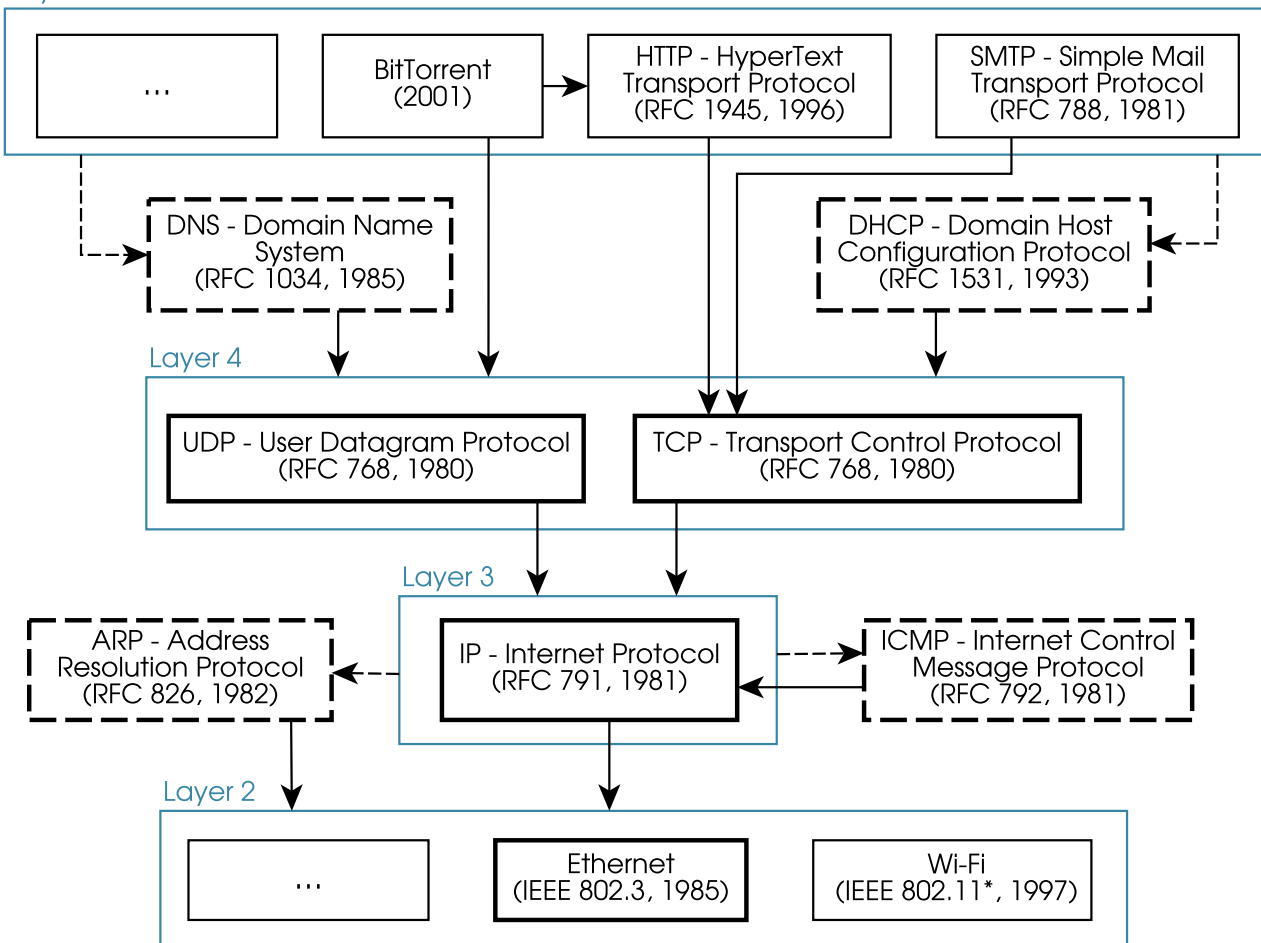
6.2 Visión general de los protocolos TCP/IP

Se definen múltiples protocolos para implementar la funcionalidad de cada capa. Estas definiciones incluyen el formato de paquete necesario, las reglas para intercambiar esos paquetes y su semántica. Estos protocolos deben entenderse como un sistema en el que cada parte tiene tareas muy concretas.

En la figura siguiente se muestran algunos de los protocolos más relevantes para las comunicaciones a través de Internet. En ella:

- Las flechas continuas indican que un protocolo se encapsula dentro de otro. Si el destino es una capa, debe usarse alguno de los protocolos de esa capa.
- Las cajas discontinuas indican protocolos auxiliares definidos para ayudar a que otros funcionen. Las flechas discontinuas indican qué capa usa qué protocolo auxiliar.
- Las cajas con líneas más gruesas indican los protocolos destacados en esta guía.

Layer 7



Ejercicio 6.1 Queremos enviar por correo electrónico la frase “Perdona, llegaré 5 minutos tarde”. Nuestro cliente de correo enviará esos datos usando el protocolo SMTP, que a su vez usa TCP (parte de la capa 4). Suponiendo que SMTP usa cabeceras en su PDU, ¿cuántas cabeceras contendrá la PDU correspondiente de la capa 2?

Ejercicio 6.2 ¿Por qué crees que la capa 3 tiene un solo protocolo (dos si cuentas IPv4 e IPv6 por separado)?

6.3 ¿Quién manda en Internet?

El Internet Engineering Task Force ([IETF](#)) se creó en 1986 para desarrollar y publicar estándares destinados a usarse en Internet. Su producto principal son los documentos “Request for Comment” ([RFC](#)), que contienen descripciones técnicas completas de esos estándares.

Estos RFC los crean grupos de trabajo de expertos, para que cualquiera los use libremente y de forma gratuita. Cualquiera puede participar en estos grupos de trabajo, incluidos investigadores, gobiernos y partes interesadas de la industria. Las decisiones se toman cuando el 80-90% de los participantes está de acuerdo, así que ninguna entidad controla por sí sola el proceso de estandarización.



“Rechazamos a los reyes, a los presidentes y las votaciones. Creemos en el consenso aproximado y en el código que funciona” — Dave Clark (IETF), 1992.

Consulta el DOI [10.1109/MAHC.2006.42](#) para una breve historia de lucha por el poder.

Los estándares RFC son descriptivos, no prescriptivos. Esto significa que nadie vigila el uso correcto de estos estándares. En cambio, los fabricantes tienen el incentivo de cumplir los RFC para que sus productos sean compatibles con los demás. El valor de los RFC (y de cualquier otro estándar) depende de que los usuarios los adopten. Por este motivo, los RFC pueden actualizarse y retirarse.

Los protocolos TCP/IP prevalecen porque son flexibles y funcionan razonablemente bien en la mayoría de las situaciones, aunque no sean óptimos en prácticamente ninguna de ellas. Además, los protocolos TCP/IP son gratuitos, a diferencia de los protocolos privativos que obligaban a comprar la solución de un fabricante concreto.



Hoy cuesta imaginar un mundo *no* dominado por la arquitectura TCP/IP, pero hasta finales de los 90 se comercializaron y mantuvieron pilas completamente distintas. Algunos ejemplos son AppleTalk e IPX/SPX, de Apple y Novell, respectivamente.

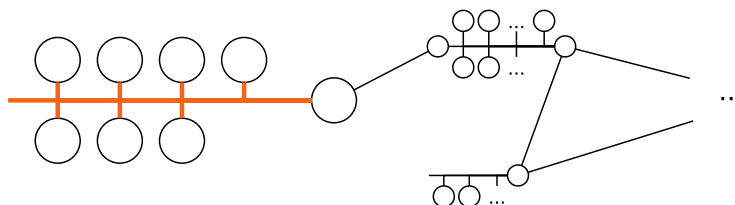
Algunos aspectos de TCP/IP, como los identificadores únicos (incluido [DNS](#), el Domain Name System) y los números reservados, tienen que ser acordados por todos para que Internet funcione correctamente. La [IANA](#) (Internet Assigned Numbers Authority) trabaja en coordinación con el [IETF](#) para definir los [RFCs](#) adecuados.



Desde 2016, la IANA está gestionada por una organización multisectorial sin ánimo de lucro ([ICANN](#) - Internet Corporation for Assigned Names and Numbers). Desde 2004, la responsabilidad de los dominios regionales (p. ej. [.cat](#), [.fr](#), [.es](#)) se transfiere a organizaciones de ámbito continental llamadas registros regionales de Internet ([RIRs](#)).

7. Capa 2: comunicación en red (LAN)

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



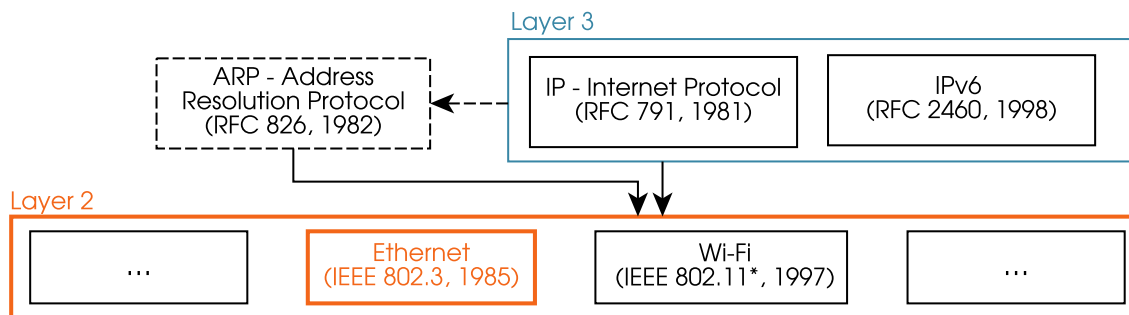
Capacidades

Los protocolos de capa 2 permiten que varios dispositivos intercambien mensajes y se identifiquen entre sí dentro de una red local (LAN, red de área local). No se puede contactar *directamente* con los dispositivos de fuera de la LAN mediante protocolos de capa 2.

La capa 2 abstrae a las capas superiores (incluidas las aplicaciones de usuario) de los detalles concretos del hardware conectado. Esto hace más fácil (léase: más barato) escribir el código una vez y ejecutarlo en cualquier sitio.

Los mensajes de capa 2 tienen una longitud limitada. La cantidad máxima de datos de payload (carga útil) que se pueden insertar en un protocolo de capa 2 se denomina unidad máxima de transmisión (MTU), p. ej. 1500 bytes en Ethernet.

Algunos protocolos de capa 2 también pueden proporcionar autenticación de usuario y protección criptográfica; Wi-Fi es un ejemplo destacado, aunque las redes cableadas también pueden emplear, p. ej. IEEE 802.1X.



Protocolos

Se han definido y se siguen definiendo muchos protocolos dentro de la capa 2. Los más importantes en la actualidad:

- Ethernet (IEEE 802.3), que se trata en Section [7.2](#).
- Wi-Fi (p. ej. IEEE 802.11be, alias Wi-Fi 7, de 2024).

Dentro de TCP/IP, la capa 2 encapsula las siguientes PDU de protocolo (ambas se tratan en el Capítulo [8](#)):

- Datagramas IPv4 e IPv6.
- Mensajes ARP.



La orden `ip link` muestra y permite configurar tus interfaces, incluida la dirección MAC.

Tu sistema operativo suele asignar automáticamente los nombres internos de las interfaces según su tipo, p. ej. `eth0` o `wlan1`. Normalmente se pueden renombrar las interfaces, porque los nombres de interfaz no forman parte de TCP/IP; se usan

exclusivamente dentro de ese sistema operativo.

7.1 Direccionamiento de capa 2

En la mayoría de las tecnologías de LAN, las direcciones de capa 2 son identificadores numéricos únicos para cada dispositivo dentro de la LAN. El tipo más frecuente son las direcciones MAC de Ethernet (técnicamente EUI-48), que ocupan 6 bytes (es decir, 48 bits). También se usan fuera de Ethernet, p. ej. en Wi-Fi y Bluetooth.

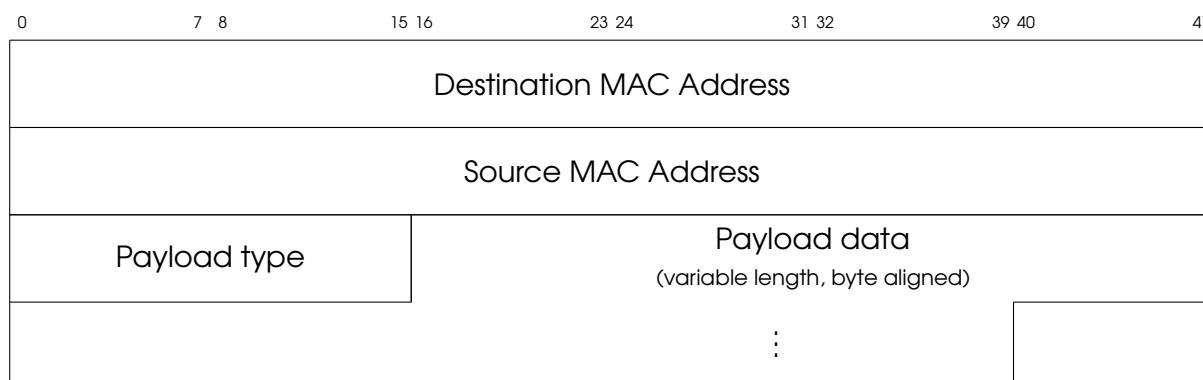
Las direcciones MAC se presentan a las personas habitualmente en el siguiente formato: `01:23:45:67:89:ab`. Algunas direcciones tienen un significado especial y no son identificadores válidos de dispositivo. Por ejemplo, `ff:ff:ff:ff:ff:ff` es la dirección de broadcast (difusión), que significa “todos los de la LAN”. Además, `00:00:00:00:00:00` se usa para representar una dirección MAC desconocida (consulta [RFC 9542](#) para una descripción completa de las direcciones MAC reservadas, incluidas las de multicast (envío a un grupo)).

El hardware de red viene con una dirección MAC integrada, única y predefinida, lista para usar. Las MAC de los dispositivos se pueden configurar a nivel de sistema operativo, pero permanecen constantes mientras están en uso.

7.2 Protocolo Ethernet de capa 2

7.2.1 Formato de paquete

La capa 2 de Ethernet define el siguiente formato, usado en todos los mensajes. Los paquetes de este tipo se denominan frames (tramas):



- **MAC de destino:** la dirección MAC del dispositivo de destino, o `ff:ff:ff:ff:ff:ff` para broadcast.
- **MAC de origen:** la MAC del dispositivo que envía el frame.
- **Tipo de payload:** identificador del protocolo encapsulado en el payload, p. ej. `0x0800` → IPv4, `0x86DD` → IPv6, y `0x0806` → ARP. A veces se le llama EtherType.
- **Payload:** contenido que se solicita enviar, p. ej. por IPv4 o ARP. La longitud máxima de este campo, la MTU, es de 1500 bytes en ethernet.

Ejercicio 7.1

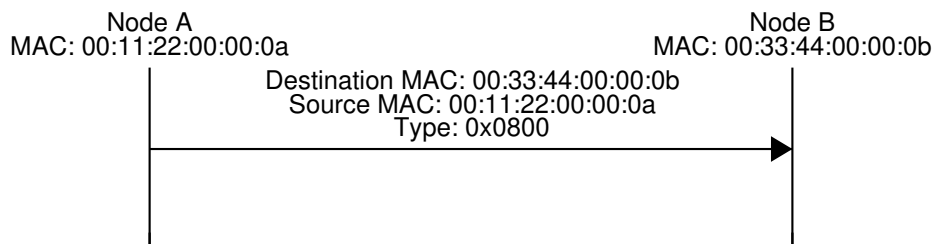
- ¿Cuántas direcciones MAC diferentes hay en Ethernet (incluidos los bloques reservados)?
- ¿Son suficientes para que cada dispositivo de la Tierra con capacidad de conectarse a internet tenga una MAC única?

Ejercicio 7.2 ¿Es válido el siguiente frame?

c2	21	90	15	bc	b6	42	40	e5	f7	5e	32	08	42	77	57
03	ca	27	6b	1f	97	e9	02	24	7f	80	fa	8b	61	59	2a
81	b6	73	b0	b4	e1	75	4f	ef	40	dd	9f	34	3c	48	67

7.2.2 Funcionamiento

Los mensajes dentro de una LAN Ethernet se envían normalmente en modo unicast (envío a un único destinatario), es decir, son mensajes de uno a uno. Esto permite que los switches (conmutadores) minimicen el consumo de energía y de ancho de banda. En unicast, el dispositivo de origen debe conocer la MAC del destino.



Ethernet también admite el broadcast a todos los nodos de la LAN. En este caso, la dirección MAC de destino debe ser ff:ff:ff:ff:ff:ff. Cuando un dispositivo recibe un frame, lo descarta salvo que el campo de MAC de destino del frame sea idéntico a su propia dirección MAC, o sea el de broadcast.

Un nodo también descartará un frame con un campo de tipo de payload (EtherType) no soportado. Si el tipo de payload sí está soportado, se usa para decidir qué parte del sistema operativo recibe el mensaje, p. ej. la pila IPv4 o el subsistema ARP.

Ejercicio 7.3 Siguiendo con el ejemplo de la figura anterior, la tarjeta de red del nodo B recibe un frame que empieza con los siguientes bytes. ¿Debería aceptarlo B?

00 11 22 00 00 0a ff ff ff ff ff ff ff ff 08 DD ...

Ejercicio 7.4 Los siguientes scripts ejemplifican cómo enviar y recibir frames Ethernet en crudo. El cliente envía 5 frames idénticos de EtherType 0x1234 y después termina. El servidor espera hasta recibir 5 frames Ethernet de ese tipo (se comprueba en las líneas 12-13) y después termina.

- Cambia la dirección MAC y el nombre de interfaz del cliente por los tuyos.
- Haz que el servidor rechace los frames que no vayan dirigidos a él.
- Amplía estos scripts para implementar una aplicación sencilla con capacidades de LAN.

Ejemplos sugeridos:

- Un servicio de eco de capa 2 que distinga entre peticiones y respuestas.
- Un chat peer-to-peer (par a par) de capa 2 que incluya el apodo del origen en todos los mensajes.
- Un servicio de calculadora de capa 2 que soporte operaciones aritméticas básicas (+, -, *, división entera //, opcionalmente división /).
- ¿Qué falta para que tu aplicación pueda conectarse al resto de Internet más allá de tu LAN?

  snippets/ethernetclient.py

```

1 import socket
2
3 interface_name = "wlp5s0" # Needs manual configuration
4 source_mac = bytes([0x00, 0x11, 0x22, 0x33, 0x44, 0x55])
5 destination_mac = bytes([0xff, 0xff, 0xff, 0xff, 0xff, 0xff])
6 packet_type = bytes([0x12, 0x34])
7 payload_data = bytes([0x78, 0x6f, 0x69, 0x20, 0x75, 0x61, 0x62])
8
9 # This socket sends Layer 2 packets directly.
10 s = socket.socket(family=socket.AF_PACKET,
11                  type=socket.SOCK_RAW,
12                  proto=socket.ntohs(socket.ETH_P_ALL))
13 s.bind((interface_name, 0))
14
15 for i in range(5):
16     frame = destination_mac + source_mac + packet_type + payload_data
17     s.send(frame)
18     print(f"Sent frame {i + 1}/5: {len(frame)=}, {payload_data=}")

```

  snippets/ethernetserver.py

```

1 import socket
2
3 interface_name = "wlp5s0" # Needs manual configuration
4
5 # This socket receives Layer 2 packets directly.
6 s = socket.socket(family=socket.AF_PACKET,
7                  type=socket.SOCK_RAW,
8                  proto=socket.ntohs(socket.ETH_P_ALL))
9 s.bind((interface_name, 0))
10
11 valid_frames = 0
12 while valid_frames < 5:
13     frame = s.recv(1514)
14     if frame[12:14] != bytes([0x12, 0x34]):
15         continue
16     valid_frames += 1
17     print(f"Received frame {valid_frames + 1}/5, {len(frame)} bytes. "
18           f"Payload: {frame[14:]:!r}")

```



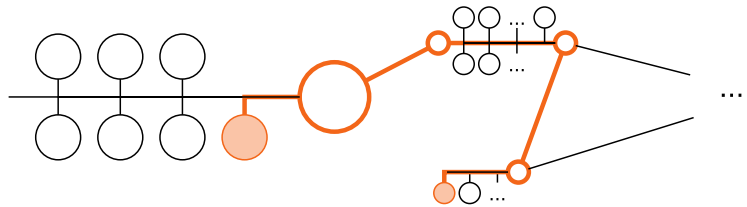
Tendrás que ejecutar estos scripts con privilegios, p. ej. con `sudo`. Como alternativa, puedes añadir permanentemente las capacidades `CAP_NET_RAW` a tu binario de python con

```
sudo setcap cap_net_raw+ep ./venv/bin/python.
```

Después de eso, puedes ejecutar `./venv/bin/python script.py` directamente sin `sudo`.

8. Capa 3: comunicación Internet(work)

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



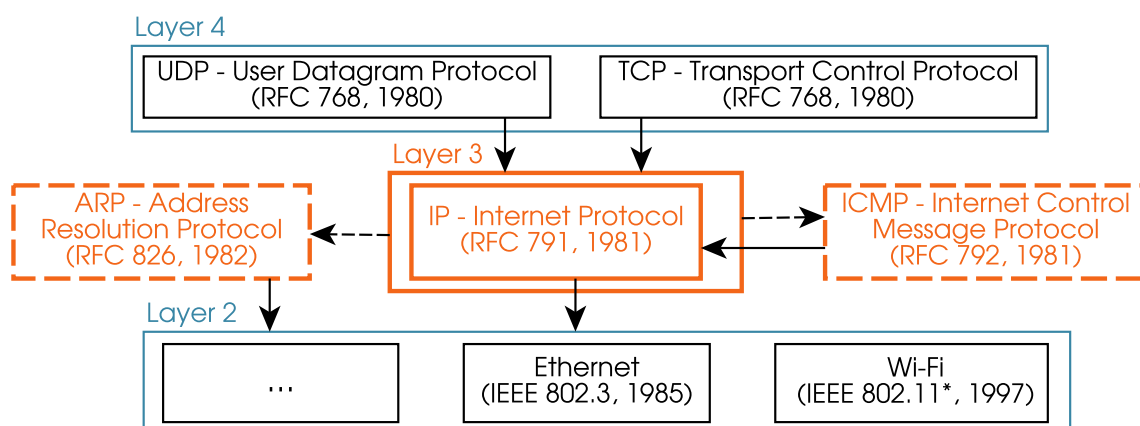
Capacidades

El Internet Protocol (IP) permite intercambiar datagramas (PDUs de la capa 3) entre dispositivos de distintas LANs (redes de área local). Su sistema de direccionamiento identifica los dispositivos de forma global y hace posible encaminar datagramas allí donde lo necesitamos.

Los dispositivos que se comunican mediante IP pueden pertenecer a LANs basadas en tecnologías diferentes y con distintos tipos de dirección MAC. Esto tiene dos implicaciones:

- Un datagrama que “cabe” en la MTU de la LAN de origen puede ser demasiado grande para la MTU de la LAN de destino (o de cualquier LAN intermedia). IP define un sistema de fragmentación para dividir datagramas, que solo se reconstruyen en el destino final.
- Cuando enviamos un datagrama, conocemos la dirección IP del dispositivo de destino, pero no necesariamente su dirección MAC. Solo necesitaremos esa MAC si formamos parte de la LAN de destino. IP usa ARP para traducir direcciones IP conocidas a direcciones MAC del tipo de la LAN de destino.

Los datagramas pueden viajar por redes que no controlan ni el origen ni el destino, por lo que pueden perderse, desordenarse y duplicarse. La capa 3 no proporciona mecanismos para tratar esto; las capas superiores deben implementar mecanismos de protección cuando sea necesario (p. ej. para implementar los flujos de datos de TCP).



Protocolos

La versión 4 de IP es el protocolo usado en la capa 3, es decir, es común a prácticamente todas las comunicaciones de Internet tal y como la conocemos hoy. Desde hace muchos años, Internet intenta hacer la transición hacia IPv6, pero el proceso no está completo y solo IPv4 proporciona cobertura global.

IPv4 delega en el protocolo ARP la traducción de direcciones IP conocidas a direcciones MAC dentro de cada LAN. ARP *no* usa datagramas IP.

Los siguientes protocolos encapsulan sus PDUs en el payload (carga útil) de los datagramas IP:

- Transport Control Protocol (TCP)
- User Datagram Protocol (UDP)
- Internet Control Message Protocol (ICMP)



Los comandos `ip address`, `ip route` y `ip neighbor` (entre otros) te permiten consultar varios aspectos de tu configuración IP.

8.1 Direccionamiento en la capa 3

Las direcciones IPv4 tienen 32 bits de longitud. Lo más habitual es presentarlas a las personas en formato quad decimal, p. ej.

142.250.201.67

Las direcciones IPv6 tienen 128 bits (16 bytes) de longitud, es decir, son 4 veces mayores que una dirección IPv4. Se expresan en formato hexadecimal, p. ej. como sigue. Fíjate en cómo dos signos de dos puntos ":" indican "rellena con ceros", y "::" solo puede aparecer una vez por dirección IPv6.

fe80::f524:a73a:e946:7c02



Cuando una dirección IPv6 se usa como parte de una URL, se rodea con corchetes, p. ej. `https://[fe80::f524:a73a:e946:7c02]/`.

8.1.1 Direcciones públicas y reservadas

La mayoría de las direcciones IP son públicas y únicas en todo Internet, es decir, la IP anterior tiene el mismo significado en todo el mundo: identifica un único dispositivo conectado. Muchas direcciones son reservadas/privadas y solo pueden usarse dentro de un ámbito concreto (p. ej. una LAN) o en una situación concreta. Algunas de las más comunes son las siguientes (hay algunas más, también para IPv6):

Bloque de direcciones IP	Ámbito de la dirección
127.*.*.*	Este ordenador (localhost)
10.*.*.*	Interno (privado)
172.16.*.*	Interno (privado)
192.168.*.*	Interno (privado)
0.*.*.*	Usos especiales
169.254.*.*	Usos especiales
255.255.255.255	Usos especiales

Ejercicio 8.1

- ¿Cuántas direcciones IPv4 distintas hay (incluyendo las reservadas y privadas)?
- ¿Son suficientes ahora y en el futuro?
- ¿Es problemático que una dirección como 192.168.0.1 la estén usando simultáneamente miles (probablemente millones) de ordenadores en Internet ahora mismo?
- ¿Cuáles de estas corresponden a direcciones link local?

8.1.2 Máscaras de red

Los 32 bits de una dirección IP pueden dividirse en dos partes usando una máscara de red:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Network ID																Device ID															

La primera parte identifica la LAN o el grupo de LANs al que pertenece un dispositivo. La segunda parte identifica un único dispositivo dentro de la LAN. La máscara de red determina

cuántos bits se asignan a cada parte.

En la figura anterior, la máscara de red asigna 10 bits al Network ID y 22 al Device ID. Esta máscara puede tomar cualquier valor entre 0 (todos los bits son Device ID) y 32 (todos los bits son Network ID).

Las máscaras de red se presentan a las personas de dos formas principales:

- $/N$, donde N es el número de bits asignados al Network ID. Se usa en el formato CIDR.
- El formato quad decimal (A.B.C.D) de N bits puestos a 1, seguidos de $(32 - N)$ bits puestos a 0.

Ejercicio 8.2 La máscara de red del ejemplo anterior puede expresarse en binario (aunque no es muy cómodo):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10 bits										32 - N = 22 bits																					

- Expresa la máscara de red del ejemplo anterior en los formatos CIDR y cuádruple decimal.
- ¿Cuál es tu IP y tu máscara de red? Exprésalas de esas mismas dos formas principales.



Las máscaras de red también se usan en IPv6, pero con un valor máximo de N de 128 en lugar de 32.

8.1.3 Bloques de direcciones IP

Las máscaras de red se usan combinadas con direcciones IP para definir bloques de direcciones IP que empiezan por los mismos N bits (donde N es la máscara de red). Por ejemplo, 192.168.3.0/24 (en formato CIDR) y 192.168.3.0/255.255.255.0 se refieren al bloque de direcciones entre 192.168.3.0 y 192.168.3.255, ambas incluidas (192.168.3.* en la notación informal de la tabla anterior).

Ejercicio 8.3 Los bloques IP pueden ser menores, iguales o mayores que una red IP. Da ejemplos en los que un bloque IP se refiera a:

- Una única IP.
- La mitad de las direcciones IP de la red 192.168.3.0/24.
- Todas las direcciones de la red 192.168.3.0/24, más 256 direcciones adicionales.
- Todas las IP posibles.

Ejercicio 8.4

- ¿Puede expresarse toda máscara de red como una dirección IP?
- Enumera todas las máscaras de red posibles.

Cuando nos referimos a un bloque de direcciones (como una LAN), la dirección IP de esta combinación IP + máscara de red debe tener todos los bits de Device ID puestos a 0. Así, 192.168.3.0/24 se refiere correctamente a una red, pero 192.168.3.1/24 se refiere en cambio a un dispositivo (el que tiene la IP 192.168.3.1) de esa red (192.168.3.0/24).

8.1.4 LANs y máscaras de red

Las máscaras de red *no* tienen por qué ser múltiplos de 8. Considera el siguiente ejemplo en el que

192.168.76.1/19 (la IP + máscara de red de un dispositivo) se usa para obtener su dirección de red (192.168.64.0/19):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
192								168								76								1							
1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	1
& &																															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
= =																															
1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
192								168								64								0							

Device's IP

Netmask

Network IP

} Device's IP

} Netmask

} Network IP

Ejercicio 8.5

- En el ejemplo anterior, en realidad solo necesitábamos transformar un valor decimal a binario, y un valor decimal a binario. ¿Cuál?
- ¿Es esto siempre cierto al convertir combinaciones IP+máscara de red en direcciones de red?
- Expresa la máscara de red en formato quad decimal.

Dado un bloque de direcciones IP + máscara de red y la dirección IP de un dispositivo, es inmediato determinar si la IP de ese dispositivo pertenece al bloque (si y solo si coinciden los primeros N bits, donde $/N$ es la máscara de red en formato CIDR).

Ejercicio 8.6 Determina si cada una de las siguientes direcciones pertenece a la misma red que el siguiente dispositivo **123.45.67.89/14**:

- **23.45.67.89**
- **123.45.203.25**
- **123.43.255.255**
- **0.45.67.1**

Una red IP (una LAN) es un bloque de direcciones, pero no todas esas direcciones pueden asignarse a dispositivos. Siempre hay dos direcciones reservadas:

- La dirección con todos los bits de nodo puestos a **0**: identifica la propia LAN.
- La dirección con todos los bits de nodo puestos a **1**: identifica el broadcast (difusión) dentro de la LAN.

Ejercicio 8.7

- Enumera todas las direcciones que pueden asignarse a dispositivos en **192.168.54.0/29**.
- Indica la dirección de broadcast de esa red.

La máscara de red $/N$ determina el número de dispositivos que pueden estar conectados al mismo tiempo a una red: $2^{32-N} - 2$. Cuando se dice que una LAN es "pequeña" o "más grande", el tamaño se refiere a ese número de direcciones de dispositivo disponibles.

A la inversa, el número M de dispositivos que necesitamos conectar a la misma LAN determina el tamaño de LAN más pequeño (la máscara de red $/N$ más larga) en el que caben todos esos dispositivos a la vez:

Máscara de red $/N$	Direcciones totales 2^{32-N}	Dispositivos máx. $2^{32-N} - 2$
$/30$	4	2
$/29$	8	6
$/28$	16	14
$/27$	32	30
$/26$	64	62
$/25$	128	126
$\vdots$	$\vdots$	$\vdots$



Todos los routers (encaminadores) necesitan una dirección IP válida por cada LAN a la que están conectados. Por tanto, los routers *sí* cuentan para el número máximo de dispositivos conectados en una LAN.

Ejercicio 8.8

- ¿Cuál es el tamaño mínimo de red en el que caben 256 dispositivos conectados?
- ¿Cuál es la máscara de red $/N$ más larga que permitiría conectarse a todo el mundo en tu ciudad?
- ¿Cuántas direcciones contiene una red $/0$?
- ¿Cuántas redes $/0$ pueden existir? ¿Reciben algún nombre?

Ejercicio 8.9 El siguiente fragmento de código toma una dirección IP y muestra sus 4 bytes expresados en cuádruple decimal y en binario. Amplía (o sustituye) el código para que puedas:

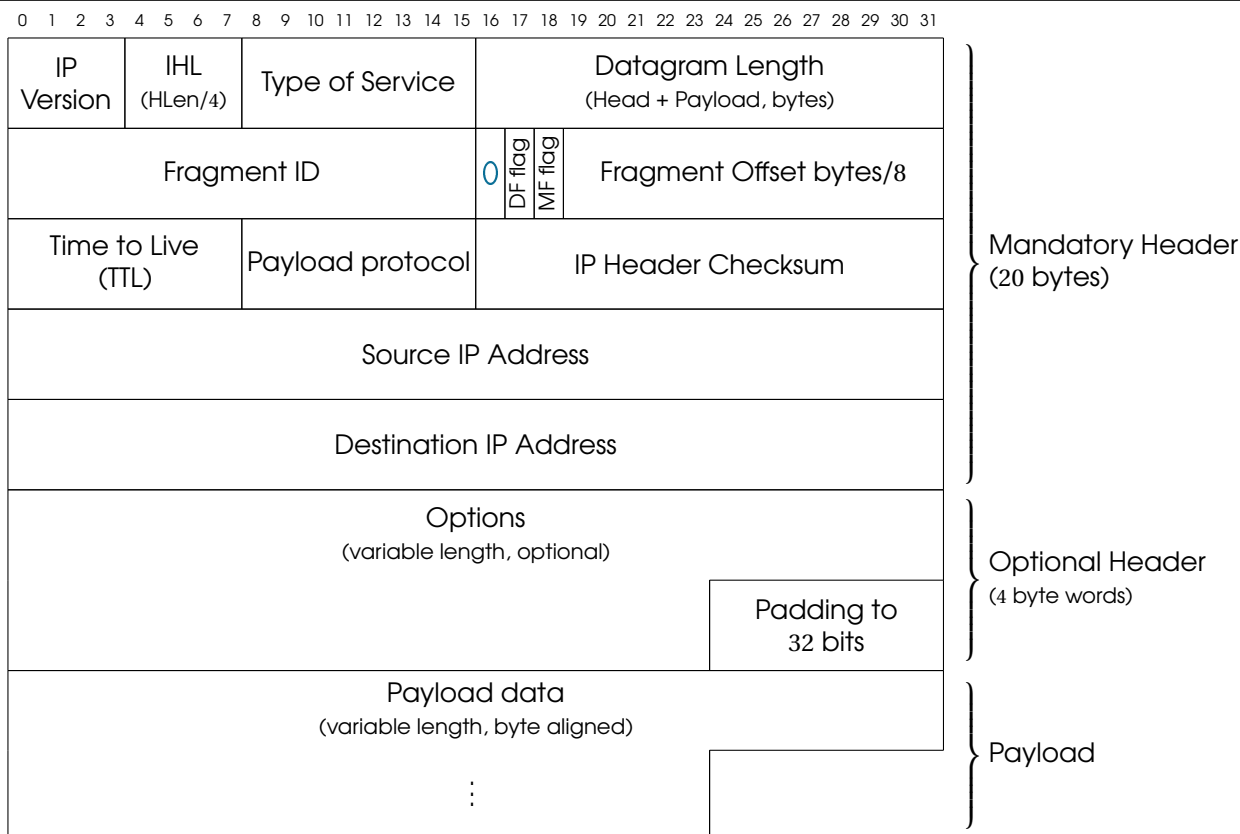
- Transformar 32 bits en una cadena en cuádruple decimal.
- Mostrar una máscara de red en formato de dirección IP y en formato CIDR.
- Dada la IP y la máscara de red de un dispositivo, determinar si otra IP pertenece a la misma red.

```
  snippets/ipcalculator.py  
1 def print_ip_binary(ip: str):  
2     parts = [int(s) for s in ip.split(".")]  
3     print(" ".join(f"{part:8d}" for part in parts))  
4     print(" ".join(f"{part:08b}" for part in parts))  
5  
6 print_ip_binary(ip = "192.168.0.1")
```

8.2 IP – Internet Protocol

8.2.1 Formato del paquete

IP define un formato de paquete con una cabecera de *al menos* 20 bytes. Pueden incluirse partes opcionales, siempre que el tamaño total de la cabecera sea múltiplo de 32 bits (4 bytes). El payload puede estar vacío, así que el tamaño mínimo de un datagrama es de 20 bytes.



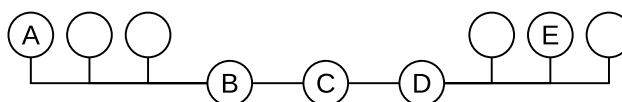
Ejercicio 8.10

- ¿Cómo puede calcularse la longitud del payload a partir de la cabecera?
- ¿Cuál es la longitud máxima del payload?
- ¿Qué valores pueden tomar los flags de la cabecera IP?
- ¿Qué es más largo, una dirección IP o una dirección MAC de Ethernet?
- ¿Es posible enviar exactamente 17 bits de datos de payload en un datagrama IP?
- ¿Cómo podemos saber si un datagrama encapsula datos TCP o UDP?
- ¿Por qué la parte opcional de la cabecera debe ser múltiplo de 32 bits?

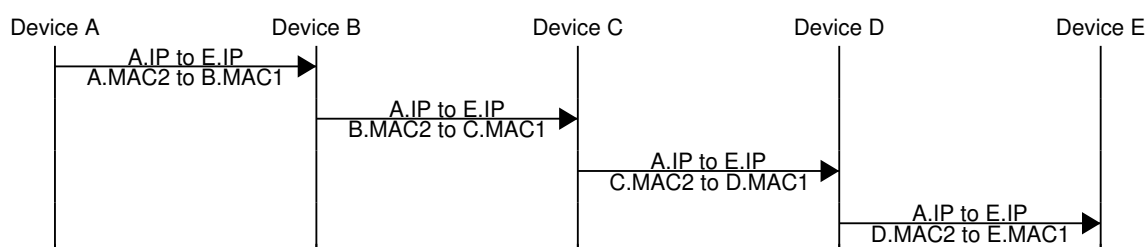
8.2.2 Funcionamiento

Cuando un dispositivo A quiere enviar un datagrama IP a un dispositivo B, las IP de origen y destino de la cabecera son fijas y no cambian para ese datagrama.

Si los dispositivos A y B pertenecen a la misma LAN, el datagrama se envía directamente (p. ej. encapsulado en un frame (trama) Ethernet). Sin embargo, IP puede llegar mucho más lejos. Considera el siguiente escenario, en el que A quiere comunicarse con E.



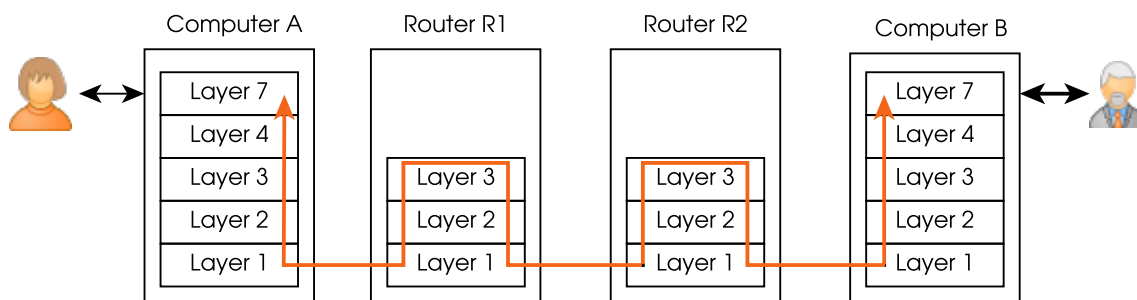
Aquí, los dispositivos de origen y destino pertenecen a LANs distintas. El datagrama se pasa primero al router del origen. Después, cada router intermedio lo pasa al siguiente hasta que el datagrama llega al último router, que está conectado a la LAN de destino. Por último, ese último router entrega el datagrama al destino.



Las direcciones IP de origen y destino no cambian a lo largo de todo el trayecto. En cambio, las direcciones MAC cambian en cada salto de LAN.

8.3 Encaminamiento IP

Los routers son los dispositivos encargados de reenviar los datagramas IP mirando la dirección IP de destino de cada uno, presente en la cabecera IP. Los routers no necesitan desencapsular las cabeceras de la capa 4 y superiores. Por tanto, sus pilas de red solo necesitan llegar hasta la capa 3 (y por eso a veces se les llama dispositivos de capa 3).

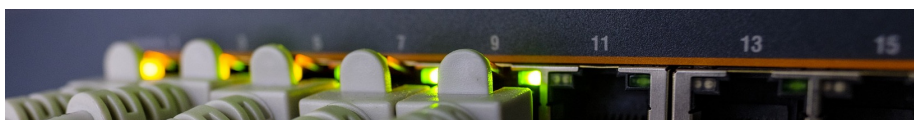


Cuando un router detecta un frame de red F en una de sus interfaces (tarjetas de red), intenta reconocer las siguientes partes de un datagrama.

Layer 2 header	IP header	Datagram payload
----------------	-----------	------------------

Si lo reconoce, el router procesa el datagrama de la siguiente manera:

1. Se descarta el frame F completo salvo que la MAC de destino (en la cabecera de capa 2) sea la de la interfaz del router que ha detectado el frame.
2. El router mira la dirección IP de destino (en la cabecera IP de F) y su tabla de enrutamiento para decidir el siguiente salto del datagrama, es decir, el dispositivo al que este router le pasará el datagrama, y qué interfaz usar (véase la Sección 8.3 más abajo).
3. El router crea y envía un frame F' haciendo los dos cambios siguientes sobre F :
 - a) La cabecera de capa 2 de F' cambia por completo:
 - La MAC de destino es la del siguiente salto, y la MAC de origen es la del router que reenvía el datagrama. La IP del siguiente salto la da la tabla de enrutamiento (véase la Sección 8.3), y su MAC se obtiene a partir de esa IP mediante ARP (Sección 8.6).
 - La MAC de origen del frame F' es distinta de la MAC de destino del frame F recibido si el router recibió F por una interfaz (p. ej. `eth7`) y lo reenvía por otra (p. ej. `eth9`).



- b) La cabecera IP de F' difiere de la de F en lo siguiente:
 - El campo TTL (tiempo de vida) se decrementa en 1. Si el TTL llega a 0, el datagrama se descarta en lugar de reenviarse.
 - Cuando el campo Options está presente, a menudo se actualiza.
 - El checksum (suma de verificación) de la cabecera se actualiza en consecuencia.



Fíjate en cómo las direcciones IP y el payload se copian sin modificar cuando se reenvía un datagrama.

Cuando un router recibe un datagrama IP para reenviarlo y ese router está conectado a

la misma LAN que la dirección IP de destino, el router realiza una entrega directa (DD). En caso contrario, el datagrama se reenvía al siguiente router en una entrega indirecta (ID). Un router sabe si debe realizar DD o ID, así como qué dirección IP de destino (y de origen) usar, gracias a su tabla de enrutamiento.

Todas las tablas de enrutamiento tienen al menos las siguientes columnas:

- **Bloque de direcciones de destino:** una o más direcciones IP que pueden expresarse como una combinación IP + máscara de red. Una fila de la tabla solo puede aplicarse a un datagrama si su IP de destino está dentro del bloque de direcciones de destino de esa fila:
 - Si solo se aplica una fila de la tabla, esa fila se usa para el encaminamiento.
 - Si se aplica más de una fila a un datagrama, se usa la fila con el bloque de direcciones de destino más pequeño (la más específica).
 Un bloque de destino default indica el bloque `0.0.0.0/0`. Como este es el bloque más grande posible, solo se aplica si no se aplica ninguna de las otras filas. La entrada default no es obligatoria.
- **Gateway/via** (pasarela): si la fila corresponde a una entrega indirecta, esta columna contiene la dirección IP del siguiente router. En caso contrario, esta columna contiene `0.0.0.0` para indicar una entrega directa.
- **Interfaz:** nombre interno de la interfaz de red (p. ej. de la tarjeta Ethernet) que se usará para entregar el datagrama. Los sistemas operativos pueden usar la convención que quieran, siempre que puedan identificarse de forma única en la tabla. Los nombres habituales empiezan por `eth`, `wlan`, `atm`, y más recientemente `en` y `wl`.



Todos los dispositivos tienen su propia tabla de enrutamiento, no solo los routers.

Ejercicio 8.11 A continuación se presenta la tabla de enrutamiento de un router pequeño.

Bloque de direcciones de destino	gateway/via	Interfaz
<i>default</i>	192.168.0.1	eth0
192.168.0.0/24	0.0.0.0	eth0
172.16.0.4/32	0.0.0.0	eth1
10.49.4.0/28	10.49.4.1	eth2
10.49.4.3/32	10.49.4.2	eth2

- Dibuja un mapa de red coherente con la tabla de enrutamiento, que contenga este router. Para las interfaces del router, indica su nombre y unas direcciones IP y de capa 2 válidas.
- ¿Cómo reenviaría este router un datagrama dirigido a una dirección IP pública?
- Da una dirección IP de destino que se procesaría usando la fila `10.49.4.0/28`, y otra que se procesaría usando la fila `10.49.4.2/32`. Piensa en un caso de uso para tener estas dos reglas.

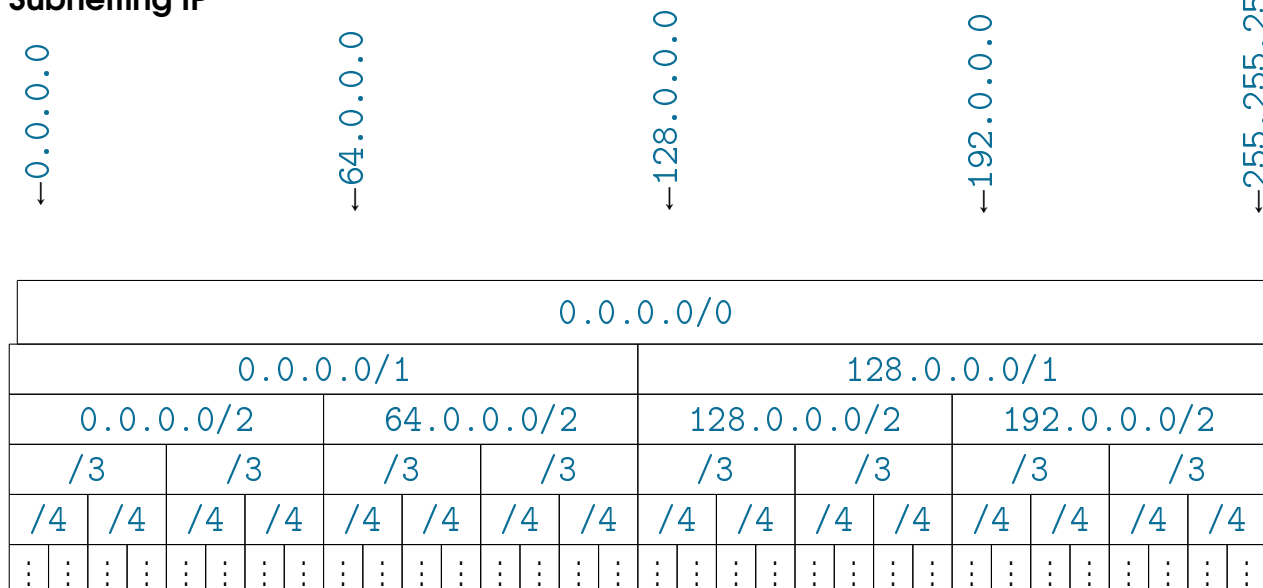
Para que Internet funcione, los routers que interconectan LANs a escala global deben estar bien configurados. A veces no es así, lo que puede provocar errores de encaminamiento. Dos problemas importantes que uno puede encontrarse son:

- “Network is unreachable”: ninguna de las filas de la tabla de enrutamiento usa un bloque de direcciones que contenga la dirección de destino del datagrama. El router no puede saber cuál debería ser el siguiente salto, así que el datagrama no se reenvía. Esto provoca un error “ICMP Destination Unreachable” (véase la Sección 8.7).
- “Time to Live exceeded”: cada vez que se reenvía un datagrama, su TTL se decrementa, como se mencionó en la Sección 8.2.1. Si varios routers están configurados de forma que el datagrama siga algún tipo de ciclo (bucle), el datagrama se reenviaría infinitamente. Cuando el TTL llega a 0, el datagrama se descarta y se produce este error. Esto provoca un error “ICMP Time Exceeded” (véase la Sección 8.7).

Ejercicio 8.12 ¿Puede producirse el error “Network unreachable” si un router tiene una entrada *default*?

Ejercicio 8.13 Propón un ejemplo de LANs interconectadas y especifica las tablas de enrutamiento necesarias para que un datagrama siga un ciclo y produzca el error “TTL exceeded”.

8.4 Subnetting IP



Las LANs pueden considerarse bloques de direcciones (p. ej. expresados en formato CIDR) con dos direcciones reservadas. A las redes accesibles directamente a través de Internet se les debe asignar un bloque válido lo bastante grande para conectar todos los dispositivos necesarios. Para asegurarse de que no haya dos dispositivos en Internet con la misma IP pública, no puede haber dos LANs que contengan la misma dirección (a menos que la dirección sea privada).

Las 2^{32} direcciones disponibles en IPv4 se dividen en bloques mediante el enmascaramiento de red. Los bloques grandes pueden dividirse por la mitad aumentando en 1 la longitud de la máscara de red, como se ve en la figura anterior. Por ejemplo, el bloque inicial $0.0.0.0/0$ (todas las direcciones) puede dividirse en dos bloques $/1$, cada uno con la mitad de las direcciones IPv4 (2^{31} – todavía demasiado grande para cualquier LAN real). Este proceso continúa de forma independiente para cada uno de los bloques resultantes, y se detiene cuando el tamaño de LAN resultante es el deseado.

Dado un bloque de direcciones $/N$ (p. ej. el de una única LAN existente), puede que queramos dividirlo en subbloques y tener en su lugar varias LANs más pequeñas (subredes). En las subredes solo podemos usar direcciones del bloque original, pero se pueden reservar bloques para un uso futuro:

Ejercicio 8.14 Eres el administrador de la red $192.168.0.0/24$ de un laboratorio de investigación. Te gustaría dividirla en las tres subredes resaltadas más abajo.

- Proporciona un diagrama de las tres redes, incluyendo un router conectado a las tres mediante tres interfaces distintas.
- Proporciona la tabla de enrutamiento *mínima* de ese router (estas redes no están conectadas a Internet).
- ¿Cuántos dispositivos pueden conectarse en total a estas tres redes, incluidos los routers?
- ¿Cuántas direcciones de la red original quedan sin asignar?

- ¿Cuál es la LAN más grande que podemos crear con las direcciones sin asignar?

192.168.0.0/24							
192.168.0.0/25				192.168.0.128/25			
192.168.0.0/26	192.168.0.64/26			192.168.0.128/26	192.168.0.192/26		
/27	/27	/27	/27	/27	/27	/27	/27

El proceso de crear varias LANs más pequeñas a partir de un único bloque de direcciones se denomina **subnetting** (división en subredes). El proceso inverso, combinar varias LANs en una mayor, se denomina **supernetting** (agregación de redes).

Dos LANs $/N$ pueden combinarse si y solo si sus direcciones de red (su IP sin máscara de red) comparten los primeros $N - 1$ bits. Este proceso puede continuar hasta llegar a una red $/0$, siempre que poseamos las direcciones IP de ambas mitades de cada supernet.

Ejercicio 8.15

- ¿Qué otra red puedes combinar con 192.168.0.0/24 para obtener una red $/23$ N_{23} ? ¿Cuál es su dirección de red?
- N_{23} forma parte de exactamente una red $/22$, N_{22} . Enumera todas las redes $/24$ que necesitarías poseer para combinarlas en N_{22} .

El **subnetting** suele basarse en cuántas direcciones IP deben estar disponibles en cada subred. Ese tamaño determina el número de bits de la máscara de red de la subred, como se vio en la Sección 8.1.4. Pueden añadirse restricciones adicionales, p. ej. hacer que las direcciones IP de una subred sean las mayores, o menores que las de otras subredes.

- 🎧 Considera por ejemplo la red 192.168.0.0/16. Queremos dividir esta red en subredes y crear la red más pequeña que pueda conectar 100 dispositivos simultáneamente.

Como se vio en la Sección 8.1.2, esto determina que, si debe crearse una subred a partir de una para albergar al menos 100 nodos, hay que dedicar al menos 7 bits de dirección ($2^6 - 2 = 62$, $2^7 = 126$) a los Device ID y los $32 - 7 = 25$ restantes a identificar la red, es decir, la subred puede ser tan pequeña como una $/25$.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
192.168																Free bits							Device ID								

Los 9 bits restantes (posiciones 16 a 24) pueden fijarse libremente si no tenemos ninguna restricción adicional. Elegir un valor de estos bits significa elegir una de las 2^9 redes de tamaño $/25$ contenidas en la red original, 192.168.0.0/16. Poner esos bits libres a 0 da el rango de IP más bajo, y ponerlos a 1, el más alto.

Cuando hay que crear varias subredes, es esencial verificar que ninguna subred está contenida en otra. En caso contrario, la misma dirección IP pertenecería a dos LANs, y el encaminamiento sería imposible. Por ejemplo, si hay que crear tres subredes para 100, 500 y 200 dispositivos a partir de 192.168.0.0/16, con rangos de direcciones en ese orden, una posible división sería:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31																
192.168																0	0	0	0	0	0	0	0	0	Device ID												→	192.168.0.0/25									
192.168																0	0	0	0	0	0	1	Device ID												→	192.168.2.0/23											
192.168																0	0	0	0	0	0	1	Device ID												→	192.168.1.0/24											

Recuerda que la dirección de red de cada subred resultante tiene todos los bits de Device ID puestos a 0, y que lo más habitual es presentar esa dirección en formato **quad decimal**.

Ejercicio 8.16

- En el ejemplo anterior, ¿por qué los bits libres de la segunda subred son 0000001 y no 0000000?
- Repite el ejercicio asignando las direcciones más altas posibles a la tercera, luego las siguientes direcciones más altas posibles a la segunda subred, y las direcciones más bajas posibles a la primera subred.
- ¿Cuál es la IP de broadcast de cada subred?

8.5 Fragmentación IP

Antes de que un dispositivo (incluidos los routers y los ordenadores normales) pueda enviar un datagrama por una LAN, debe verificar que su longitud en bytes no es mayor que la MTU de esa LAN. En caso contrario, el datagrama debe fragmentarse en dos o más datagramas más pequeños que sí quepan en la MTU de la LAN.

Los fragmentos más pequeños son datagramas completos con cabecera y payload. Por tanto, más fragmentos implican mayor sobrecarga. Cada uno de esos fragmentos se envía y se reenvía individualmente como cualquier otro datagrama, y puede que haya que refragmentarlo si atraviesa otra LAN con una MTU aún menor.

Cuando un datagrama necesita fragmentarse, la pila de red sigue los siguientes pasos:

1. Se comprueba el flag DF (don't fragment) del datagrama original. Si $DF=1$ pero se necesita fragmentación, el datagrama no se fragmenta y se produce un error.
2. Los datos de payload se dividen en trozos de tamaño como mucho S , de forma que $S + H = MTU$ (H es la longitud de la cabecera del datagrama, típicamente 20 bytes). Solo el último trozo puede tener un tamaño menor que S . Esto garantiza que ningún fragmento supere la MTU, que es precisamente el objetivo de la fragmentación.
3. La cabecera del datagrama original se copia en la de los fragmentos con las siguientes modificaciones:
 - El campo Datagram ID se fija al mismo valor en todos los fragmentos de un datagrama. El SO se asegura de elegir un ID que su pila de red no haya usado recientemente. El campo Datagram ID se ignora en los datagramas no fragmentados.
 - El campo Offset (desplazamiento) de cada fragmento indica la posición (índice de byte) de su primer byte de datos de payload dentro del payload del datagrama antes de la fragmentación. Este campo no se expresa en bytes, sino en palabras de 8 bytes. Dado un offset en bytes, el campo Offset se calcula dividiendo entre 8. Por tanto, los offsets de los fragmentos solo pueden ser múltiplos de 8.
 - El flag MF (more fragments) se pone a 1 (activado) en todos los fragmentos salvo en el último. Esto debe cumplirse incluso si uno o más fragmentos se refragmentan en una LAN posterior. Así, al fragmentar un datagrama D con $MF=1$, ninguno de los fragmentos resultantes
4. Los datagramas fragmento se envían igual que se enviaría un datagrama normal. Los fragmentos pueden refragmentarse, pero solo los reensambla el host (equipo conectado a la red) de destino.

Ejercicio 8.17 Dados el Offset y el flag MF, ¿cómo podemos diferenciar un datagrama que no ha sido fragmentado de un fragmento de datagrama?

Ejercicio 8.18 ¿Por qué sería problemático poner $MF=0$ en el último datagrama producido al fragmentar un datagrama con $MF=1$?

Ejercicio 8.19 Un dispositivo necesita enviar un datagrama con una cabecera IP de 20 bytes y una longitud de payload de 2170 bytes. El dispositivo está conectado a LAN1, y el datagrama debe atravesar LAN2 y LAN3 hasta llegar a la LAN de destino, LAN4. Las MTU de estas LANs son las siguientes:

LAN	LAN1	LAN2	LAN3	LAN4
MTU	1500	710	4096	864

- Indica el offset y la longitud del payload (ambos en bytes), así como los flags MF y DF de los 4 datagramas que llegan a LAN4 como resultado del envío del datagrama (suponiendo que no hay pérdidas ni duplicaciones).
- ¿Puede el dispositivo de origen conocer las MTU de las LANs intermedias y de destino?

Ejercicio 8.20 Los datagramas que contienen fragmentos pueden llegar duplicados y/o desordenados. ¿Cómo debería tratar la pila de red receptora estos datagramas?

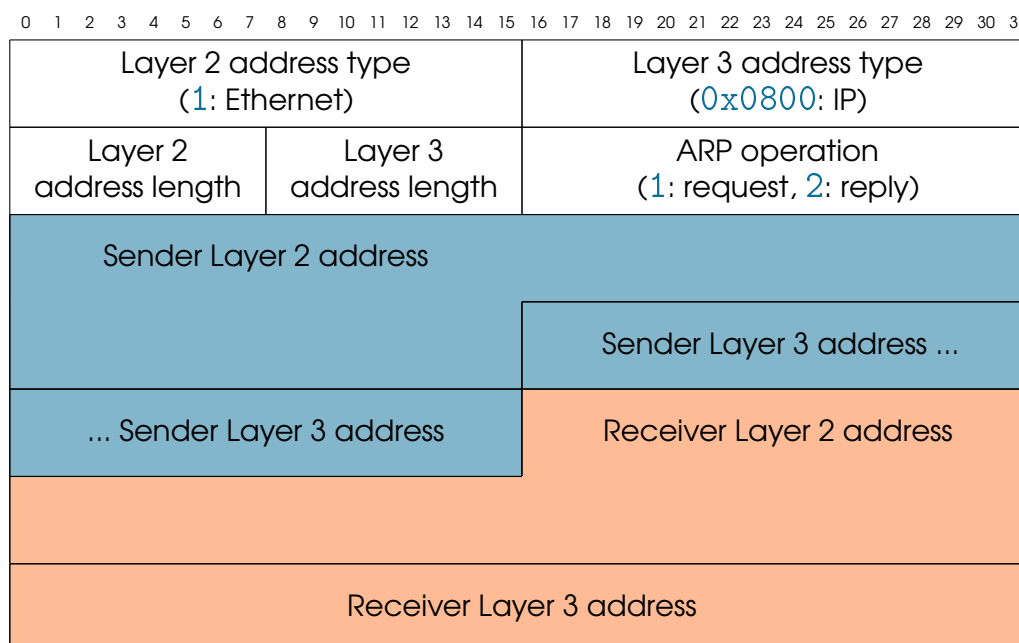
8.6 ARP – Address Resolution Protocol

El Address Resolution Protocol (ARP) lo usa la capa 3 (IP) para obtener la dirección MAC (física) de destino correspondiente a una dirección IP conocida.

Los paquetes ARP se envían como payload de frames de capa 2 (p. ej. Ethernet). Los paquetes ARP *no* son datagramas y *no* contienen cabeceras IP.

La IP que consulta la capa 3 es la del router en la entrega indirecta y la del dispositivo de destino en la entrega directa. Recuerda también que la IP de destino del datagrama no cambia durante el reenvío.

8.6.1 Formato del paquete



El protocolo ARP *no* usa cabeceras IP.

“Sender” y “receiver” deben interpretarse como “... de este mensaje ARP”.

Ejercicio 8.21

- ¿Cuál es el propósito de los 4 primeros campos de un paquete ARP?
- ¿Qué efecto tienen sobre los campos restantes?

Ejercicio 8.22 ¿Cuántas direcciones válidas aparecen en un frame Ethernet que transporta una petición ARP? ¿Y en una respuesta ARP?

8.6.2 Funcionamiento

Cuando un nodo **(A)** necesita traducir una dirección IP a una dirección MAC, envía un frame de broadcast (MAC de destino `ff:ff:ff:ff:ff:ff` que contiene un paquete ARP. En ese paquete ARP, la operación se fija a REQUEST (1), la IP de destino es la que se está traduciendo, y la MAC de destino se fija a `00:00:00:00:00:00`.

Cuando una pila de red **(B)** ve un paquete de petición ARP, envía un paquete de respuesta ARP solo si su IP configurada es la de la dirección IP de destino del ARP. Esa respuesta es idéntica a la petición con estos cambios:

- El código OP del paquete ARP se fija a REPLY (2).
- “Origen” se refiere ahora a **(B)** (en la petición se refería a **(A)**), y “destino” a **(A)**.
- Las 4 direcciones del paquete están rellenas en una respuesta (no pueden ser `00:00:00:00:00:00`).

Ejercicio 8.23 Cuando se ejecuta el comando `ip route` en un nodo, contiene:

```
default via 192.168.0.1 dev wlp5s0 src 192.168.0.2 metric 600 [...]
```

A su vez, el comando `ip link` contiene las siguientes líneas:

```
2: enp42s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 state DOWN [...]
   link/ether d8:43:ae:61:ed:a4 brd ff:ff:ff:ff:ff:ff
3: wlp5s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP [...]
   link/ether 5c:e4:3a:18:97:1b brd ff:ff:ff:ff:ff:ff
```

- El nodo necesita realizar la entrega indirecta de un datagrama, pero no conoce la dirección MAC de su router gateway. Indica el valor de todos los campos (incluyendo la cabecera Ethernet y el mensaje ARP) de la petición ARP que el nodo enviará para obtener la MAC del router.
- Describe los valores de los campos de la respuesta ARP (puedes usar cualquier dirección válida como MAC del router).

No es probable que la dirección MAC asociada a una IP cambie muy a menudo. Cuando se recibe una respuesta ARP válida, el SO actualiza una caché añadiendo (o actualizando) las traducciones IP ↔ MAC recibidas. Si el SO necesita enviar un datagrama a esa IP antes de que expire esa entrada de la caché, se usa directamente la dirección MAC de la tabla sin necesidad de mensajes ARP adicionales. El comando `ip neighbor` muestra el contenido de esa tabla, p. ej.

```
192.168.0.1 dev wlp5s0 lladdr cc:2d:14:fc:22:60 REACHABLE
```



ARP puede usarse para escanear qué IP están en uso en una LAN. En particular, un nodo puede usar ARP para verificar que su dirección IP configurada no la está usando nadie más.

ARP se propuso en una época en la que la seguridad de las redes todavía no era un problema. Como resultado, no proporciona ningún mecanismo para verificar que una respuesta ARP dada sea correcta. Esto puede aprovecharse para envenenar la tabla caché ARP de un dispositivo enviando a la víctima respuestas ARP gratuitous (no solicitadas) con asociaciones IP ↔ MAC falsas.

Los SO modernos suelen ignorar las respuestas ARP no solicitadas, y los switches (conmutadores) modernos filtran los mensajes ARP que no coinciden con las direcciones IP y MAC reales. Aun así, IPv6 prescinde de ARP en favor de un protocolo más complejo, Neighbor Discovery (ND).

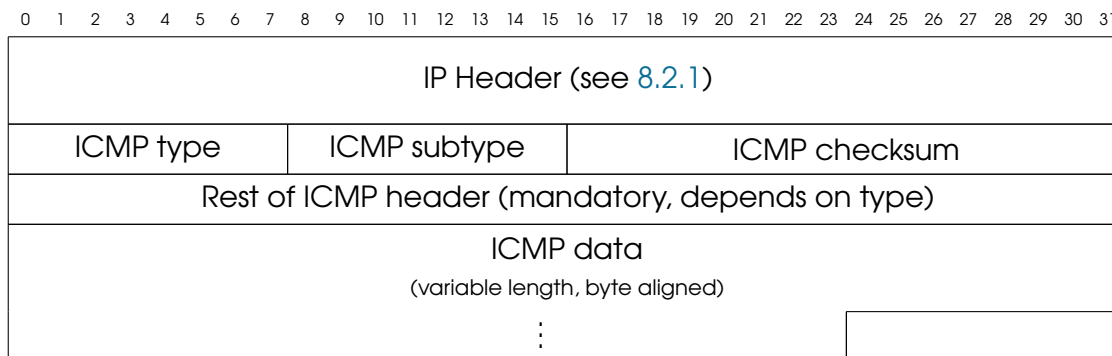
Ejercicio 8.24 ¿Qué impacto podría tener envenenar la caché ARP de una víctima, cambiando la MAC almacenada del router gateway por la MAC del atacante?

8.7 ICMP – Internet Control Message Protocol

El Internet Control Message Protocol (ICMP) es un protocolo auxiliar (técnicamente parte de IP) que se usa para probar y depurar las comunicaciones de (inter)red.

8.7.1 Formato del paquete

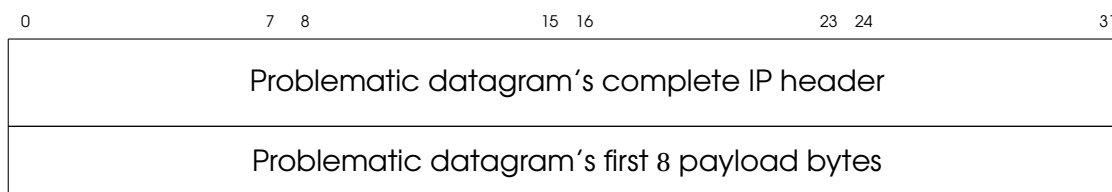
Los mensajes ICMP se encapsulan como payload de datagramas IP. En la cabecera IP, el campo Payload protocol se fija a `0x01` ("ICMP").



Las combinaciones tipo/subtipo ICMP más comunes que probablemente encuentres hoy en día:

Tipo ICMP	Subtipo ICMP	¿Error?	Significado
8	0	No	Mensaje ICMP Echo Request (ping).
0	0	No	Mensaje ICMP Echo Reply (respuesta ping).
3	0–16	Sí	Destination Unreachable. El subtipo explica la causa.
11	0	Sí	Time to Live Exceeded: el TTL ha llegado a 0.

Para estos mensajes ICMP de error, el campo de datos ICMP contiene información sobre el datagrama que provocó ese mensaje ICMP de error:



Para los mensajes echo request y echo reply, el campo de datos ICMP es arbitrario dentro de los límites de los datagramas IP. La fragmentación de datagramas ICMP (p. ej. payloads de ping grandes) es posible, pero muchos routers ignoran los fragmentos ICMP.

8.7.2 Funcionamiento

Los mensajes de error ICMP los envían los routers y los hosts normales, disparados por otros datagramas:

- La familia de mensajes "Destination unreachable" se envía cuando un datagrama no puede entregarse a su destino, p. ej. por una [tabla de enrutamiento](#) mal configurada, un puerto cerrado (véase la Sección 9), o simplemente porque el host de destino no está encendido.
- El mensaje "TTL exceeded" se envía cuando un router recibe un datagrama para entrega indirecta con TTL= 1 (véase la Sección 8.3).

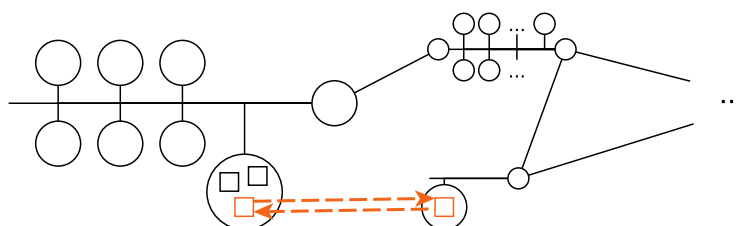
Para evitar cantidades infinitas de tráfico, *no* se producen errores ICMP cuando los datagramas causantes son:

- Datagramas ICMP (de error o no).
- Fragmentos de datagrama, salvo el primero.

Ejercicio 8.25 ¿Cómo podemos diferenciar el primer fragmento de un datagrama de los fragmentos siguientes?

9. Capa 4: Transporte

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



Capacidades

La capa de transporte comunica procesos (programas) que se ejecutan en hosts (equipos conectados a la red) potencialmente distintos. Estos procesos se identifican mediante direcciones de capa 4 llamadas puertos.

La capa de transporte ofrece dos formas distintas de comunicación: sin conexiones (UDP, orientada a mensajes) y con conexiones (TCP, orientada a flujo).

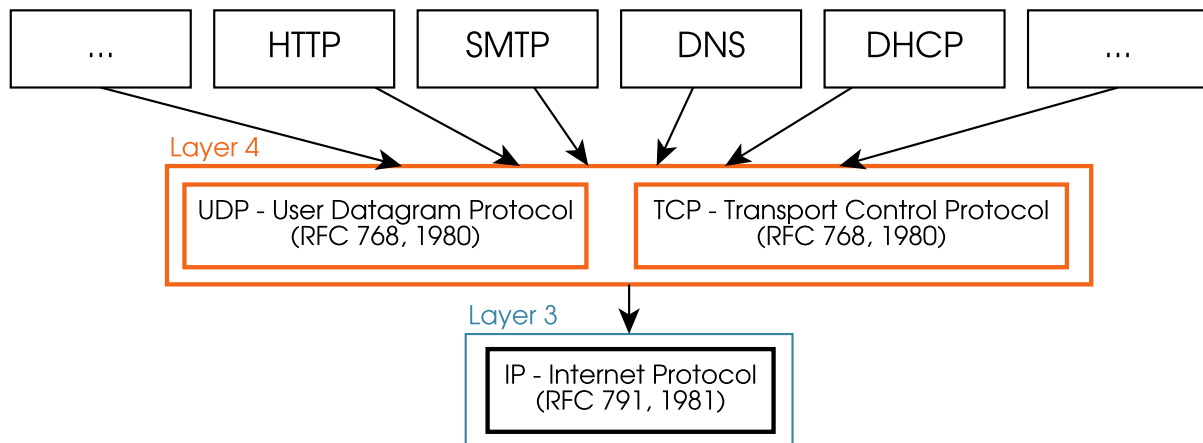
UDP no establece una conexión con el host de destino porque la prioridad es enviar unos datos de payload (carga útil) lo más rápido posible. En su lugar, UDP envía paquetes llamados datagramas de usuario directamente al destino, sin esperar ningún tipo de confirmación de recepción por parte de la capa UDP del destino. Las cabeceras UDP son simples y pequeñas, de modo que el overhead (sobrecarga) de este protocolo también se mantiene pequeño.

TCP establece una conexión antes de enviar cualquier dato de payload para asegurarse de que ambos extremos están listos. Además, TCP espera un acuse de recibo (ACK) de la recepción de cada byte de datos de payload. En caso contrario, los datos se retransmiten tras un periodo de espera, lo que le da a TCP su fiabilidad. TCP también usa una forma muy ingeniosa de limitar la velocidad de intercambio de datos (control de flujo) para que ordenadores rápidos y lentos puedan coexistir y comunicarse mediante TCP/IP. Para ofrecer todas estas prestaciones, TCP es un protocolo más complejo que UDP, usa cabeceras más grandes e introduce un mayor overhead.

Ejercicio 9.1 ¿Cuáles de las siguientes aplicaciones crees que usan UDP y TCP?

- Juegos multijugador
- La WWW
- BitTorrent
- Streaming de vídeo

Protocolos



La capa de transporte está en lo más alto de la pila que proporciona el sistema operativo. Prácticamente todas las aplicaciones y, por tanto, sus protocolos de comunicación, emplean TCP, UDP (o incluso ambos) para intercambiar datos por la red. En la figura anterior se muestran algunos protocolos importantes que usan TCP o UDP.

Tanto TCP como UDP encapsulan sus PDU en datagramas IP. Generalmente se usa IPv4, pero IPv6 también es una posibilidad cuando está disponible.



Ni TCP ni UDP cambian en absoluto cuando se usa IPv6 en lugar de IPv4, es decir, no existe un protocolo TCPv6. En su lugar, `tcp6` y otras expresiones que incluyen TCP y 6 se refieren a TCP (el de siempre) sobre IPv6.

9.1 Direccionamiento de capa 4

Las “direcciones” de TCP y UDP se llaman puertos y son enteros sin signo de 16 bits. Para las personas, los puertos se expresan principalmente de dos formas:

- Como un número, p. ej. 22, 80 o 443.
- Como el nombre del servicio que suele usar ese puerto, p. ej. `sshd`, `http`, `https`.

Cuando se recibe una PDU de capa 4, el sistema operativo usa el puerto de su cabecera para saber cuál de los procesos en ejecución debe recibir los datos. A su vez, los procesos tienen que hacer `bind` a un puerto TCP o a un puerto UDP (o a ambos) para que el sistema operativo sepa de qué puerto esperan datos.

Solo un proceso puede hacer `bind` a un puerto a la vez para enviar datos, pero los puertos TCP y UDP se tratan de forma independiente:

- Si un proceso está asociado al puerto `TCP/100`, no recibirá tráfico UDP dirigido al puerto 100.
- Si un proceso está asociado al puerto `TCP/100`, otro proceso (o incluso el mismo) puede hacer `bind` a `UDP/100`.

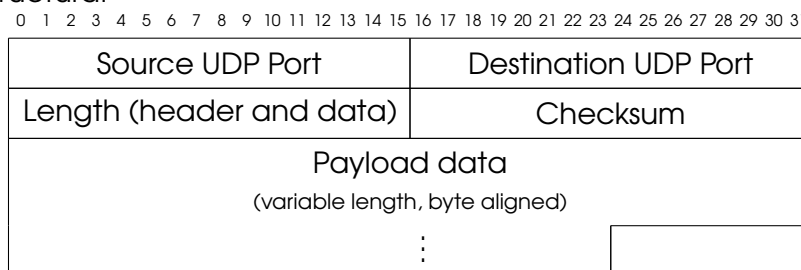
El puerto número 0 está reservado, así que el rango de puertos utilizable es `1–65535`. Los puertos `1–1023` solo se asocian a procesos con privilegios (p. ej. un servidor `http`), mientras que las aplicaciones cliente (p. ej. un cliente `http`) solo pueden hacer `bind` a los puertos `1024–65535` en función de su disponibilidad.

Ejercicio 9.2 ¿Cuántos servidores distintos se pueden ejecutar en una máquina para ofrecer alguna funcionalidad a otros hosts de la red?

9.2 UDP – User Datagram Protocol

9.2.1 Formato del paquete

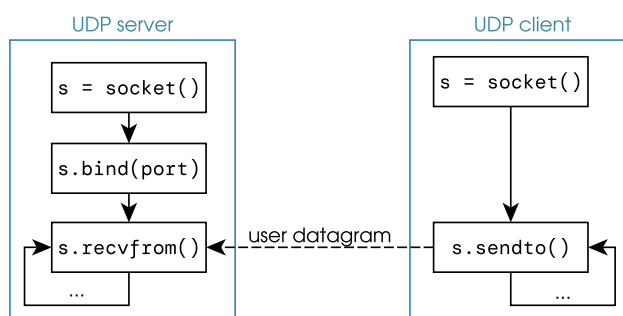
Todos los datagramas de usuario que usa UDP tienen una cabecera de exactamente 8 bytes y la siguiente estructura:



Ejercicio 9.3

- ¿Cuál es el tamaño máximo de payload que se puede enviar en un único datagrama de usuario?
- ¿Es deseable la fragmentación cuando se usa UDP?

9.2.2 Funcionamiento



Los clientes UDP crean primero un socket (punto de conexión) y después envían directamente los datagramas de usuario con `sendto`. No hace falta establecer ni cerrar ninguna conexión.

Los servidores UDP deben hacer primero bind al puerto que se va a exponer a los clientes y después recibir cada datagrama con una llamada individual a la función de lectura (`recvfrom`).

La función `bind` es puramente interna al servidor: implica cambios en su configuración (la tabla que relaciona números de puerto y procesos), pero no genera ningún mensaje.

Ejercicio 9.4

- ¿Puede funcionar un servidor UDP sin enviar ningún dato por la red?
- ¿Puede un cliente recibir tráfico UDP sin hacer `bind` a un puerto?
- ¿Pueden dos hosts distintos hacer `bind` al mismo puerto y aun así intercambiar datagramas de usuario?

Ejercicio 9.5 El siguiente fragmento de código muestra el esqueleto básico de un servidor UDP:

 snippets/udpserver.py

```

1 import socket
2
3 port = 4567
4 s = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)
5 s.bind(('', port))
6 while True:
7     user_datagram = s.recvfrom(1024)
8     print(f"Received UDP on {port=}: {user_datagram!r}")

```

- Ejecuta el servidor y, en otra terminal, ejecuta `ncat -u localhost 4567` y escribe algo de texto para probar el servidor.
- Implementa el extremo cliente de la comunicación y pruébalo contra el servidor.
- ¿Qué información importante obtienes si cambias `s.recvfrom` por `s.recvmsg`?

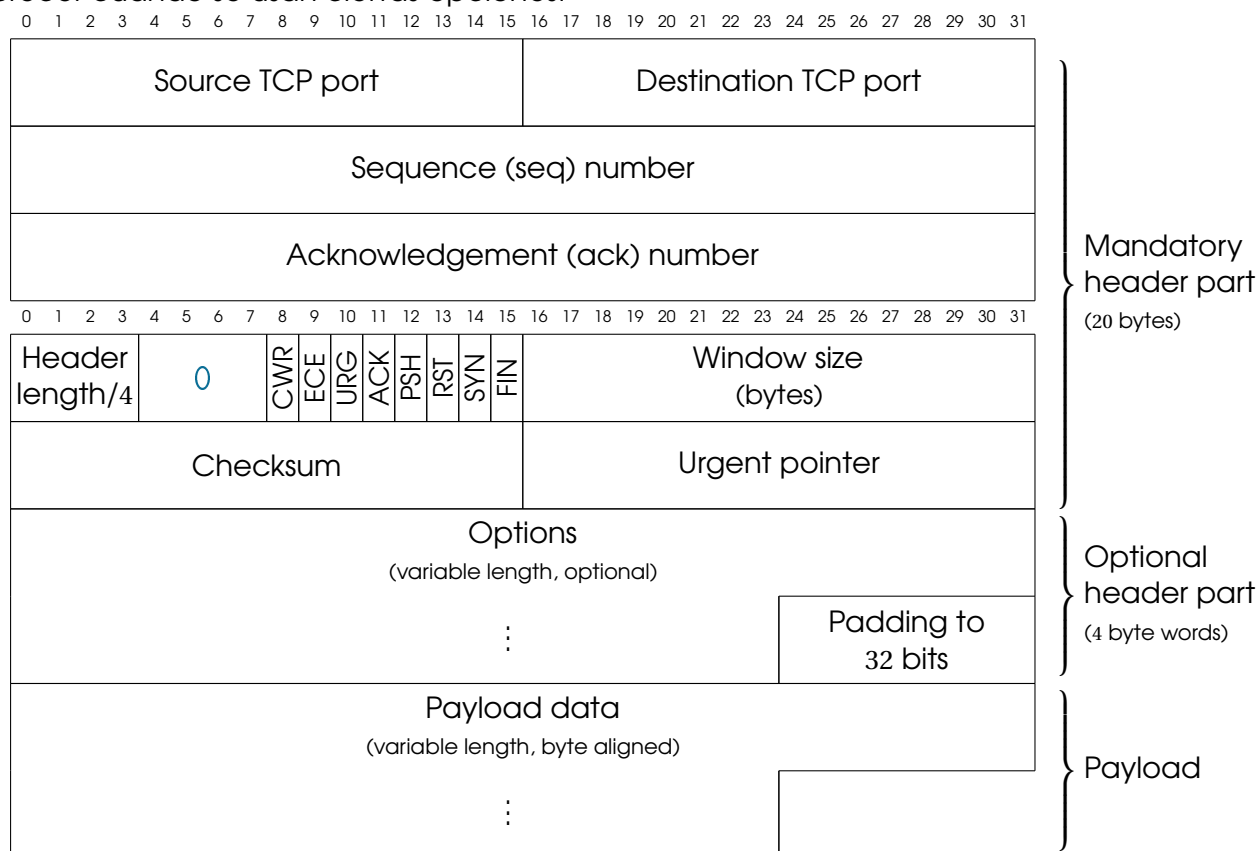


Este código usa `sockets` UDP normales, así que no necesita permisos de administrador ni la capacidad `CAP_NET_RAW`, como sí ocurría en el Ejercicio 7.4, que en cambio requería acceso a `sockets` de capa 2 (p. ej. Ethernet).

9.3 TCP – Transport Control Protocol

9.3.1 Formato del paquete

Las PDU de TCP se llaman *segmentos*. Su cabecera tiene al menos 20 bytes, pero puede crecer cuando se usan ciertas opciones:



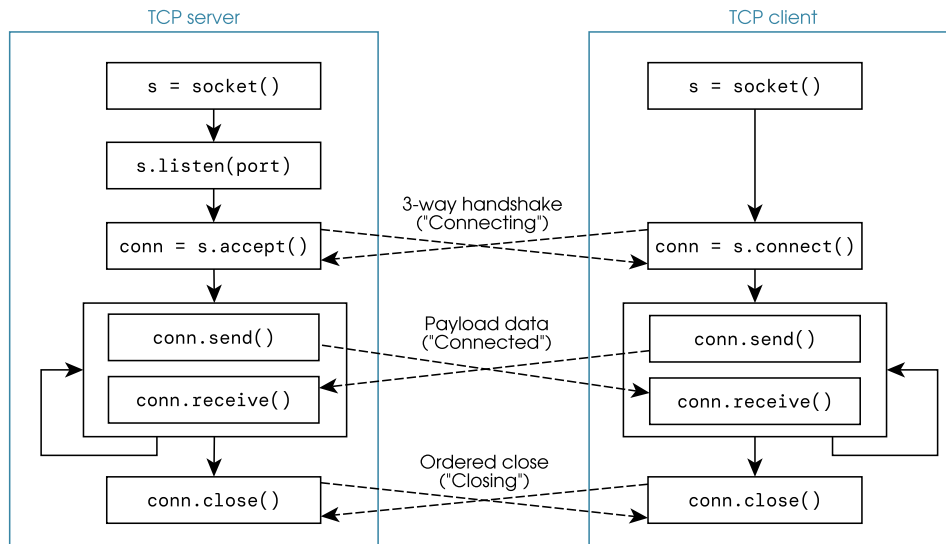
Las cabeceras TCP contienen ocho flags de 1 bit. De ellos, cuatro se usan para funcionalidades centrales de TCP que se tratan en las siguientes secciones. En el orden en que aparecen:

- Flag ACK (acuse de recibo) – “*mi* número de ack es válido”
- Flag RST (reset) – “*aborta* la conexión”
- Flag SYN (sincronización) – “estoy abriendo *mi* extremo de la conexión”
- Flag FIN (finalización) – “no enviaré más datos de *payload*”

Ejercicio 9.6 ¿Cómo puede saber el extremo receptor el tamaño de las opciones y de los datos de payload?

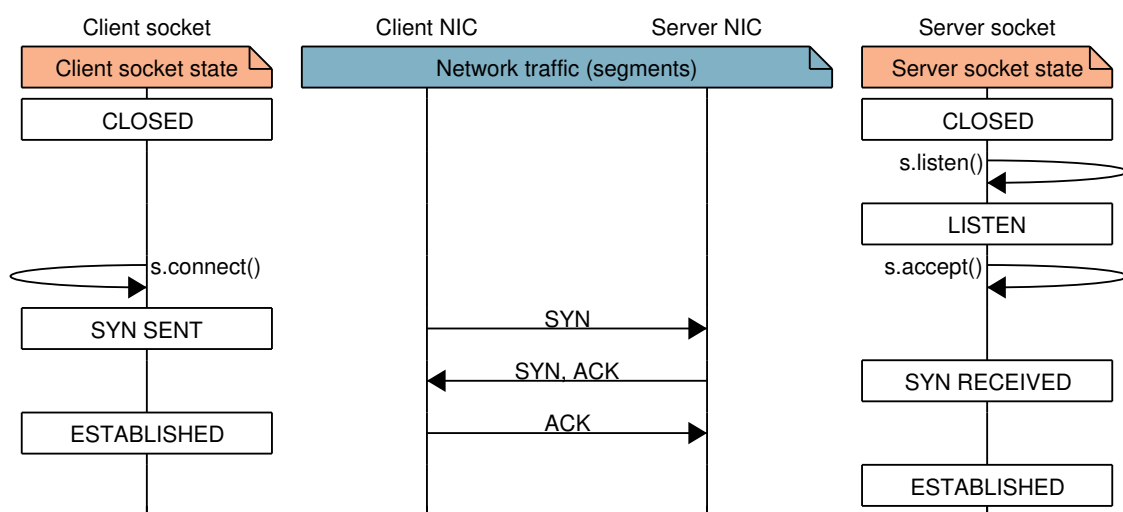
9.3.2 Funcionamiento

TCP es un protocolo stateful (con estado). Los sockets TCP transitan entre múltiples estados durante la comunicación. Cada extremo de la comunicación tiene su propio socket con su propio estado. A grandes rasgos, las conexiones se pueden clasificar como “Conectando”, “Conexión establecida” o “Cerrando la conexión”. Los datos de payload solo están pensados para intercambiarse durante “Conexión establecida”.



Los sockets TCP pueden estar en uno de 11 estados. Estos estados, junto con dos contadores de 16 bits (número de ACK (#ack) y número de secuencia (#seq)), hacen posible la fiabilidad en la comunicación que se espera de TCP. El significado de estos estados y sus transiciones se explican en los siguientes capítulos.

9.4 TCP: “Conectando”



Cuando se crea un socket, empieza como CLOSED. El extremo que actúa como servidor empieza a escuchar en el puerto configurado para el servicio y pasa al estado LISTEN. En este estado no se intercambia payload.

Una conexión se establece mediante un 3-way handshake usando los flags SYN y ACK de la cabecera. Cada extremo de la comunicación alcanza el estado ESTABLISHED en un momento distinto gracias a este handshake. Cuando lo hacen, están listos para intercambiar datos de payload.



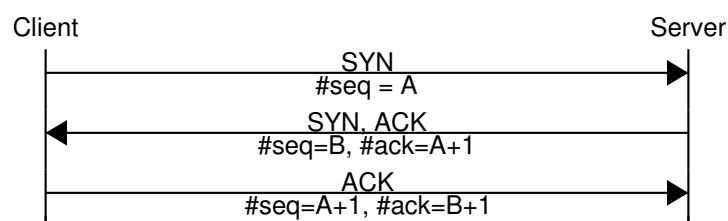
En TCP, ambos extremos se comunican como peers (pares) iguales; no importa quién inició la comunicación. Dicho esto, normalmente un extremo realiza una “apertura activa” (el cliente) y el otro una “apertura pasiva” (el servidor).

Cada extremo de la comunicación lleva la cuenta de sus contadores de número de ACK (#ack) y número de secuencia (#seq). Cada extremo elige el valor de #seq por separado (y normalmente al azar): $A, B \in [0, 2^{32} - 1]$.

Una sola vez durante la apertura de la conexión, cada extremo envía exactamente un segmento con el flag SYN activado. Estos segmentos intercambian (sincronizan) sus números de secuencia iniciales A y B .

Los segmentos con el flag ACK activado (todos salvo el primer segmento del cliente) tienen su #ack fijado a “el siguiente #seq que se espera del otro extremo”. Este #ack debe leerse como “he recibido correctamente todos los segmentos correspondientes a #ack, sin incluirlo”.

Los números #ack y #seq esperados durante el 3-way handshake son:



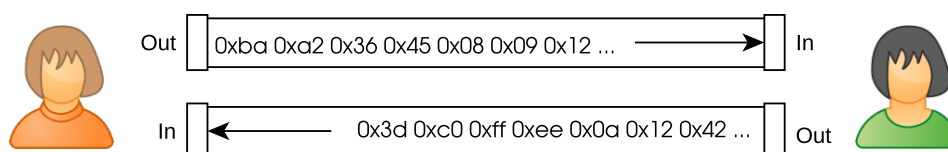
Ejercicio 9.7

- ¿Cómo puede saber un extremo que el otro ha recibido el #seq que ha elegido?
- ¿Por qué el tercer segmento tiene su #seq fijado a $A + 1$ y no a A ?
- ¿Qué #seq tendrá el siguiente segmento que envíe el servidor?

9.5 TCP: “Conexión establecida”

9.5.1 Flujos y números de secuencia

TCP ofrece la ilusión de un flujo de datos: dos tuberías de datos en los sentidos de entrada y salida (una conexión full-duplex). Estas tuberías son fiables , es decir, se establecen mecanismos para evitar la pérdida, la duplicación y el reordenamiento de los datos.



A cada byte de datos de payload se le asigna un número de secuencia (#seq) en la cabecera TCP. El primer byte de datos de payload tiene #seq = $A + 1$, donde A es el número de secuencia inicial establecido durante la apertura de la conexión (ver Sección 9.4). A cada nuevo byte de datos se le asigna el siguiente #seq (sin huecos).

#seq	A	$A + 1$	$A + 2$	$A + 3$	$A + 4$	$A + 5$	$A + 6$	$A + 7$	$A + 8$	$A + 9$	...
contenido	$\geq$	0x48	0x69	0x21	0x0a	0x48	0x6f	0x77	0x20	0x61	...
		'H'	'i'	'!'	'\n'	'H'	'o'	'w'	' '	'a'	...

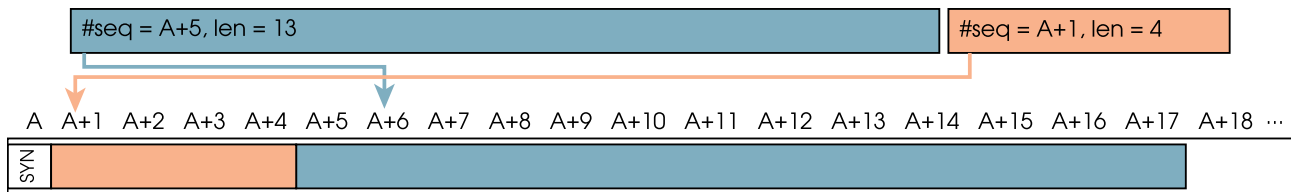
Ejercicio 9.8 El #seq que viene después de $2^{32} - 1$ es 0. ¿Por qué?

Los datos de payload en TCP siempre se envían junto con la *primera* posición que ocupan en el flujo y su longitud. Por ejemplo, a partir de los datos mostrados arriba, se podrían producir dos segmentos:

1. #seq = $A + 1$, len = 4, datos = 0x48 69 21 0a ('Hi!\n')
2. #seq = $A + 5$, len = 13, datos = 0x48 6f 77 20 61 72 65 20 79 6f 75 3f 0a

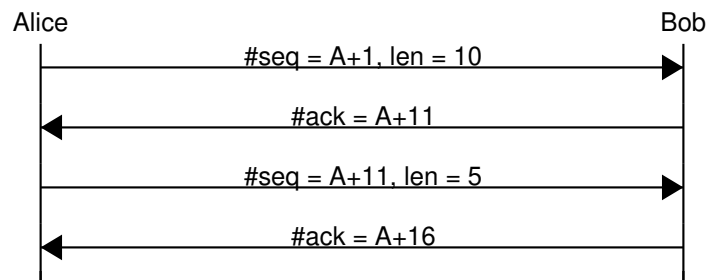
Ejercicio 9.9 ¿Cuál sería el #seq del siguiente segmento con nuevos datos de payload?

El extremo receptor de un segmento TCP usa esta información para recomponer una copia de los datos en el orden correcto, aunque los propios segmentos lleguen desordenados. Estos datos se pasan a la capa superior a través de un buffer (memoria intermedia).



9.5.2 Fiabilidad y retransmisión

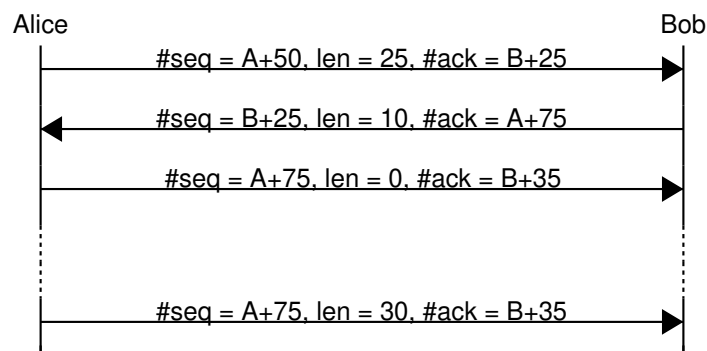
TCP necesita que todos los datos transmitidos sean confirmados con un ACK por el otro extremo. Siempre que se reciben datos en un segmento TCP, el extremo receptor devuelve una confirmación aunque en ese momento no necesite enviar ningún dato. El número de ACK ($\#ack$) de la cabecera TCP se usa para decirle al otro extremo que todos los datos hasta (sin incluirlo) $\#ack$.



El campo $\#seq$ de una cabecera TCP se refiere a los números de secuencia del emisor. El campo $\#ack$ se refiere a los números de secuencia del receptor.

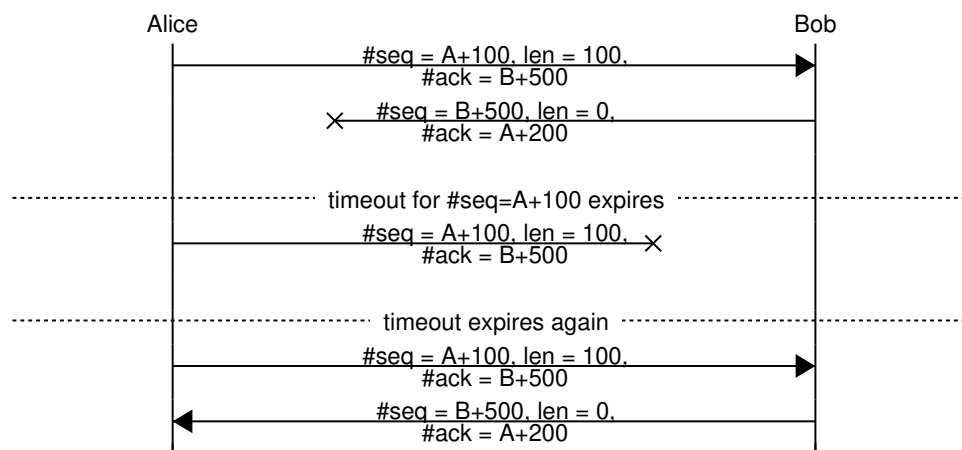
En las confirmaciones TCP se incluyen datos de payload siempre que haya datos disponibles. A la inversa, el valor más reciente de $\#ack$ se incluye siempre que se envían datos, es decir, las confirmaciones hacen piggyback (viajan a cuestas) sobre los datos de payload.

Por ejemplo, si Alice envía una petición de 25 bytes, Bob puede enviar su respuesta de 10 bytes y la confirmación de los datos de Alice en el mismo segmento TCP. Alice necesita confirmar los datos de Bob y, si hace falta, envía enseguida un segmento sin datos. Alice retomará el envío de datos más adelante cuando lo necesite.



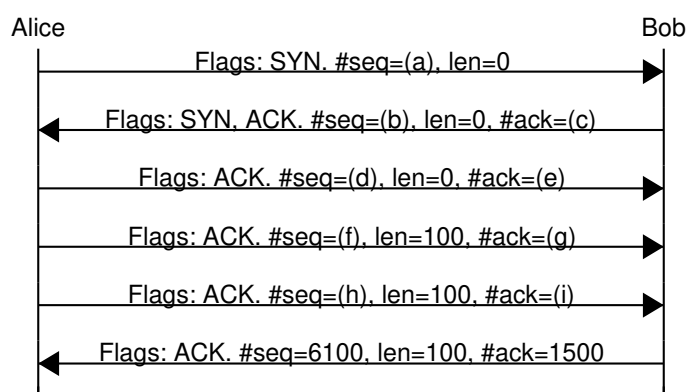
Un segmento TCP que no se confirma tras un cierto periodo de espera se retransmite. Ese tiempo lo decide el sistema operativo de cada extremo a partir del tiempo medio que tarda en enviarse un segmento y recibirse una respuesta (el tiempo de ida y vuelta, RTT).

Las confirmaciones se envían aunque esa confirmación sea idéntica a otra anterior. Esto es así porque las confirmaciones también se pueden perder.



Ejercicio 9.10 Suponiendo que ambos extremos siguen TCP correctamente:

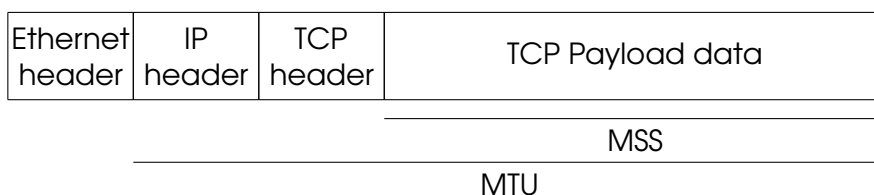
- Proporciona un conjunto de valores válidos para (a) a (i).
- ¿Cómo cambia la historia si (f) y (h) son idénticos/distintos?



9.5.3 Control de flujo

Los datos que hay que transmitir se suelen producir a ráfagas. Nos gustaría enviar todo lo posible lo antes posible, pero TCP impone dos límites.

El primer límite, el tamaño máximo de segmento (MSS), es la cantidad máxima de datos de payload por segmento permitida en una conexión TCP.



La razón de este límite es evitar la fragmentación, por motivos de rendimiento, p. ej. por el elevado coste en tiempo y memoria del reensamblado.

Por defecto, el MSS es de 536 bytes en IPv4, pero se puede negociar con las opciones de la cabecera TCP durante la fase de apertura de la conexión. En IPv6, el MSS por defecto es de 1220 bytes.

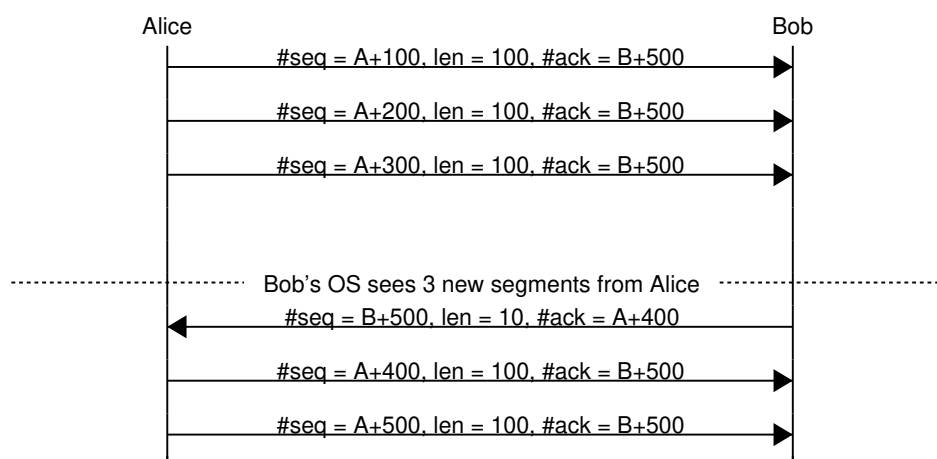
Ejercicio 9.11 Ethernet tiene una MTU de 1500 bytes. Suponiendo que los datagramas viajan solo por 2 redes Ethernet:

- ¿Cuál es el mayor MSS que no puede provocar fragmentación, MSS_{max} ?
- ¿Cuál es el overhead para MSS_{max} y para $MSS_{max} + 1$?

El segundo límite afecta a la cantidad máxima de datos de payload que se pueden enviar antes de que el otro extremo responda con un nuevo #ack: el llamado tamaño de la ventana de transmisión. Un host no enviará datos (por muchos que haya disponibles o por urgentes que sean) hasta que lleguen esos #ack. Cuando el ordenador más lento responde, lo hace

siempre con el #ack más reciente.

Ejercicio 9.12 ¿Cuáles son los tamaños de MSS y de ventana de transmisión más probables en este ejemplo?



El valor de la ventana de transmisión se anuncia en todos y cada uno de los segmentos TCP: este valor es un campo obligatorio de las cabeceras TCP. A medida que avanza la conexión, cada host puede cambiar el valor que anuncia en función de su propia carga y necesidades.



Cada host anuncia su propio tamaño de ventana.

Este límite sirve para evitar que los ordenadores rápidos desborden a los más lentos (o sobrecargados): sin el límite, el ordenador más lento podría necesitar un buffer de tamaño infinitamente creciente para almacenar los segmentos aún no procesados que le envía el ordenador más rápido. Con este límite, en el punto de saturación ambos extremos envían segmentos al mismo ritmo, aunque tarden tiempos muy distintos en producirlos y procesarlos.

9.6 TCP: “Cerrando la conexión”

9.6.1 FIN, #seq y #ack

Una vez la conexión está ESTABLISHED, sigue así hasta que uno de los dos extremos decide cerrarla.

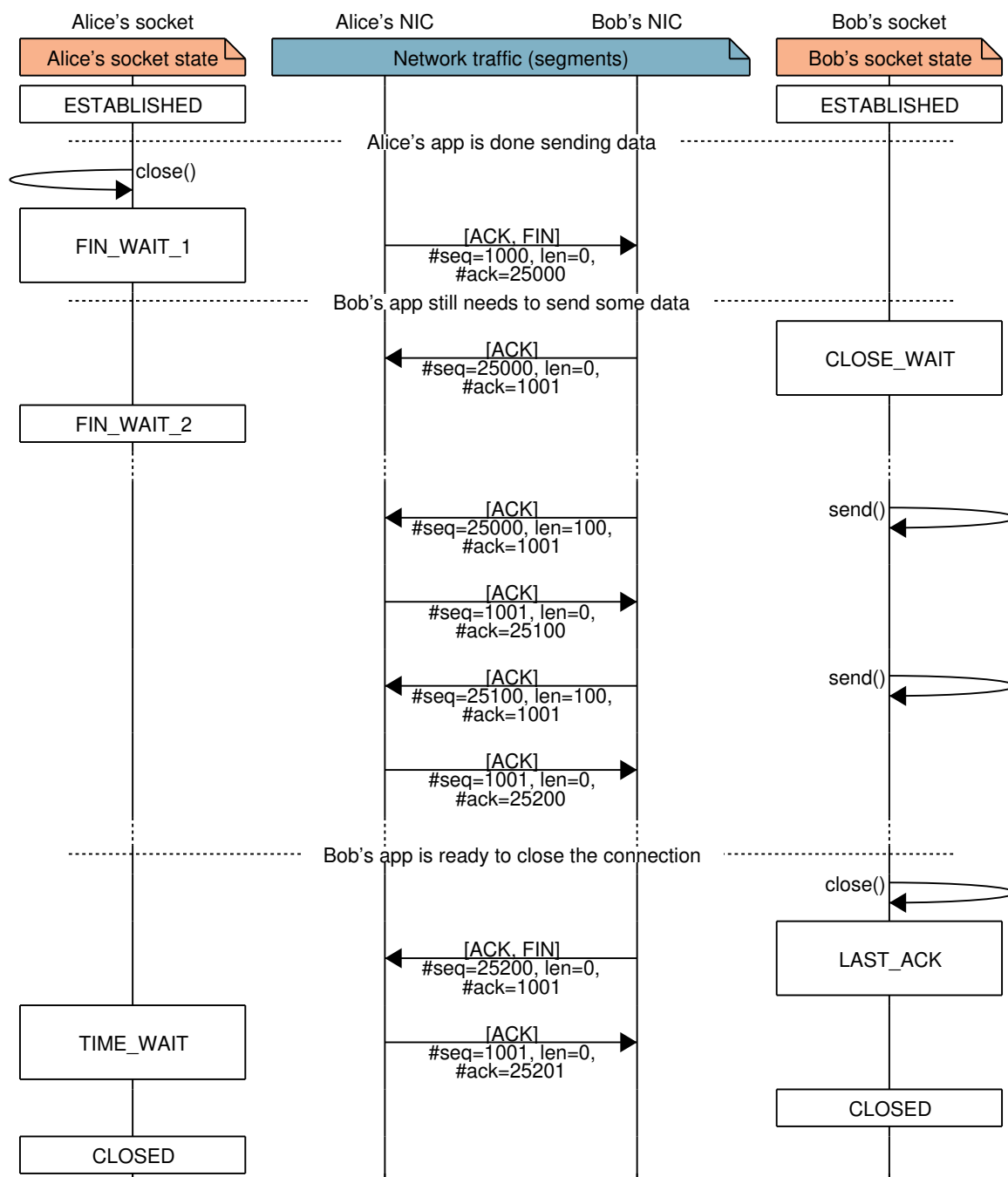
Un nodo anuncia que ha terminado de enviar datos y que quiere cerrar la conexión enviando un segmento con el flag FIN activo en la cabecera. El otro extremo puede seguir enviando datos, en cuyo caso espera respuestas con valores de #ack actualizados. El otro extremo también puede optar por cerrar la conexión en cuanto ve el primer FIN. En cualquier caso, si se envían k bytes de datos y A es el #seq inicial, entonces se espera un segmento con #ack = $A + k + 2$.

#seq	A	$A + 1$	$\dots$	$A + k$	$A + k + 1$
contenido	SYN	Datos $\dots$	$\dots$	$\dots$ datos	Σ

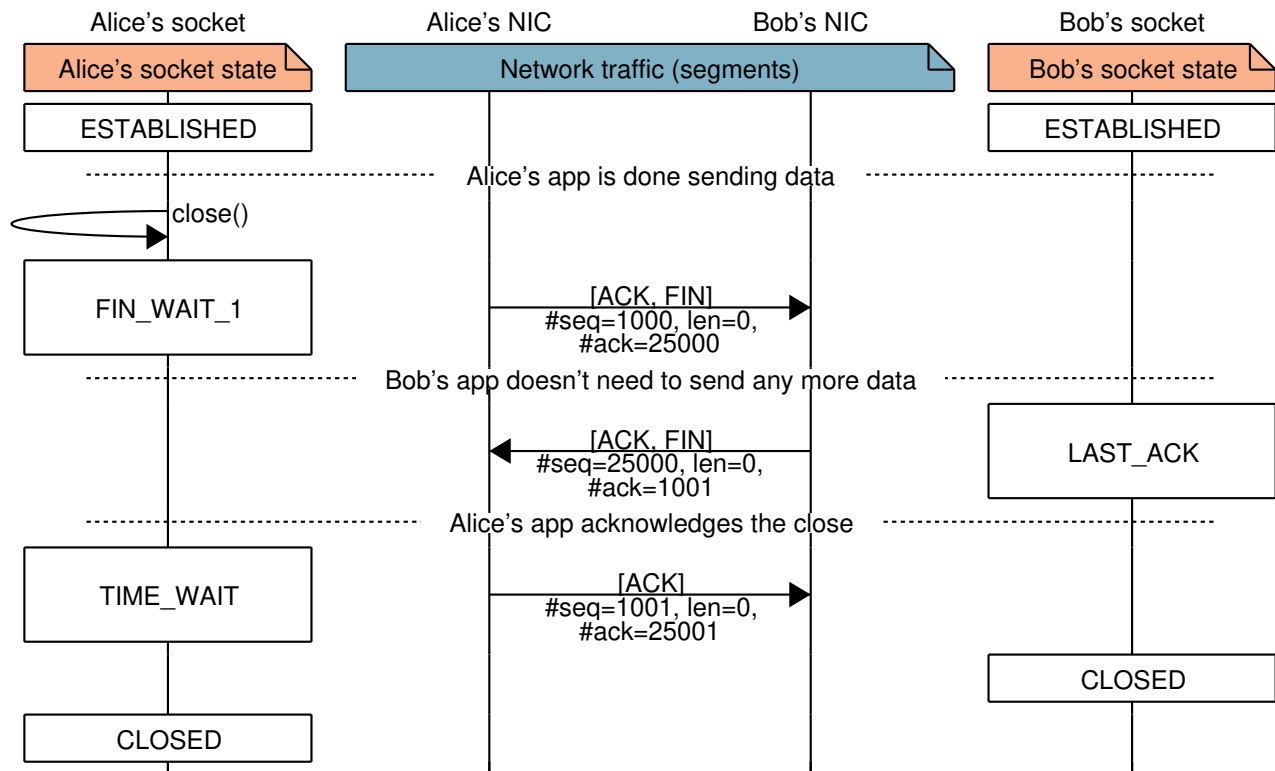
Fíjate en que el flag FIN ocupa por sí mismo un número de secuencia y debe confirmarse igual que el SYN (ver Sección 9.4), p. ej.:

9.6.2 Cierre bilateral

Después de que un extremo cierre su lado de la conexión, el otro puede seguir enviando datos indefinidamente. Cuando el segundo lado termina de enviar datos, también envía un segmento con el flag FIN activado.



Fíjate en que, si el otro extremo quiere terminar la conexión en cuanto recibe un segmento FIN, el proceso de cierre se condensa en 3 segmentos en lugar de 4, p. ej.



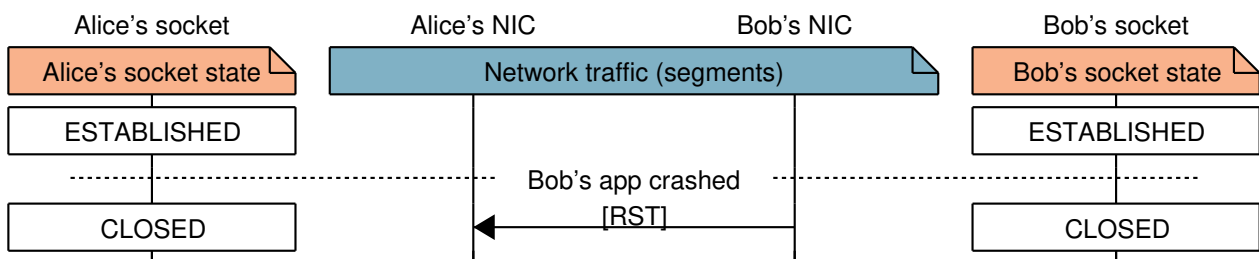
Fíjate en que:

- Cada extremo envía solo un segmento con el flag FIN, salvo en caso de retransmisión.
- El segmento con el flag FIN puede contener datos de payload. En ese caso:
 - Los datos de payload ocupan números de secuencia *anteriores* al del FIN.
 - El otro extremo debe confirmar (ACK) tanto los datos de payload como el FIN.

9.6.3 Cierre unilateral

A veces hay que terminar una conexión y no hay tiempo ni interés suficientes para pasar por el proceso de cierre bilateral de varios pasos. Puede ser el caso, p. ej. cuando un proceso se cuelga, un puerto TCP o UDP dado está bloqueado y en otras situaciones de error.

En este escenario, el host que ha detectado la situación anómala simplemente envía un segmento con su flag RST (reset) activado. Los extremos receptores no deben confirmar el RST; en su lugar, se liberan directamente los recursos de la conexión, p. ej.

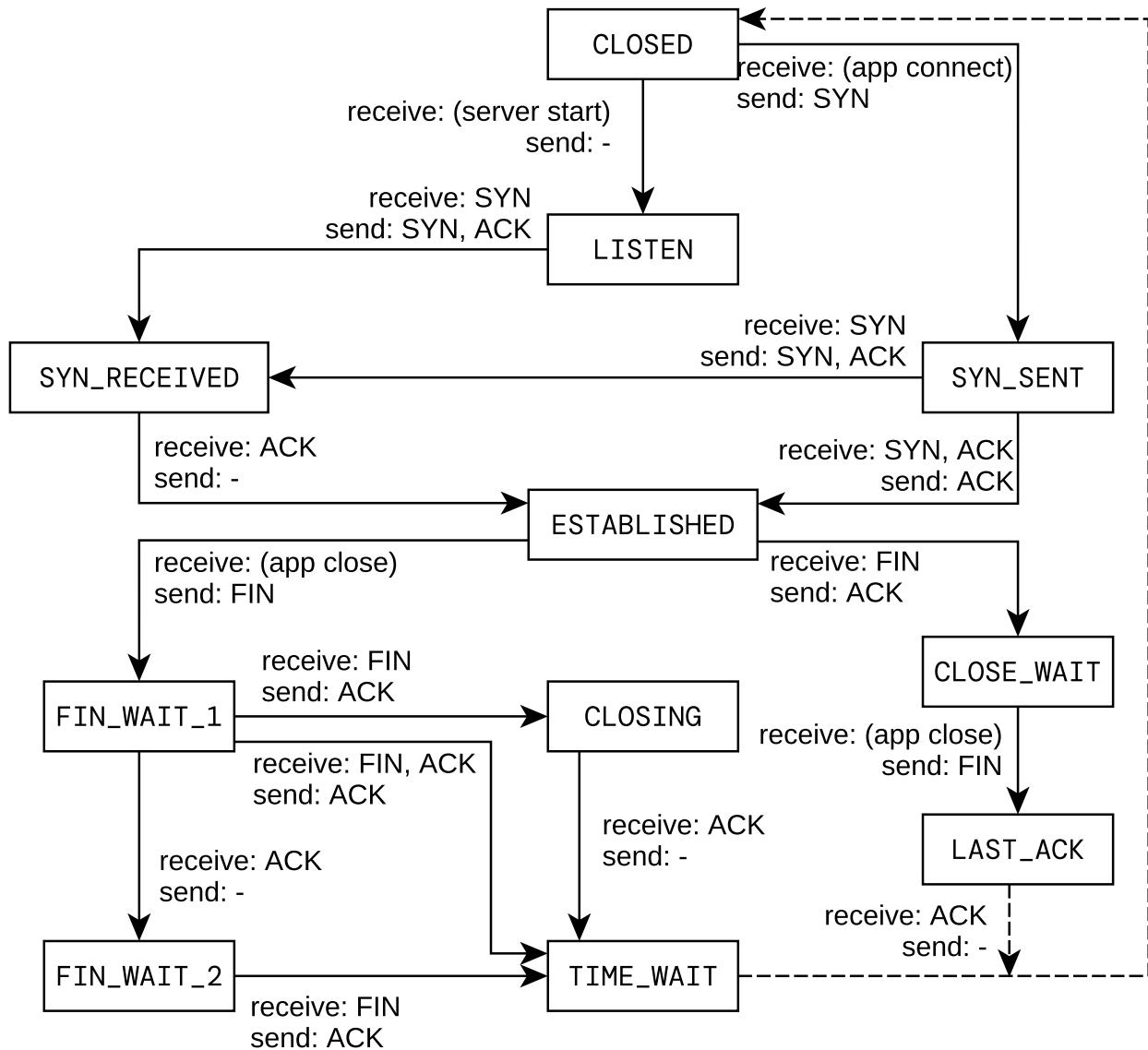


Seguir el procedimiento de cierre de conexión bilateral o unilateral ayuda a reducir los recursos de cómputo necesarios para mantener las pilas de red de miles de millones de dispositivos de todo el mundo.



Una conexión *no* se puede volver a abrir después de cerrarse. Al flag RST se le podría haber llamado, con más precisión, "terminación forzosa".

Ejercicio 9.13 El siguiente diagrama muestra los 11 estados de TCP y (casi) todas las transiciones de estado posibles.



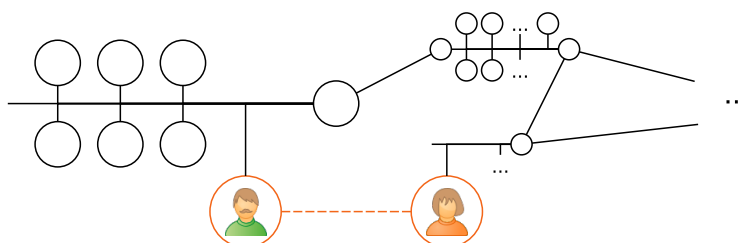
- Indica el ciclo de vida del socket de un servidor HTTP como un camino en el grafo.
- Indica el ciclo de vida del socket del cliente HTTP correspondiente como un camino en el grafo.
- El estado **CLOSING** no se describe en las secciones anteriores. ¿Cuál crees que es su propósito?



Este diagrama está adaptado de "TCP/IP Illustrated, Volume 2: The Implementation," de Gary R. Wright y W. Richard Stevens.

10. Capa 7: Aplicación

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



La capa superior de la pila de red TCP/IP incluye todas las aplicaciones que usan TCP o UDP como transporte. Es, de largo, la que contiene el mayor número de protocolos, y donde esos protocolos se crean y se reemplazan más rápidamente.

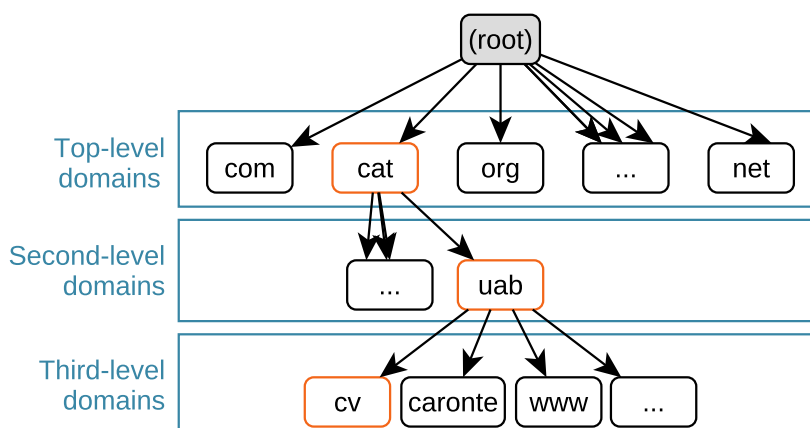
Además de las aplicaciones de usuario, existen varios sistemas y protocolos auxiliares que ayudan a configurar y operar todos los hosts (equipos conectados a la red) de Internet. En este capítulo se abordan algunos de los más críticos.

10.1 DNS – Domain Name System

El protocolo DNS se encarga de traducir nombres de dominio como ietf.org o cv.uab.cat en direcciones IP que se pueden colocar dentro de datagramas.

Los servicios DNS funcionan tanto sobre TCP como sobre UDP, es decir, se pueden habilitar ambos protocolos de transporte. En ambos casos se usa el puerto 53.

10.1.1 Jerarquía



Los nombres de dominio de Internet son secuencias de nombres separados por puntos. Estos nombres se presentan de lo específico a lo general, de forma jerárquica. Por ejemplo, en cv.uab.cat, [cv](http://cv.uab.cat) pertenece a uab.cat, y [uab](http://uab.cat) a cat. El nombre situado más a la derecha, p. ej. cat o com, debe ser uno de los dominios de primer nivel de la IANA.

Los servidores DNS se organizan de la misma forma jerárquica que esos nombres de dominio. Hay 13 servidores root DNS (Root-A a Root-M), que son responsables de todos los dominios de primer nivel. Puedes preguntar a cualquiera de ellos por com, pero también por su, y sabrán de ellos.

Eso no significa que un servidor raíz conozca todos los dominios que hay *por debajo* de su dominio de primer nivel (p. ej. uab.cat): solo conocen esos dominios de primer nivel. (p. ej.

cat). El trabajo de los servidores DNS raíz es redirigirte a otro servidor DNS encargado del dominio de primer nivel que has solicitado.

De forma similar, los servidores DNS responsables de cat conocerán cualquier dominio *.cat existente (p. ej. uab.cat), pero no sus subdominios (p. ej. cv.uab.cat). El trabajo de esos servidores DNS de dominio de primer nivel es redirigirte a otros servidores DNS autoritativos para el dominio de segundo nivel solicitado (p. ej. uab.cat). Los dominios de segundo nivel pueden entonces estructurar su zona de nombres como deseen, incluyendo cualquier número y profundidad de subdominios, así como la traducción de nombre a IP *dentro* de su zona de autoridad.

Ejercicio 10.1 Todo el mundo usa DNS constantemente. ¿Cuáles serían los riesgos de tener un sistema centralizado de traducción de nombres en lugar de la naturaleza distribuida del DNS?

10.1.2 Registros

El DNS puede verse como una base de datos distribuida con distintos tipos de entradas llamadas registros DNS. Los principales son:

A: Un registro A asocia un nombre de dominio a una dirección IPv4.

AAAA: Un registro AAAA asocia un nombre de dominio a una dirección IPv6.

NS: Un registro NS apunta a un servidor DNS que puede ayudar.

MX: Un registro MX proporciona el nombre de dominio del servidor de correo que gestiona los mensajes de un dominio.

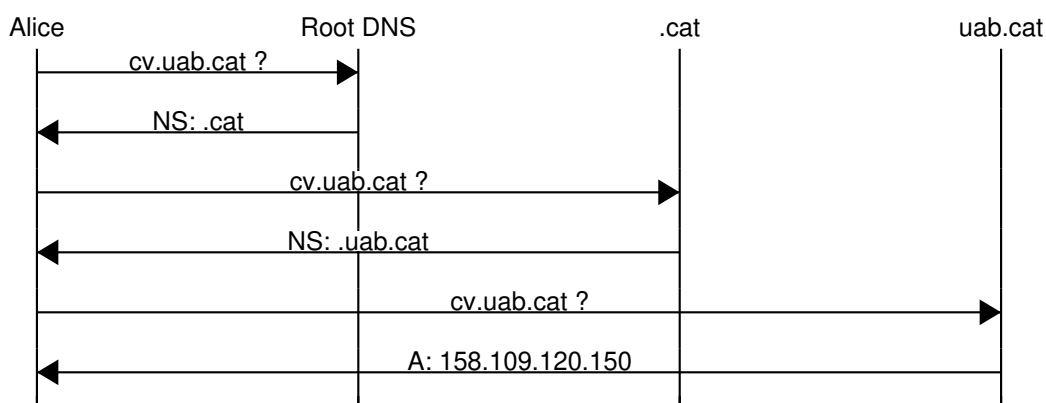
CNAME: Un registro CNAME asocia un nombre de dominio a otro nombre de dominio.

Las aplicaciones hacen peticiones a los servidores DNS indicando el tipo de registros que les interesan. Las respuestas del servidor pueden ser para los registros A o MX solicitados, pero el cliente también puede recibir una redirección a otro DNS (NS) o un alias a otro nombre de dominio (CNAME).

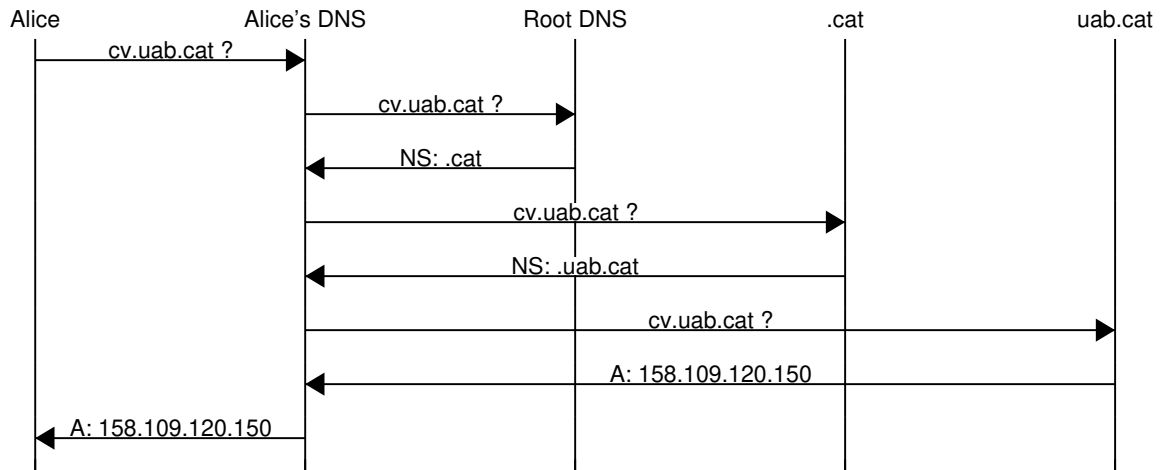
10.1.3 Resolución

La naturaleza distribuida y jerárquica de la base de datos DNS hace que a veces sea necesario intercambiar varios mensajes para satisfacer una consulta DNS.

Por ejemplo, Alice puede realizar una resolución iterativa completamente por su cuenta. Las dos primeras respuestas contienen un registro NS que apunta al siguiente servidor DNS y la dirección IP de ese servidor DNS, pero no la dirección IP del cv.uab.cat solicitado.



El proveedor de Internet (ISP) de Alice o su empresa ofrecerán probablemente servidores DNS cuya tarea es recibir las peticiones de Alice y resolverlas por ella de forma recursiva.



La combinación de la resolución recursiva y las tablas de caché hace del DNS un sistema bastante eficiente, con una disponibilidad extremadamente alta y baja latencia.

Ejercicio 10.2 Puedes usar las herramientas `host` y `dig` para hacer consultas DNS y ver los resultados. Por ejemplo, mi salida de `dig uab.cat -t ns` contiene las siguientes líneas:

```
;; QUESTION SECTION:
;uab.cat.                IN      NS
;; ANSWER SECTION:
uab.cat.                 172800 IN      NS      dns.uab.es.
;; ADDITIONAL SECTION:
dns.uab.es.             172112 IN      A        158.109.0.1
```

- ¿Qué tipo de registro solicité?
- ¿Qué dos tipos de registro recibí?
- ¿Por qué recibí dos tipos de registro?
- Realiza una resolución iterativa manual de `cv.uab.cat`. Puedes empezar con algo como `dig @202.12.27.33 cv.uab.cat` para consultar al servidor raíz M.

10.2 DHCP – Dynamic Host Configuration Protocol

Para participar plenamente en Internet, un host necesita tener correctamente configurados los siguientes aspectos:

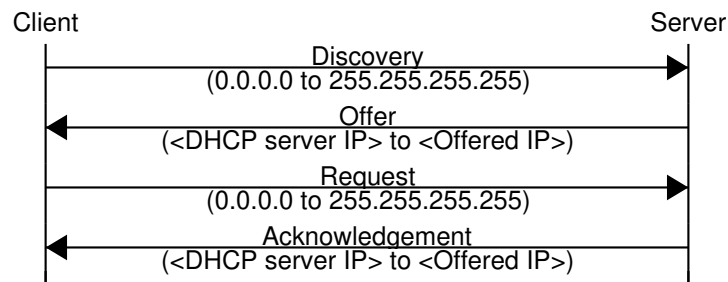
- **Dirección IP:** La dirección IP es necesaria para identificar un nodo en la red, y para que ese nodo sepa qué mensajes van dirigidos a él.
- **Máscara de red:** Los nodos necesitan conocerla para determinar cuándo entregar los datagramas directamente y cuándo indirectamente.
- **IP del gateway (pasarela):** Se necesita la dirección IP de al menos un router (encaminador) para entregar indirectamente mensajes con destino fuera de la LAN (red de área local) actual.
- **Dirección IP del servidor DNS:** Aunque no es estrictamente necesaria para usar TCP/IP, la dirección de un servidor DNS es extremadamente útil, e imprescindible para resolver nombres de dominio en direcciones IP.

Tradicionalmente, estos parámetros había que configurarlos manualmente en cada ordenador. Hoy en día, la mayoría de los ordenadores de usuario se configuran automáticamente con DHCP. Los sistemas operativos modernos incluyen un cliente DHCP que consulta periódicamente la LAN hasta quedar configurado.



El hardware de red viene con una dirección MAC por defecto preconfigurada. Además, un nodo necesita conocer su dirección MAC para utilizar el protocolo DHCP. Por tanto, las direcciones MAC no se configuran mediante DHCP.

DHCP funciona sobre UDP. Los servidores DHCP escuchan en el puerto 67, y los clientes escuchan en el puerto 68. El intercambio DHCP ideal consta de 4 mensajes: Discovery, Offer, Request, Acknowledgement.



■ Discovery

El objetivo de este mensaje es encontrar servidores DHCP dispuestos a proporcionar una configuración.

En la primera comunicación, el cliente no conoce la IP del servidor DHCP. En su lugar, usa:

- Su dirección MAC como dirección MAC de origen.
- El broadcast (difusión) de la LAN (`ff:ff:ff:ff:ff:ff`) como dirección MAC de destino.
- `0.0.0.0` como IP de origen, lo que significa "desconocida".
- La dirección de local broadcast , `255.255.255.255`, como IP de destino.

■ Offer

Cero o más servidores responderán al cliente con ofertas de configuración. Estas configuraciones (p. ej. direcciones IP) no son propiedad del cliente hasta que el servidor envía un Acknowledgement DHCP.

El servidor lleva la cuenta de un pool de direcciones que asigna dinámicamente a los clientes. Las direcciones IP se ofrecen durante un periodo de tiempo fijo, tras el cual expiran a menos que se renueven (véase más abajo). Las direcciones que no se renuevan vuelven al pool y se ofrecen de nuevo a los clientes cuando hacen peticiones.

- El servidor puede usar su dirección IP real y la ofrecida al cliente.
- El cliente sabe que es el destinatario porque el servidor se fija en la dirección MAC de origen del Discovery del cliente.

■ Request

El cliente elige una de las ofertas recibidas y pide que se complete la asignación.

- El cliente usa la dirección de broadcast local, `255.255.255.255`, como destino.
- El cliente mantiene `0.0.0.0` como origen porque la dirección ofrecida todavía no es suya.

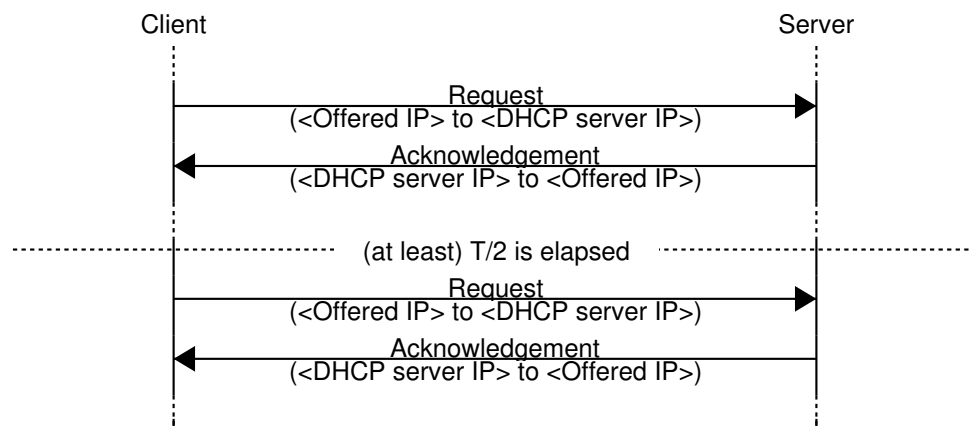
■ Acknowledgement (ACK)

El servidor confirma la petición. Una vez el cliente recibe este ACK, empieza a usar la configuración ofrecida como propia. Este ACK no debe confundirse con el de TCP.

■ Negative Acknowledgement (NACK)

También es posible que el servidor deniegue la petición, p. ej. si ha pasado demasiado tiempo desde la oferta. En este caso, se envía en su lugar un Acknowledgement negativo (NACK). Si se recibe un NACK, el cliente debe volver a la fase Discover.

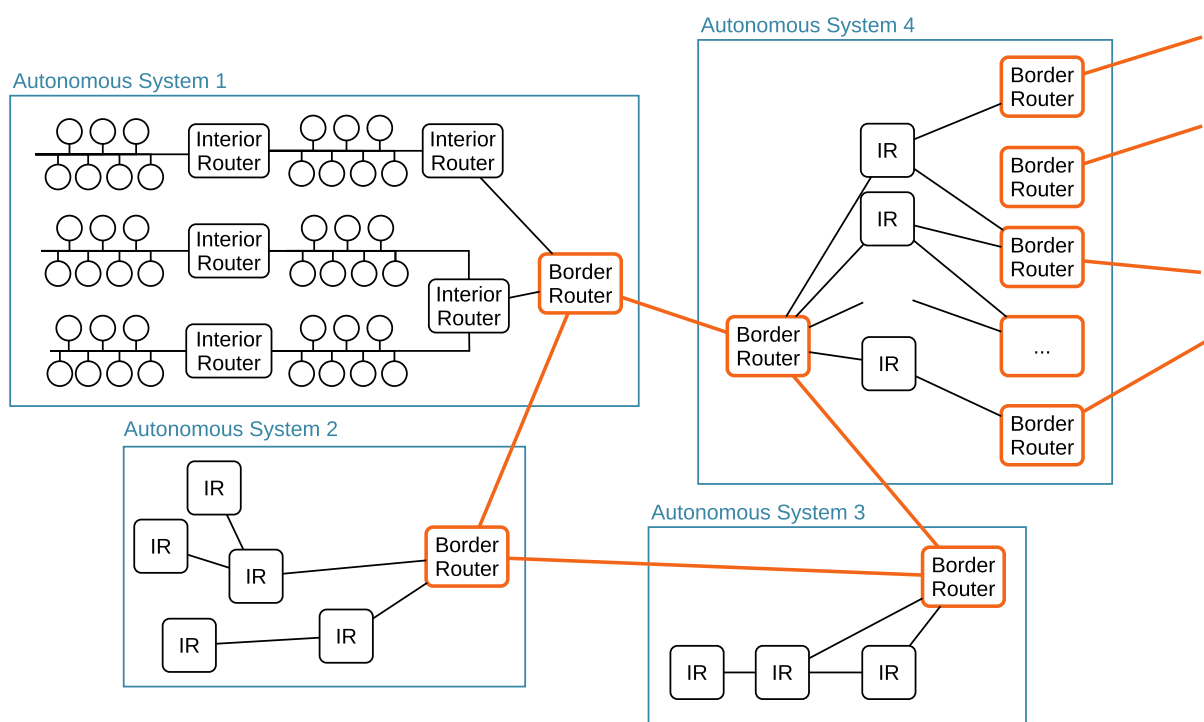
Cuando un servidor DHCP cede una dirección a un cliente, lo hace durante un periodo de tiempo limitado T . Este periodo lo fija el servidor DHCP y se puede configurar. Una vez transcurrido $T/2$, los clientes pueden intentar renovar la configuración cedida. Para ello, el ciclo Request-Acknowledgement se repite a lo largo del tiempo hasta que alguno de los dos extremos no logra completarlo a tiempo, o el servidor DHCP envía un NACK.



Ejercicio 10.3 El típico router doméstico implementa tanto un cliente DHCP como un servidor DHCP. Explica cómo es posible y por qué resulta útil.

10.3 Sistemas autónomos

El encaminamiento se realiza un salto a la vez, usando tablas de enrutamiento como se ha visto en la Sección 8.3. Esas tablas deben coordinarse a gran escala para garantizar la conectividad del grafo hasta todos los hosts de Internet.



10.3.1 Organización

Para facilitar esta coordinación, las LAN se agrupan en sistemas autónomos (AS). Estos gestionan bloques de direcciones IP no solapados ($w.x.y.z/N$) cuya propiedad y gestión son independientes. Muchas empresas, universidades e ISP tienen sus propios AS, que están listados públicamente. Por ejemplo, la red de la UAB pertenece al AS13041.

Los puntos de intercambio de Internet (IXP) son un tipo especial de AS cuyo propósito es interconectar otros AS, en lugar de alojar servicios y/o clientes. Por supuesto, hacer funcionar estos IXP no es gratis, así que se establecen relaciones comerciales con los AS interconectados. Algunos IXP se comprometen a dar servicio a cualquiera que cumpla sus requisitos básicos, no están controlados por ningún operador concreto y ofrecen a sus clientes precios transparentes y a menudo equitativos. Este tipo de IXP se denomina punto neutro (NP), en contraposición a los puntos no neutros usados, e.g., solo por un único operador de red.

Todos los AS operan routers frontera (BR) para interconectar AS. Cuando un router frontera recibe un datagrama con destino dentro del AS, lo reenvía a través de los routers internos (IR) del AS. De forma similar, cuando un host dentro de un AS necesita contactar con un host de otro AS, los datagramas se reenvían por routers frontera hasta que llegan al AS de destino y, finalmente, a la LAN de destino.

10.3.2 Configuración de la tabla de enrutamiento

Configurar manualmente las tablas de enrutamiento de todos los routers frontera y routers internos sería demasiado costoso y propenso a errores. En su lugar, se emplean distintos algoritmos para configurarlas automáticamente.

- Dentro del AS, los routers internos y los routers frontera usan uno de los protocolos de gateway interior disponibles. A menudo se usa RIP (Routing Information Protocol) por su configuración relativamente sencilla. Dicho esto, los AS pueden elegir libremente el algoritmo que usan internamente en su AS.

RIP implementa una variación del algoritmo de camino más corto de Bellman-Ford para minimizar el número de saltos que los datagramas deben recorrer dentro del AS. Periódicamente (p. ej. cada 30 s), cada IR y BR:

1. Anuncia su distancia *percibida* al resto de routers del AS.
2. Actualiza sus distancias percibidas a partir de los anuncios de otros routers.
3. Actualiza en consecuencia su tabla de enrutamiento hacia todas las LAN del AS.

Ejercicio 10.4 RIP usa el número de saltos como métrica de coste. ¿Por qué puede resultar problemático? ¿Cómo se puede resolver este problema?

- Entre AS, solo hay que configurar los routers frontera; los routers interiores simplemente necesitan dirigir el tráfico exterior a través de los routers frontera.

Los routers frontera ejecutan un protocolo de gateway exterior además del protocolo de gateway interior usado dentro del AS. Los distintos AS deben ponerse de acuerdo en el algoritmo que usan, así que normalmente se usa BGP (border gateway protocol).

BGP es un protocolo más complejo que RIP. Esta complejidad es necesaria porque BGP coordina routers frontera pertenecientes a distintos AS, es decir, con distintos propietarios:

- RIP usa un único número (la distancia, es decir, el número de saltos) para guiar un algoritmo de camino más corto en el grafo. BGP usa caminos en lugar de distancias. Esto permite a BGP elegir rutas con más saltos, pero más deseables para el AS, p. ej. en términos de tiempo, coste económico, carga interna, etc.
- BGP se diseñó teniendo en mente la seguridad y la confianza, e incluye mecanismos para verificar la autenticidad de los caminos anunciados por un router frontera.



BGP también puede usarse como protocolo de gateway interior del AS. Normalmente esto solo lo hacen AS muy grandes.

Ejercicio 10.5 ¿Cómo puede BGP traducir el interés económico en configuraciones de encaminamiento?

Misiones

La última parte de esta guía propone 9 problemas complejos llamados misiones con un aire de ciencia ficción retrofuturista. Estos problemas vertebran los contenidos de las partes anteriores, con el foco puesto en las aplicaciones y con el objetivo de alcanzar una comprensión profunda de los conceptos.

Se anima al alumnado a trabajar todas estas misiones en orden y a elaborar soluciones en forma de informes formales. También se le anima a compartirlas con su comunidad de aprendizaje inmediata (p. ej., su clase) y a intercambiar comentarios mediante cartas formales.

11	Misiones	66
11.1	Misión 1: Compartir es vivir	
11.2	Misión 2: Scroll infinito	
11.3	Misión 3: Pastoreando el caos	
11.4	Misión 4: Inter { planetary net	
11.5	Misión 5: El dinero es la raíz	
11.6	Misión 6: Molestar a dos planetas enteros	
11.7	Misión 7: TCP o no TCP	
11.8	Misión 8: ¿Me hablas a mí?	
11.9	Misión 9: Pasillo interplanetario	

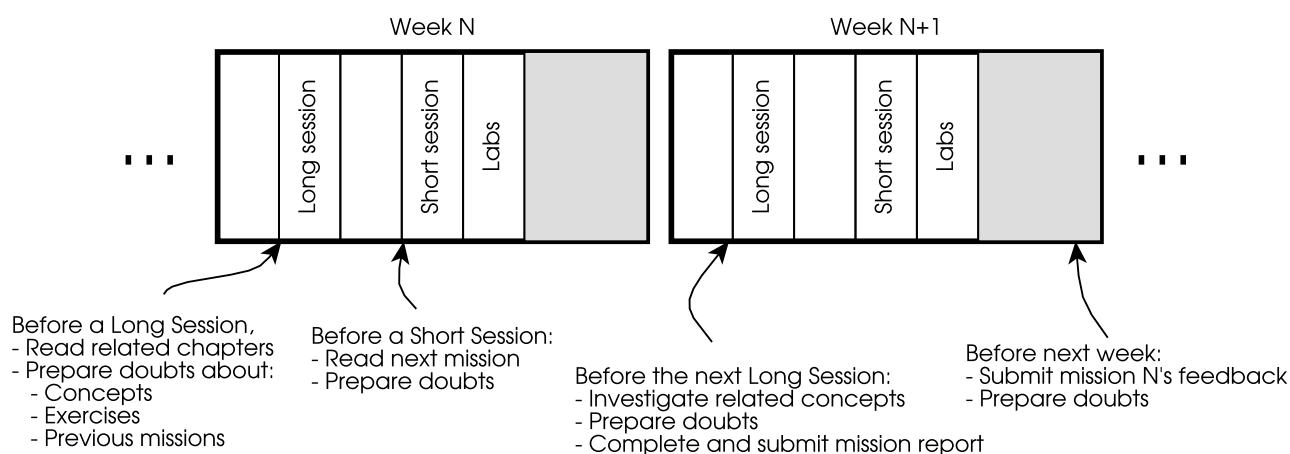
11. Misiones

Contenidos de las misiones

		Misiones								
Capítulos		M1	M2	M3	M4!	M5!	M6	M7!!	M8	M9
§2	El velo analógico-digital	✓	○							
§3	Direccionamiento y topologías de LAN	✓	○	○	○	○				○
§4	Formato de paquete y MTU	✓	✓	○	○	✓	✓	✓	○	
§5	Pilas de red y encapsulación	↔	✓	○	○	○	✓	○		
§6	Internet: visión general de TCP/IP	↔	↔	↔	✓	○	○	○	○	○
§7	Capa 2: comunicación en la LAN		✓	○	✓	✓		○	○	
§8.1	Direccionamiento de capa 3		↔	✓	✓		○			○
§8.2	Función y cabecera de IP		↔	✓	○	✓	○			
§8.3	Encaminamiento IP				✓	○		○		✓
§8.4	Subredes IP			✓	✓					✓
§8.5	Fragmentación IP		↔			✓		○		
§8.6	ARP				↔	✓		○	○	
§8.7	ICMP				↔				✓	
§9.1	Direccionamiento de capa 4						✓		✓	
§9.2	UDP							✓	○	
§9.3	TCP						↔	✓	○	
§9.4	Apertura de conexión TCP						✓	✓	○	
§9.5	Intercambio de datos TCP						↔	✓	○	
§9.6	Cierre de conexión TCP							↔	✓	
§10.1	DNS								✓	
§10.2	DHCP									✓
§10.3	ASs, IXPs, NPs									✓

✓: foco principal — ○: repaso — ↔: anticipo — !: misión crítica

Ritmo sugerido



11.1 Misión 1: Compartir es vivir



Hace unos 75 años, una expedición secreta fue enviada al espacio con el objetivo de colonizar otro planeta. La expedición llegó sana y salva a su destino, pero el aterrizaje destruyó sus instrumentos de comunicación y los dejó aislados. Tú eres un colono de tercera generación, futuro ingeniero, y esta es la primera misión que te asigna la asamblea: compartir la nueva y reluciente calculadora. **Por favor, lee todo con atención.**



Importante: Se anima a cada estudiante a publicar un **informe formal** (en PDF) en el "Foro de intercambio de la Misión 1" del Campus Virtual antes de la siguiente sesión de 2h. Después de eso, contamos contigo para publicar una **carta de respuesta formal con comentarios** (también en PDF) sobre al menos otra solución, antes de la siguiente sesión de 1h.

Detalles del informe

Ahora mismo, la colonia consta de 3 bases separadas (3 edificios). Estas bases tienen 3, 200 y 5000 dispositivos electrónicos, respectivamente. La base con 3 dispositivos se ha construido específicamente para alojar una calculadora nueva, potente y con gran consumo de energía (es uno de los 3 dispositivos). Todo el mundo en la colonia quiere tener acceso a la calculadora, pero ninguno de los dispositivos tiene capacidades de red *todavía*. Tu objetivo es proponer un diseño de red convincente que conecte y satisfaga a todo el mundo.

Prepara un breve informe con tu diseño, proponiendo soluciones claras y completas a los siguientes puntos:

1. ¿Cómo se cruza la **barrera analógico-digital** para permitir la comunicación digital? Sé concreto en las tecnologías que propongamos.
2. ¿Cómo se conectan físicamente los dispositivos? Aporta un diagrama con la **topología** exacta.
3. ¿Qué **sistema de direccionamiento** se usará para identificar de forma única cada dispositivo? Da dos ejemplos de dirección, uno de ellos el de la nueva calculadora.
4. ¿Qué **formato de paquete** emplearán los dispositivos? Aporta un diagrama de formato concreto y los bytes de un paquete de ejemplo completo.

Detalles de los comentarios

Después de publicar tu solución en el foro, busca otra y **reclámala** respondiendo con un breve mensaje "Yo daré mi opinión" (una carta de comentarios por solución).

Después prepara una carta formal en PDF y publícala en el mismo hilo cuando esté lista (puedes encontrar un ejemplo de carta de comentarios en el Campus Virtual). En esta carta, incluye tus comentarios (tanto positivos como negativos), explicando todo aquello que consideres relevante o que creas que puede ayudar a esa solución. Asegúrate de tratar los siguientes puntos:

1. ¿Ves algún fallo importante o algún punto incompleto que impediría que la propuesta se hiciera realidad?
2. Si volvieras a hacer tu informe, ¿adoptarías alguna de las propuestas de esta solución?
3. ¿Has podido interpretar el contenido del mensaje incluido en el informe? Incluye los bytes de una posible respuesta, según tu comprensión de ese informe.

11.2 Misión 2: Scroll infinito



Tras revisar la primera ronda de informes, la asamblea de la colonia está entusiasmada con la idea de explorar este nuevo concepto de *red de ordenadores*. Sin embargo, antes de invertir en ninguna de las propuestas, la asamblea pide diseños más completos y concretos. En particular, tendrás que añadir la posibilidad de enviar mensajes arbitrariamente largos, potencialmente infinitos, y prestar atención a la estructura de la pila de protocolos de red (detalles abajo).

Se espera que cada uno de vosotros publique un **informe formal** (en PDF) en el “Foro de intercambio de la Misión 2” del Campus Virtual antes de la siguiente sesión de 2h con una nueva misión programada (consulta el calendario en el Campus Virtual). Después, contamos contigo para que publiques una **carta de respuesta formal con comentarios** (también en PDF) para al menos otra solución, antes de la sesión de 1h posterior.

Detalles del informe

El nuevo informe debe ser autónomo, de manera que quien lo revise no necesite leer el primero. La propuesta que describes en este informe no tiene por qué seguir exactamente la primera: puedes tomar prestadas tantas ideas e introducir tantos cambios como consideres oportuno.

1. Resume la **capa física y la disposición topológica** de tus redes de ordenadores (tratadas en los puntos 1 y 2 de la misión anterior).
2. Indica la MTU de tu red y **por qué es necesario definir un valor de MTU finito**. Con esta restricción, explica **cómo permitirás enviar mensajes de longitud arbitraria (infinita)**.
3. Describe el **sistema de direccionamiento** (actualizado) y sus limitaciones. Puedes mantener o cambiar el sistema que propusiste en la misión anterior siempre que te asegures de que es válido.
4. Propón una **pila de protocolos de red de 3 capas** (llamémoslas L3, L2 y L1, de arriba abajo) en la que la funcionalidad de longitud arbitraria del mensaje se resuelva exclusivamente en L3, el direccionamiento exclusivamente en L2, y L1 se centre únicamente en las conversiones digital/analógica y analógica/digital.
5. Incluye un ejemplo en el que se envíe un mensaje de longitud 1,5 veces la MTU. En tu ejemplo, proporciona los **bytes de entrada y de salida de cada capa** del origen y del destino de la comunicación.

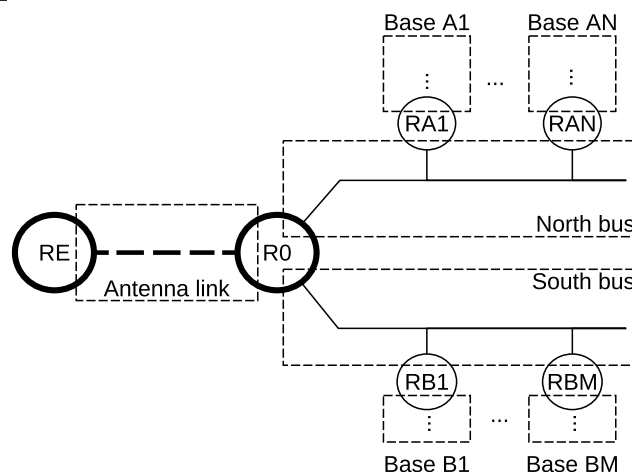
Detalles de los comentarios

Utiliza un proceso de comentarios similar al de las misiones anteriores, asegurándote de tratar:

1. ¿Ves algún fallo importante o algún punto incompleto que impediría que la propuesta se hiciera realidad?
2. ¿Existe algún límite real a la longitud del mensaje en la propuesta que estás revisando?
3. ¿Has podido interpretar el contenido del mensaje incluido en el informe? Incluye los bytes de una posible respuesta de la misma longitud, según tu comprensión de ese informe.
4. ¿Hay alguna parte de la propuesta que revisas que mejore la tuya?

11.3 Misión 3: Pastoreando el caos

Debido a diferencias políticas, y a pesar de los esfuerzos de la asamblea de la colonia, cada base del planeta adoptó una tecnología de LAN (red de área local) distinta (!). Aunque las comunicaciones son satisfactorias dentro de cada base, por el momento es imposible conectar directamente dos ordenadores cualesquiera de dos bases diferentes. Una vez más, la colonia te pide ayuda.



Gracias a los sacrificios de todos, la colonia está ahora en condiciones de construir e instalar un nuevo router (encaminador) y dos buses cableados. Estos se representan con una línea más gruesa en el diagrama de abajo. Tu objetivo a largo plazo es hacer posible la comunicación entre redes entre dos nodos cualesquiera de dos bases cualesquiera. Por ahora, diseñarás la infraestructura física y lógica.

Detalles del informe

Todas las bases emplean el protocolo IPv4, y hay varias bases conectadas tanto al bus Norte como al bus Sur. Cada base contiene una única LAN. Coordínate con los demás para que *no haya dos LANs del mismo tamaño*.

1. Dentro de tu grupo:

- Determina cuántas direcciones necesitas para que cada dispositivo pueda identificarse de forma única dentro de tu LAN. Elige el tamaño mínimo de LAN que puede satisfacer este requisito, y cuantifica el “espacio” libre disponible en tu LAN.
- Indica cuántos de los 32 bits de una dirección IPv4 necesitas para identificar un único dispositivo conectado a la LAN de tu base. ¿Qué concepto usado en IP está más relacionado con este número? ¿Cómo puede ser útil esta estructura?

Network ID	Host ID
------------	---------

2. En coordinación con los otros grupos del aula:

- Crea una LAN para el bus Norte, y describe todas las direcciones físicas y lógicas conectadas a ella.
- Asegúrate de que los bloques de direcciones IPv4 asignados a las LANs son de tamaños diferentes y no se solapan.
- Usando la pizarra o un documento compartido, enumera todas las LANs, sus tamaños y sus rangos de direcciones (actualizados).

Detalles del feedback

Utiliza un proceso de feedback similar al de las misiones anteriores, asegurándote de tratar:

1. ¿Están las LANs correctamente dimensionadas?
2. ¿Hay alguna colisión entre los bloques?
3. ¿Extrajo el informe original toda la información sobre la estructura de la dirección IP?

11.4 Misión 4: Inter { planetary net

Tras unos años de uso, la colonia por fin valora interconectar redes heterogéneas mediante el protocolo IP. Tanto, que ha invertido en una potente HugeAntenna™ que, cree, puede conectarla con la Internet de la Tierra.



La asamblea de la colonia está preocupada porque no sabe si las bases podrían comunicarse entre sí y, sobre todo, con la nueva antena. Además, teme la despótica represalia de la Tierra si no logra conectarse a Internet. De nuevo, piden tu ayuda.

⚠ ¡Atención! Esta misión es crítica. Tómate tu tiempo, pide toda la ayuda que necesites y mantén el ánimo. Las misiones previas pueden considerarse un entrenamiento para esta – se recomienda encarecidamente terminirlas con cuidado antes de abordarla.

Detalles del informe

Recuerda la topología de red pedida en la misión anterior. A estas alturas deberías tener acceso a un acuerdo consensuado sobre la distribución de direcciones físicas y lógicas de los dispositivos de la colonia (si no, ¡dilo!).

Considera los *routers* (encaminadores) de la colonia: $\{RA_i\}_{i=1}^N$ están conectados al Bus Norte, $\{RB_i\}_{i=1}^N$ al Bus Sur, y R_0 une ambos buses. La nueva antena se conectará a un tercer controlador de interfaz de red (NIC) de R_0 .

Tu objetivo es ayudar a configurar todas las máquinas de la colonia (sobre todo los routers) para que la información fluya entre bases y, crucialmente, entre la colonia y la Internet de la Tierra.

1. Resume las direcciones físicas y lógicas de todas las máquinas de la colonia para que tu informe sea autocontenido. Se sugiere reutilizar la configuración de la misión anterior tras corregir cualquier problema detectado. Usa el formato CIDR siempre que proceda.
2. Da las tablas de enrutamiento funcionales más simples que se te ocurran para estas máquinas:
 - Un router RA_i a tu elección, y una máquina no router M_i conectada a RA_i .
 - Un router RB_j a tu elección, y una máquina no router M_j conectada a RB_j .
 - El router R_0 .
3. Crea dos diagramas de carriles que muestren *todos* los paquetes transferidos dentro de la colonia en los dos intercambios siguientes. Para cada paquete de estos diagramas, indica *todas* las direcciones que contiene. Especifica en algún lugar la MTU de todas las LANs (redes de área local) implicadas en los intercambios.
 - M_i envía datos a M_j , y M_j devuelve una respuesta.
 - M_i envía un mensaje a un servidor en la Tierra, pero no recibe respuesta.

Detalles de la retroalimentación

Usa un proceso de retroalimentación similar al de las misiones previas, asegurándote de tratar:

- ¿Hay algún problema en la asignación de direcciones físicas y lógicas?
- ¿Se empleó correctamente el formato CIDR?
- ¿Contienen las tablas de enrutamiento todos los campos necesarios?
- ¿Puede simplificarse más alguna de las tablas?
- ¿Se representan bien las entregas directas e indirectas en los diagramas de carriles?
- ¿Son las MTU coherentes con los diagramas?

11.5 Misión 5: El dinero es la raíz

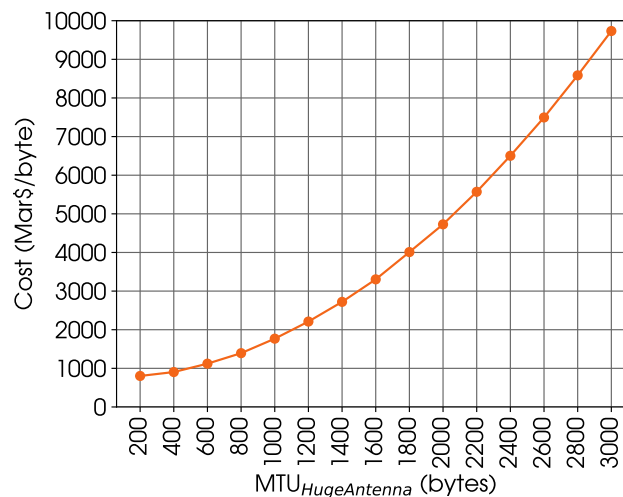
Toda la colonia bulle de energía intelectual tras instalar la HugeAntenna™ y conectarla a Internet de la Tierra – ¡piensa en todo lo bueno que se han perdido hasta ahora! Por desgracia, ahora que la colonia está enganchada a Internet, el tiránico gobierno de la Tierra ha decidido encarecer mucho el uso del enlace de la antena interplanetaria. Los precios abusivos causan graves problemas en la sociedad de la colonia, demostrando otra vez que el dinero – o, quizá, su falta – puede ser la raíz de todos los males. ¿Ayudarás a la colonia a recuperar la estabilidad financiera?



⚠ ¡Atención! Esta misión es crítica. Tómate tu tiempo, pide toda la ayuda que necesites y mantén el ánimo. Las misiones anteriores pueden considerarse un entrenamiento para esta – se recomienda encarecidamente terminirlas con cuidado antes de abordarla.

Detalles del informe

La Tierra acepta negociar el precio en Mar\$ (Dollaroos marcianos) por byte transmitido por la HugeAntenna™. El precio depende de la MTU de la antena, según la figura inferior. Observa que $MTU \in \{200, 400, \dots, 3000\}$. La asamblea necesita un análisis cuantitativo que justifique la mejor elección de MTU.



Para simplificar, considera un escenario representativo en el que un usuario de una base de la colonia envía *payloads* (carga útil) de exactamente 1900 bytes directamente sobre IP.

- Da un mapa topológico de la red de la colonia hasta el enlace entre la HugeAntenna™ y la Tierra, sin incluirlo. En esas LANs (red de área local), indica la MTU más pequeña que evita la fragmentación dentro de la colonia. Puedes reutilizar la topología de misiones anteriores o crear una nueva.
- Considera al menos 4 valores posibles para la MTU de la HugeAntenna, 2 menores que 1900 bytes y dos mayores. Para cada valor de $MTU_{HugeAntenna}$, indica:
 - Cuántos paquetes se envían por la Huge Antenna para transmitir el payload.
 - Qué campos no son idénticos en esos paquetes (solo si se envía más de un paquete).
 - El coste total de enviar el payload.

Coordínate con los demás ingenieros para hallar el valor óptimo global de $MTU_{HugeAntenna}$.

- Compara el tráfico total (en bytes) transmitido dentro de la colonia al enviar el primer payload (con todas las cachés vacías) frente al segundo (con las cachés llenas y válidas).

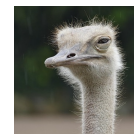
Detalles de la retroalimentación

Sigue un proceso de retroalimentación similar al de misiones anteriores, asegurándote de abordar:

- ¿Están las MTU de la colonia configuradas según lo pedido?
- ¿Se considera correctamente la fragmentación al determinar el coste?
- ¿Se contabiliza bien la sobrecarga debida a ARP al calcular el tráfico total?

11.6 Misión 6: Molestar a dos planetas enteros

Han pasado años en la colonia desde que ayudaste a configurar la gran antena que os permite comunicaros con la Tierra. Hace poco, tu colonia ha descubierto *otra* colonia secreta, esta en Saturno – ¡y la Tierra ni se molestó en decírnoslo!



La asamblea ha decidido gastar una pequeña broma y te pide ayuda de nuevo. Tu objetivo es secuestrar la próxima conexión TCP Saturno ↔ Tierra y enviar un comunicado falso haciéndote pasar por Saturno. La asamblea pide que el comunicado tenga clase.

Detalles del informe

La Ingeniera Jefe de la colonia ha hecho reconocimiento y está dispuesta a ayudarte. Ha reunido los siguientes datos:

- Tu HugeAntenna™ de la colonia se comunica con otra HugeAntenna™ de la Tierra. La Tierra usa esa misma antena para comunicarse con otra HugeAntenna™ de Saturno. Cuando la Tierra recibe el primer paquete de una conexión TCP, verifica su origen.
- Tras ese primer paquete, la Tierra no verifica el origen de ningún paquete. De hecho, los mensajes falsos o inesperados recibidos después se ignoran sin más, sin activar ninguna alarma. Con sincronización cuidadosa, usarás esto para inyectar tus paquetes falsificados.
- El primer día de cada mes, a las 4:20 a. m. en punto (hora de Nueva Barcelona, Tierra), la Tierra recibe el primer paquete de una conexión TCP de Saturno. Estas conexiones sirven para intercambiar comunicados oficiales, y la asamblea quiere que secuestres la próxima.
- Saturno usa una pila TCP/IP anticuada e insegura con este fallo: siempre fija a 0 el número de secuencia de TCP en el primer mensaje de una conexión TCP.
- No podrás ver los mensajes intercambiados entre la Tierra y Saturno. Sin embargo, la Ingeniera Jefe logró capturar la siguiente frame (trama) Ethernet, que inició la conexión TCP anterior (llamémosla C_{-1}) entre Saturno y la Tierra (00 d0 va seguido de f1 bb y 45 10 va seguido de 00 28):

```
00 09 bf 42 07 64 00 d0    f1 bb 0f 5a 08 00 45 10
00 28 d2 71 40 00 40 06    00 00 ca f7 44 05 4c 4c
15 15 9f ee 00 a9 00 00    00 00 00 00 00 00 50 02
01 00 00 00 00 00
```

Con esto en mente, la asamblea solicita un informe que contenga los siguientes aspectos:

1. Dibuja un diagrama de carriles del establecimiento de la conexión anterior (C_{-1}) entre Saturno y la Tierra. Además, tabula *todos* los campos de cabecera presentes en estos paquetes (y sus valores).
2. Explica tu plan de ataque, *i.e.*, cómo piensas secuestrar la conexión. Indica claramente:
 - Qué suposiciones haces sobre la próxima conexión (p. ej., qué campos seguirán idénticos o podrían cambiar).
 - Cuántos mensajes enviarás y su sincronización prevista.
3. Muestra la secuencia completa de bytes de una frame que contenga tu comunicado de broma.

Detalles del feedback

Emplea un proceso de feedback (retroalimentación) similar al de las misiones anteriores, abordando estos aspectos clave:

- ¿Se decodificó correctamente la frame capturada por la Ingeniera Jefe?
- ¿Lograste idear un plan que requiera menos mensajes?
- ¿Pudiste decodificar el comunicado de broma? ¿Tenía clase?

11.7 Misión 7: TCP o no TCP



Años de éxito continuado han consolidado el uso de IPv4 para interconectar redes dentro de Marte y con otros planetas. En este tiempo, los equipos de ingeniería de la colonia han descubierto las limitaciones de IPv4 por sí solo y la necesidad de usar al menos uno de los principales protocolos de transporte al comunicarse con la Internet de la Tierra.

Implementar esta cuarta capa y añadirla a las pilas de red existentes de cada nodo de la colonia es una tarea abrumadora. Por ello, en una primera fase solo se implementará uno de los dos principales protocolos de transporte. La colonia pide tu ayuda para comparar cuantitativamente ambos protocolos y decidir cuál elegir.

⚠ ¡Atención! Esta misión es muy crítica. Tómate tu tiempo, pide toda la ayuda que necesites y mantén el ánimo. Las misiones anteriores son un entrenamiento para esta — se recomienda encarecidamente terminarla con cuidado antes de abordarla.

Detalles del informe

La asamblea te pide que consideres con cuidado estos puntos en tu informe:

1. Para que el informe sea autocontenido, incluye la topología de red de la colonia en formato CIDR cuando proceda y especifica valores de MTU razonables (puedes reutilizar los de misiones anteriores). Completa la descripción indicando el ancho de banda (velocidad del enlace, en bytes por segundo – supongamos que es constante).
2. Uno de los dos protocolos de transporte candidatos usa un parámetro llamado MSS. Indica cuál y especifica el mayor valor que permitiría comunicar dos máquinas cualesquiera de la colonia sin requerir fragmentación. Usa ese valor en el resto del informe.
3. Elige un nodo N_1 de una base conectada directamente al Bus Norte y otro nodo N_2 de una base conectada directamente al Bus Sur. Para cada uno de los dos protocolos de transporte principales, calcula el tiempo total desde que N_1 empieza a transmitir hasta que N_2 recibe el primer byte de datos de payload (carga útil). Para ello, supón:
 - Si el protocolo está orientado a conexión, los datos solo se transfieren una vez establecida la conexión.
 - Todos los nodos y routers (encaminadores) tienen potencia de cálculo infinita, es decir, encapsular/desencapsular datos no introduce retardo.
 - Las velocidades de transmisión siguen siendo finitas, tal como las definiste antes en tu informe.
 - Las tablas de enrutamiento son óptimas.
 - Todas las cachés empiezan vacías para cada protocolo y sus entradas siguen siendo válidas una vez fijadas.

Te animamos a incluir diagramas de carriles para apoyar tu respuesta.

4. ¿Qué protocolos de transporte incluyen mecanismos de control de congestión? ¿Qué condiciones harían falta para una transmisión continua de N_1 a N_2 sin esperas debidas al control de congestión?
5. Indica tres casos de uso, es decir, aplicaciones de usuario: el primero adecuado solo para uno de los protocolos de transporte, el segundo adecuado para el otro y el tercero adecuado para ambos.

Detalles de la retroalimentación

Usa un proceso de retroalimentación similar al de misiones anteriores, asegurándote de tratar estos aspectos clave:

- ¿Es el MSS elegido coherente con los valores de MTU especificados?
- ¿Se calcula bien el tiempo de transmisión de cada paquete?
- ¿Ves algún problema con el encaminamiento, la fragmentación o ARP?
- ¿Se consideran adecuadamente todos los elementos del control de congestión?
- ¿Están bien elegidos los casos de uso propuestos?

11.8 Misión 8: ¿Me hablas a mí?



Tu colonia lleva un tiempo utilizando con éxito (según Saturno, también *abusando de*) la pila TCP/IP para conectarse a la Internet de la Tierra. Sin embargo, hasta ahora, en cierto sentido no existías porque no tenías un *nombre*. Eso está a punto de cambiar, porque la colonia va a instalar su primer servidor de Domain Name System (DNS), que será autoritativo para el nuevo dominio de primer nivel (TLD) `.mars`. Se necesita tu ayuda para configurar los nuevos servicios que se ofrecerán a todo el Sistema Solar.

Detalles del informe

1. Resume la topología de red de tu colonia e incluye los rangos de direcciones IP y los valores de MTU asignados a cada LAN (red de área local) usando notación CIDR. Esto hará que tu informe sea autónomo y ayudará a los revisores a darte su valoración (puedes reutilizar o actualizar la configuración de sesiones anteriores).
 2. Indica claramente qué máquina alojará tu servidor DNS y su dirección IP.
 3. Especifica 5 servicios $S_1, S_2, \dots, S_5$ (distintos de DNS) que se alojarán en tu colonia. Uno de ellos se alojará en la misma máquina que el servidor DNS. Para cada servicio, proporciona:
 - Una breve explicación del servicio y del protocolo de capa de aplicación empleado.
 - La entrada o entradas DNS relacionadas con este servicio que se incluirán en el nuevo servidor DNS.
 - Un ejemplo de consulta (*p. ej.*, `service.mars`) para la que se devolverán esas entradas, quién enviará esa consulta y cuándo.
 4. La máquina que aloja el servidor DNS aloja también otro servicio. ¿Cómo puede distinguir entre los dos tipos de tráfico?
 5. Alguien que usa el nodo N_1 de tu colonia va a implementar un script para detectar cuándo los servicios de Internet dejan de estar disponibles. Para ello, empleará mensajes ICMP periódicos para *comprobar* que están en línea. Explica cómo funcionará exactamente y, mediante diagramas de carriles, describe todo el tráfico dentro de la colonia derivado de una única *comprobación* para cada uno de los siguientes escenarios:
 - a) N_1 comprueba el servicio `cv.uab.cat` usando resolución iterativa sobre UDP empezando en el servidor raíz M.
 - b) N_1 comprueba el servicio S_1 (indica su nombre) usando resolución iterativa sobre TCP empezando en el servidor raíz M.
 - c) N_1 comprueba el servicio S_1 usando resolución recursiva sobre UDP con el servidor DNS4EU sin filtrado.
- En los diagramas, para cada mensaje intercambiado, indica:
- El tipo de mensaje (de la capa superior) y, si procede, su subtipo.
 - Todas las direcciones implicadas.
 - Toda la información relativa a DNS contenida en el mensaje.

Detalles de la valoración

Ten en cuenta requisitos y un proceso similares a los de las misiones anteriores, asegurándote de abordar los siguientes aspectos clave:

- ¿Se está usando ICMP de una forma lógica y eficaz?
- ¿Se manejan correctamente la resolución DNS iterativa y la recursiva?
- ¿Está representado correctamente el ciclo de vida de la conexión TCP?

11.9 Misión 9: Pasillo interplanetario



Gracias en parte a tu ayuda configurando la red y los servicios de la colonia, tu colonia florece con actividad cultural y científica. El pasillo principal del puerto espacial interplanetario de tu colonia recibe cada día a miles de visitantes de distintos planetas. Estos visitantes exigen conectividad a Internet rápida y sencilla desde este pasillo, y la asamblea de la colonia te ha designado para ayudar con la configuración.

Detalles del informe

La asamblea te ha pedido que configures un servidor DHCP para los dispositivos de los pasajeros. El Sistema Autónomo número 7847 te presta amablemente las 1024 direcciones más altas de su bloque, que serán las proporcionadas por este servicio.

Cada 2 horas llega al puerto espacial una nave con hasta 300 personas (cada una con un único dispositivo ávido de Internet). Algunas se marchan de inmediato mientras otras permanecen más tiempo en el puerto espacial, pero como mucho 1000 dispositivos necesitan estar conectados al mismo tiempo. Justo antes de que el puerto espacial comience a operar por la mañana, todas las direcciones gestionadas por el servidor DHCP deben estar disponibles para su asignación, porque las 5 primeras naves vienen completamente llenas.

1. Averigua quién te está prestando generosamente las 1024 direcciones IP públicas, y cuál es su bloque completo.
2. Proporciona una tabla de enrutamiento actualizada para el router (encaminador) R_0 . Este es el router conectado directamente a la HugeAntenna™ de la colonia, a los buses Norte y Sur, y ahora a la nueva LAN (red de área local) dedicada a los pasajeros. Puedes suponer que esta nueva LAN está conectada a una interfaz recién añadida.
3. ¿Cuál es el Lease Time más largo que puedes configurar que sea compatible con la avalancha matinal? Justifica tu respuesta.
4. Usando diagramas de carriles, proporciona un escenario concreto en el que el máximo de 1000 dispositivos esté conectado simultáneamente en algún momento durante la hora punta. Indica cuántos mensajes de cada subtipo DHCP intervienen.
5. Si un pasajero se escondiera en la terminal del puerto espacial durante la noche, ¿qué mecanismo usarías para asegurar que su dirección IP asignada esté disponible para su asignación a la mañana siguiente?

Detalles de la retroalimentación

Considera requisitos y un proceso similares a los de las misiones anteriores, asegurándote de tratar los siguientes aspectos clave:

- ¿Es correcta la tabla de enrutamiento, y tan simple como sea posible?
- ¿Has encontrado un Lease Time más largo que sea compatible con los requisitos?
- ¿Ves algún problema en la descripción del escenario de 1000 dispositivos concurrentes?

Términos técnicos

3-way handshake, 51, 52

A, 60

AAAA, 60

ACK, 50

acknowledgement
number, 51

address, 14

analog, 25

ARP, 43

arp cache, 44

AS, 63

ASCII, 8

authoritative, 60

bandwidth, 30

BGP, 64

big endian/little endian, 8

bind, 48, 49

bitdepth, 8

bitwise, 9

body, 17

boolean logic, 9

BR, 64

broadcast, 29, 30

buffer, 53

checksum, 38

CIDR, 34, 40

client/server, 14

CNAME, 60

connection, 25

data, 8

datagram, 20

decapsulate, 21

default routing table, 39

device, 12

DF, 42

DHCP, 61

digital, 25

direct/indirect delivery, 39

DNS, 27, 59

dns cache, 61

DNS record, 60

dns resolution, 60, 61

domain name, 59

encapsulation, 20, 21

end-to-end, 14

ethernet, 28, 30

ethertype, 30

exterior gateway protocol,
64

field (packet), 17

field (physics), 11

FIN, 50

fixed-point/floating-point,
8

flag, 18

flow control, 47

fragmentation, 42

frame, 20

full duplex, 52

gratuitous, 44

header, 17

hexadecimal, 9

hop, 38

HTTP, 48

HTTPs, 48

hub, 13

IANA, 27

ICANN, 27

ICMP, 45

IETF, 27

information, 8

interface, 38

interior gateway protocol,
64

internal router, 64

internet, 25

IP, 32, 36

IP address, 33

IP address block, 34

IPv4, 28–30

IPv6, 28, 29

ISP, 60

IXP, 63

LAN, 13, 14

layer, 20, 21

lease, 62

local broadcast, 62

MAC, 28

medium, 10

message, 8

message/stream, 19

metadata, 17

MF, 42

MSS, 54

MTU, 16

multiplex, 10

MX, 60

name zone, 60

netmask, 33, 34, 40

network, 13

network card, 12

network interface, 14

NP, 63

NS, 60

offset, 19, 42

overhead, 16, 19

packet, 16, 19

packet format, 17, 18

packet switching, 16

parallel/serial, 10

path, 18, 64

payload, 17

PDU, 20–22

physical layer, 23

physical port, 13

piggyback, 53

ping, 45

point-to-point, 12

poison, 44

pool, 62

port, 47

privative, 27

process, 14

protocol, 25, 26

public/private IP, 33

quad decimal, 33–35, 41
query, 60
reliability, 47, 51, 52
renew, 62
reply, 60
request, 60
retransmission, 47, 53
RFC, 27
RIP, 64
RIR, 27
root DNS, 59
router, 14, 15
routing table, 38, 39, 45, 63, 64
RST, 50
RTT, 53

segment, 50
sequence number, 51
sequencing, 20
service, 14
sign, 8
socket, 49–51
sshd, 48
stack, 20–23
stateful, 51
stream, 25
subnet, 40, 41
subnetting, 41
supernet, 41
swimlane diagram, 15
switch, 13–15
SYN, 50

TCP, 25
TCP/IP, 25
throughput, 17
top-level domain, 59, 60
topology, 13
transmission window, 54
transport, 47
TTL, 38

UDP, 26
unicast/multicast, 29, 30
user datagram, 47, 49

WAN, 13, 14
wi-fi, 28